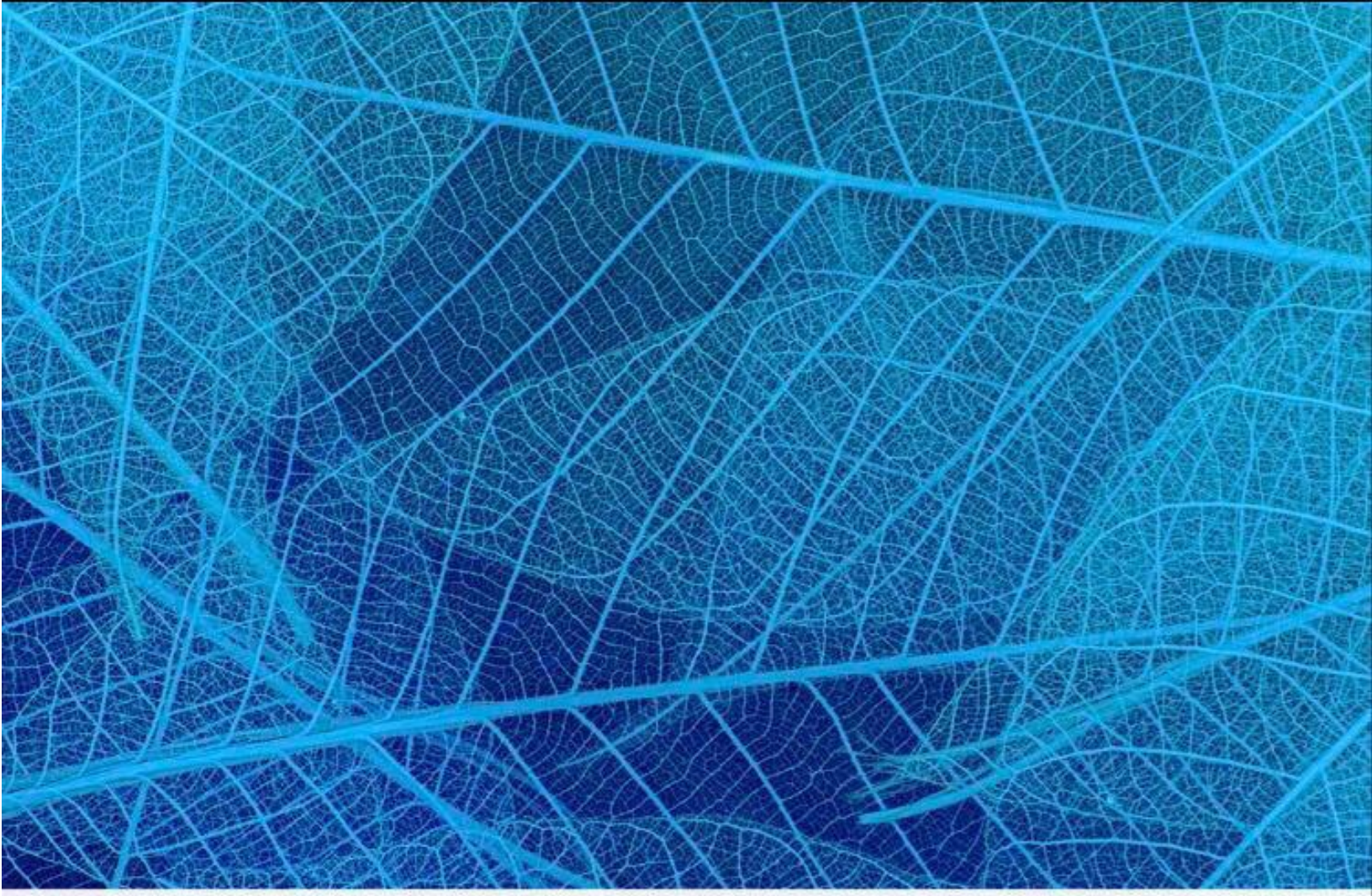


Philippine Information Technology Journal

Vol. XI No. 2
July - December 2018
ISSN 2012-0761

Published by
COMPUTING SOCIETY OF THE PHILIPPINES
http://www.csp.org.ph

Philippine Information Technology Journal

A Publication by Computing Society of the Philippines

Volume 11 • Number 2 • JULY-DECEMBER 2018 • ISSN 2012-0761

Editorial Board

Gregg Victor Gabison, *Editor in Chief*
University of San Jose- Recoletos, Cebu City
Philippines

Jaime D.L Caro, Associate Editor
University of the Philippines, Diliman
Philippines

Allan A. Sioson, *Associate Editor*
COBENA Bussiness Analytics and Strategy, Inc.
Philippines

Rachel Edita O. Roxas, Associate Editor
National University, City of Manila
Philippines

Ma. Mercedes T. Rodrigo, *Associate Editor*
Ateneo de Manila University, Quezon City
Philippines

Randy S. Gamboa, Associate Editor
University of the Southeastern Philippines, Davao
Philippines

Vladimir Y. Mariano, *Associate Editor*
RMIT University Vietnam, Ho Chi Minh City
Vietnam

Henry N. Adorna, Associate Editor
University of the Philippines, Diliman
Philippines

CALL FOR PAPERS

The Philippine Information Technology Journal (PITJ), founded in 2008, is a publication of the Computing Society of the Philippines (CSP), Inc. It appears twice yearly and publishes original peer-reviewed papers on Information Technology, Information Systems, and other applications of Computer Science.

All contributed papers must be written concisely in English and must be submitted electronically to:

The Editors, Philippines Information Technology Journal
Vidal Tan Hall, University of the Philippines Diliman
Quezon City 1101 Philippines
email: subcription@ittc.up.edu.ph

Each paper should have a maximum of fifteen pages, including an abstract of up to 200 words. Submissions may be in Microsoft Word (.doc or .docx) format, LibreOffice Writer (.odt) format, or in LaTeX. Authors submitting in Microsoft Word or LibreOffice Writer format are also required to submit the same paper in PDF. Authors submitting in LaTeX are required to submit all other auxiliary files such as graphic files and LaTeX cls and sty files. Authors must typeset their papers using the ACM templates available at http://www.acm.org/sigs/pubs/proceed/template.html

All references must be listed alphabetically with respect to the first author's last name and cited in the text by square-bracketed numbers. When applicable, each reference item should include the name of the authors, title of the article, journal name, volume, number, month, year, publisher, and pages.

Papers that have been published or submitted to other journals or books will not be considered for publication in the PITJ. Revised or extended versions of papers published in conference, workshop, or symposium proceedings may be submitted to PITJ with proper acknowledgement and cover note. The PITJ owns the copyright to the technical contributions it published.

SUBSCRIPTION

The annual subscription price is PHP 1,000.00 or USD 40.00 (excluding postage and freight charges). All subscription correspondences should be addressed to: subcription@ittc.up.edu.ph

Foreword

I am writing this Foreword for Volume 11 No. 2 of the Philippine Information Technology Journal (PITJ). For this release, PITJ presents selected papers based on the program committee score reviews in the 1st Computing Society of the Philippines (CSP) Special Interest Group for Information and Computing Education Conference of 2018. This conference was held last October 4-6, 2018, in Cebu Institute of Technology-University (CITU), Cebu City, Philippines.

The three(3) day conference serves as a relevant platform for information and computing professionals, teachers, graduate school and undergraduate students, and researchers to interact and share their expertise and research work in computing, computer science, computer engineering, computational science, information science, information technology, and computing education.

PITJ will indeed serve as an excellent reference for other computing researchers and scientists all around the globe. I also believe that this will catalyze further study and research in the areas mentioned above.

Congratulations to all authors. We appreciate your dedication and efforts.

GREGG VICTOR D. GABISON
Editor in Chief

Philippine Information Technology Journal

A Publication by Computing Society of the Philippines

Volume 11 • Number 2 • JULY-DECEMBER 2018 • ISSN 2012-0761

CONTENTS

A Mobile Augmented Reality Application With User-defined Virtual Elements **4**

by Melquizedek Borbon and Ma. Mercedes Rodrigo

A Review on Process-Oriented Approaches for Analyzing Novice Solutions to Programming Problems **10**

by Maureen M. Villamor

Automatic Recognition of Music Emotion on the Lyrics using Multinomial Naïve Bayes **23**

by Courtney Love A. Peñaflorida, Leah Joy L. Ylaya, Eric V. Magto, and Gregg Victor D. Gabison

DAGIT: An All-in-One Directory, Navigation, Traffic, and Parking Advisory Mobile Application for Dumaguete City **27**

by Joy M. Dy, Justin Carlos A. Malana, Alyssa L. Palencia, Clarice Pocot, and Dave E. Marcial

Portable Java-based Client/Server Application of Classical Item Analysis Theory **33**

by Arjay F. Abio

Wheel-spinning in a Novice Programming Context within a Meaningful Gamification System **42**

by Rejeenald M. Flores, Jenilyn L. Agapito, and Ma. Mercedes T. Rodrigo

Philippine Information Technology Journal, Vol. 11, No. 2, July - December 2018

A Mobile Augmented Reality Application With User-defined Virtual Elements

Melquizedek Borbon
Ateneo de Manila University
Quezon City, Philippines
milkyborbon30@gmail.com

Ma. Mercedes Rodrigo
Ateneo de Manila University
Quezon City, Philippines
mrodrigo@ateneo.edu

ABSTRACT

More than a few mobile augmented reality (AR) applications are rapidly growing, especially for educational purposes. However, most of these applications have system-generated virtual objects. As a result, users will have limitations to those virtual objects. This study presents CUOVO, a mobile AR application where the users could create their own two-dimensional (2D)virtual objects using their sketches or from an image and superimpose it into the physical world. This way, the users will have the freedom of customizing their virtual objects based on their imagination. The users could create one or more virtual objects, which also measures their creativity. Furthermore, users could also interact with their created virtual objects through placing it in any location as longs as it is within the bounds of the system-generatedsurface, and moving it in any direction and change its sizes.

KEYWORDS

Augmented/Mixed Reality, Mobile Application, Image Processing, Mobile Devices

ACM Reference format:

Melquizedek Borbon and Ma. Mercedes Rodrigo. 2018. A Mobile Augmented Reality Application with User-defined Virtual Elements. *In Proceedings of Information and Computing Education Conference (ICE 2018). Cebu City, Philippines, 6 pages.*

1 Introduction

The Augmented Reality (AR) is a real-time combination of multimedia elements, such as virtual 2-Dimensional (2D) objects, 3-Dimensional (3D) models, and audio and video clips, into the real environment [29]. This creates an illusion of virtual elements seamlessly merged with the real world. AR is one of the leading modern advances in HCI technology and is widely used by a variety of application domains [16]. Medicine, engineering, entertainment, and education are some examples of the domains. However, most of the studies that developed a system with AR technology have fixed virtual contents, where the model of these virtual elements was already predefined on the system, and the positions of these virtual elementsare attached to the physical object. Because of this, the creativity of the users will be limited in a way that they cannot create their AR contents and change their position to the real world according to the users' preference.

This paper presents CUOVO, a type of AR application where the users can create their virtual objects through sketching or capturing an object from an existing image. In this way, users will have the freedom to customize their virtual objects basedon their visual imagination. Studies have shown that using mental imagery as a strategy in teaching can enhance the learning of the users [13, 14, and 15]. In the context of the proposed application, mental imagery occurs when users create virtual objects to overlay the real world based purely on their visual imagination. Furthermore, the users could also control their created virtual content, such as its size and location in the realworld.

2 Review of Related Literature

This section discusses the existing literature that reviewed different AR-related studies on the development of technology. It also discusses related studies that developed interactive AR applications applied in education and the limitations of existing user-based AR applications.

2.1 Development of AR Technologies

AR refers to the real-time integration of multimedia elements into the real environment [29], implemented in different ways by many researchers in various fields such as medicine, engineering, entertainment, and education to visualize virtual content in the real world. AR had already proven to have many potential uses with positive effects. According to Lim and Park [7], where they analyzed 13 literature of developed AR books to identify the impacts on education, they have found out that

AR-integrated books improved reading comprehension, memory retention, attentiveness, interactivity, imagination, and problem-solving. These AR books use the technology of overlaying a visual representation of the words and text found in the book's content. In terms of how the AR technology will be delivered to the users, the study of Akçayır, M., & Akçayır, G. [19] reviewed 68 literature of AR studies for education and found out that most of the AR applications were built for mobile devices. Henrysson, Billinghurst & Ollila [2] stated that mobile devices are an ideal platform for AR applications. However, some AR applications carried out the research and development with a technological approach and a little user-based experimentation. Swan & Gabbard [13] conducted a study that surveyed and categorized the user-based studies of AR applications. Out of 266 AR-related articles published from 1992-2004, only 38 articles had HCI characteristics, and only 21 articles defined a formal user-based study. Following this study with a different method, Dunser, Grasser & Billinghurst [3] published survey literature of AR researches with techniques of user-evaluation published between the years of 1993 and 2007. They have reviewed 557 AR publications, but only 161 have conducted a user evaluation. Furthermore, these publications include those who only have informal user evaluations and informalcollection of user feedback. Their studies only showed a need to develop further AR applications with the implementation of proper user-based experimentation.

2.2 Existing AR Applications in Education

AR had been extensively used in the area of education. One of the ways in implementation is by integrating the technology into physical objects such as traditional books to enable interactive media in education. This kind of AR application is called an AR book. “MagicBook”, an AR-integrated book developed by Billinghurst, Kato, and Poupyrev [17], explored how physical objects can be used to bring users in between reality and virtuality seamlessly. Their application allows the users to see 3D objects virtually displayed out of the book's pages using a handheld AR display (HHD). Taketa et al. [21] made a virtual pop-up book where the users can see 3D objects on the pictures of the book as they read it using a head-mounted display (HMD). Clark & Dunser [1] proposed a novel interactive AR experience where the users can give colors to the pages of the book, and their application directly plots the colored page into the virtual 3D models, thus showing the texture of the 3D object to be the same as the color of the page. Another AR book designed specifically on the field of Science is the ARBOOK, developed by Ferrer-Torregrosa et. al. [14]. Their application focused on giving information about the anatomy of the lower limb as the text and image content of the book. Since then, several developed free and commercial applications were developed on this kind of AR books, such as QuiverVision [25] and Chromville [8].

Several studies have also applied AR technology in enhancing the learning and experience of storytelling in children’s storybooks. Saso, Iguchi & Inakage [30] developed a pop-up book with AR technology with aural integration of dialogue and narratives. Grasset et al. [26] created a similar system with an additional interaction that uses tangible objects, such as a paddle attached to the book, to control the virtual objects. Rohaya et al. [11] designed an interactive AR children playbook that teaches preschool students in learning numbers. Their study was developed further to bring it to a mobile-based environment with an added interaction that uses the users' fingers to interact with the virtual objects [5]. Vate-U-Lan [24] created a AR 3D pop-up book for teaching the English language in grade 3 students based on a storytelling technique.

However, most of the applications mentioned required a desktop computer, and users would have to use either an HHD or an HMD to experience the AR contents. Implementing the AR technology in desktop computers is not recommended due to its hardware limitations [31]. Wagner & Schmalstieg [9] have reported that using these devices affects the dexterity of the users, offers impractical use, and unacceptable to the general users. In addition, the virtual 3D objects were already generated on the application, thus users could not create their virtual contents.

2.3 User-based AR Systems

One way of allowing the users to create their AR contents is through sketch-based systems where they can draw or sketch their desired objects to display in the real world. Users can sketch objects using traditional pen and paper or on a tablet PC and show them as AR virtual objects. The following studies implemented this method. Hagbi et al. [20] developed a cooperative in-place sketching game called Sketchaser. The game was based on the capture the flag concept, and the users can create their design of the game arena through sketching. De Lima et al. [12] also explored combining the visualization of AR interface into an interactive sketch-based interface where they presented a narrative system that dramatizes interactive stories in AR on a sheet of paper and allows user interaction with the virtual objects through drawing. These systems recognize drawings of users and display them as AR virtual objects. However, the caveat in these systems is that the sketches are only limited to symbols and shapes, which the system will recognize to generate a virtual object, and the system already generated these virtual objects. Thus, it does not allow the users to design their AR content. On the other hand, some systems have this feature. Xin, Sharlin & Sousa [18] developeda handheld mixed reality 3D sketching interface, called the Napkin Sketch, which permits the users to produce their 3D sketches. The users can draw on the tablet PC as the sketching platform, and thesystem displays their drawings on top of physical paper. Bergig,El-Sana & Billinghurst [23] developed a system that analyzes 2D sketches of a mechanical system drawn by the users and generates a corresponding 3D object as the overlaid AR content. However, users’ drawings were restricted to basic trihedral orthographic models, and their system setup requires a desktop computer equipped with a web camera. The [18] and [23] focus mainly on developing the application as a design tool. Feng, Yang & Xiao [15] developed the “MagicToon,” a sketch-based AR mobile system that allows the

users to create 3D textured objects directly from their 2D drawings and display themas virtual objects in an AR scene. Their system offers many interactions where the user can perform operations such as rotatingand scaling the size, duplicate, and add animations to the 3D objects. Still, MagicToon had limitations in that they stated that the 3D generation cannot create sharp objects like cubes andthat the 2D drawings were used as textures for both the back and front faces of the generated 3D object. In addition, the users would haveto make the animations of the 3D virtual objects manually. Their application also uses marker-based AR in which the sketches serveas the marker. Thus, the virtual content will only show once the camera detected the drawings. This limits the positioning of the virtual content because the camera must track the sketches.

In contrast to the existing studies, this research will use the exact 2D sketches of the users. This will give the users the freedom to create their AR contents and bring the metaphor of giving life to the users' drawings. This system will also provide animations for the sketches automatically.

3 Methodology

This research aims to create CUOVO, an AR application that allows users to create their virtual elements. The core feature of the proposed AR application can superimpose the user-created elements, such as in an image ora sketch, into the real world. This feature will have two methods. First, the mobile device's camera will capture the image or sketch and the system will analyze it to acquire the desired object using the OpenCV [22], an open-source computer software library containing many algorithms for image processing such as photo manipulation identifying objects and detecting patterns. The second method will be the overlaying of the acquired object into the real world using the ARCore [4], a platform for developing AR applications on smartphones. The ARCore uses marker-less tracking, thus users will be able to place their created AR object anywhere they want. These features will then be combined into a single application using Unity3D [32], a platform for building 2D, 3D, Virtual Reality (VR), and AR apps. They will be made for mobile devices to provide flexibility and portability. Using the Lean Touch [6], a Unity3D asset that handles gestures (e.g., Drag, Pinch) and touch inputs, the users can interact with the virtual objects. The development of the proposed application also addresses some limitations of the features of existing AR applications listed in table 1. These limitations are as follows. Predefined AR elements, in which the AR elements are generated only by the system. Marker-based AR, where the system uses some images as markers that serve as the anchor of the AR elements. Desktop compatibility, where the application can only be used in a desktop setup. User-created AR elements, where systems allow the user creation of AR elements. Marker-less AR, where the system does not use images as markers. Lastly, mobile device compatibility, where the application can also be used in a mobile device.

Table 1: Features of Existing AR Applications

	Predefined AR elements	Marker-based AR	Desktop Compatibility	User-created AR elements	Marker-less AR	Mobile Device Compatibility
Billinghurst, Kato, & Poupyrev [17]	✓	✓	✓			
Taketa et al. [21]	✓	✓	✓			
Clark & Dunser [1]	✓	✓	✓			
Ferrer-Torregrosa et al. [14]	✓	✓	✓			
Saso, Iguchi & Inakage [30]	✓	✓	✓			
Grasset et al. [26]	✓	✓	✓			
Tomi & Rambli [5]	✓	✓	✓			✓
Vate-U-Lan [24]	✓	✓	✓		✓	
Hagbi et al. [20]	✓	✓	✓			
De Lima et al. [12]	✓	✓	✓			
Xin, Sharlin & Sousa [18]	✓	✓	✓	✓		✓
Bergig, Hagbi, El-Sana & Billinghurst [23]	✓	✓	✓	✓		
Feng, Yang & Xiao [15]		✓	✓	✓		✓

4 Results

The development of CUOVO resulted in the following features as listed in table 2. It superimposes AR elements created purely by the users, thus it does not have predefined AR elements.This has been made possible by extracting the desired object from the main image captured by the camera with the help of some built-in

functions from the OpenCV library, overlaying the extracted image into the real world using the ARCore platform and compiling both features into a mobile application using the Unity3D.

Table 2: Current features of CUOVO

	Predefined AR elements	Marker-based AR	Desktop Compatibility	User-created AR elements	Marker-less AR	Mobile Device Compatibility
CUOVO				✓	✓	✓

4.1 Extraction of object

The process of extracting the object first captures an image, eithera drawn sketch or a photo, as long as the desired object is within the camera's bounds to acquire the desired result. The system then automatically extracts the desired object from the image taken using a marker-based image segmentation through a watershed algorithm to identify the regions of the possible desired object from the image and find the largest contour area, which determines the desired object. The segmentation process was taken from [27]. The system reads the original image (Figure 1a) and converts it into a grayscale image (Figure 1b) to apply thresholding. Thresholding is used to create a binary image or simply a black and white image. After getting the binary image (Figure 1c), it is applied with morphological operations: erosion and dilation, to eliminate noisesand smaller objects and determine the regions or parts of the image filled in the filling process. Combining the resultof erosion and dilation, the marker image is acquired (Figure 1d). Then, the watershed algorithm is applied to find the enclosed parts of the desired object (Figure 1e). The system then finds the largestcontour area based on the result after using the segmentation process to identify the desired object and create a mask tothat area (Figure 1f). Lastly, the object is extracted from the originalimage using the mask (Figure 1g).

4.2 Overlaying the extracted object

The system then makes the extracted objects into virtual elements and brings them to the AR environment using the ARCore platform. The ARCore detects horizontal or vertical surfaces called planes (Figure 2a) and displays it on the image captured by the smartphone's camera. This is done by detecting and searching for clusters of distinct feature points that appear on common surfaces. The ARCore library provides built-in functions that are used to display the planes.

However, the ARCore cannot correctly detect the horizontal surfaces without texture, such as a floor with white color. To make it closer to reality, the system allows the users to place their virtual objects only on the textured horizontal flat surfaces, making it appear as if their created objects are lying on the floor or a table (Figure 2b). The system supports marker-less tracking to place their virtual objects anywhere freely they want as long as within the bounds of the detected plane.

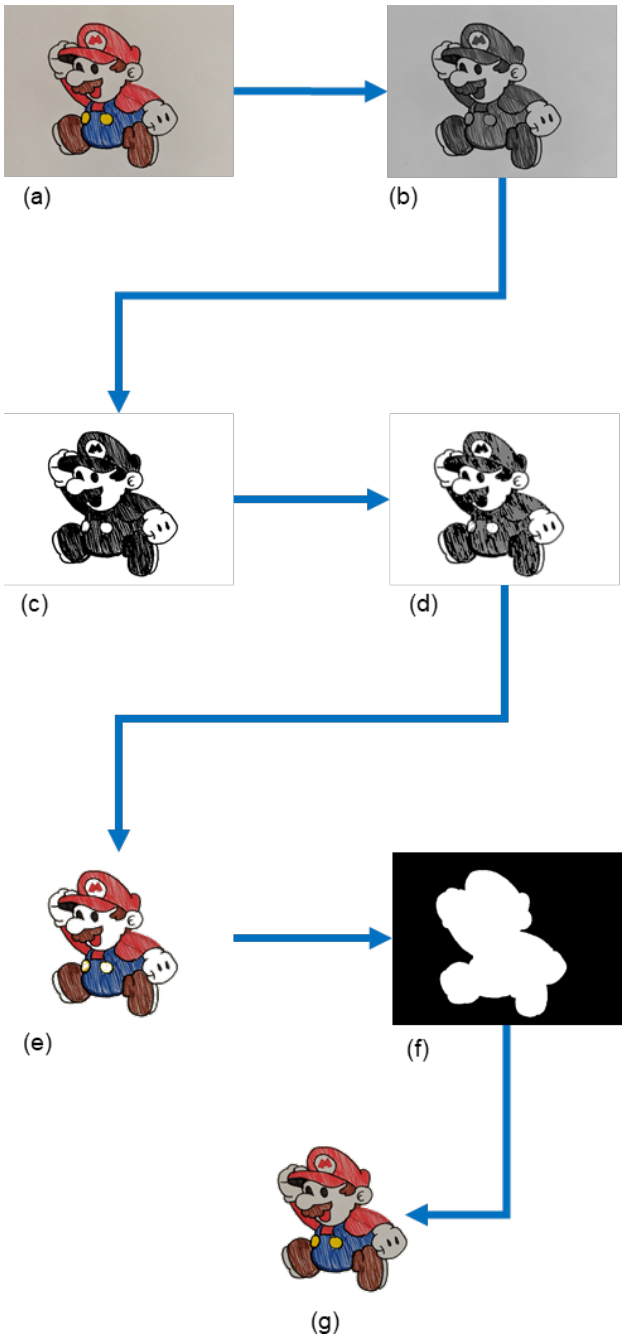


Figure 1: Process of object extraction. (a) The original image. (b)The grayscale image. (c) The binary image. (d) The markers after combining the result of erosion and dilation. (e) The result of watershed segmentation. (f) The largest contour area creates the mask. (g) The extracted image.



(a) (b)

Figure 2: (a) The horizontal surface detected by the system using ARCore and (b) the virtual object placed above it.

To bring the extracted image into the AR environment, the system first loads the image and converts it into a sprite image. This is because sprites support the transparency of an image, and the original image will be compressed into a smaller size. When the user taps on the screen, the system initializes the sprites and displays them on the screen. The default location of the virtual object is based on the position of the touch.

4.3 Virtual object interaction

The system supports selecting one or more virtual objects to move it in any direction by dragging in the smartphone screen, changing its sizes by pinching the screen inwards to make it bigger or pinching outwards to make it smaller. This was done using some of the functions of LeanTouch. The LeanTouch, LeanSelect, and LeanFingerTap classes were imported on the application to activate the handling of touch gestures. Then, the LeanSelectable, LeanScale, and LeanTranslate classes were attached to the virtual object prefab to make the object selectable, change its size, and move it in any direction, respectively.

4.4 Feedback from stakeholders

The CUOVO AR application was presented to some departments and units of Ateneo de Manila University (ADMU), namely Ateneo Art Gallery, Ateneo Grade School, and Fine Arts Department, to gain insights on the possible use-cases for the implementation of the application. The stakeholders had suggested different possible use-cases listed in table 3.

4.5 Limitations

Based on the developed AR application, it still has limitations. First, the surroundings might affect the robustness of the object extraction, such as when there are shadows and reflections on the sketch or image, thus the system includes it in the extraction process. This issue can be solved by making sure that the image to be extracted must have no shadows and reflections. Second, the object extraction process also fails when the image is too complex, such as when the object to be extracted is from a painting, thus the system does not provide the desired object. Finally, the application cannot extract properly the object from an image that is not enclosed. To solve this problem, the image must be re-outlined to make it enclosed.

Table 3 Possible use-cases suggested by the stakeholders

Department/ Unit	Suggested Use-case Application	Description
Ateneo Art Gallery	Virtual Sketch	Users can virtually place their drawings on the paintings and sculptures.
	Collage with Scavenger hunt-like Application	Users will have to look for specific shapes found on the portion of paintings and combine them like a collage.
Ateneo Grade School	History Class Application	Users can draw their ideas of a historical place and overlay it in the real world.
	Storytelling Application	Users can draw the characters of a story and use them as AR elements
Fine Arts Department	Tableau game-like Application	Tableau is a game where players will pose for any actions that represent a scenario [10]. In the AR application, Users can draw any objects and place them virtually on the other players to guess the scenario.
	Mad Libs game-like Application	Mad Libs is an interactive story game where a player asks the other players to insert a specific type of word to fill in the blanks in the story [28]. Within an AR application, users will have to draw the words instead of writing them.

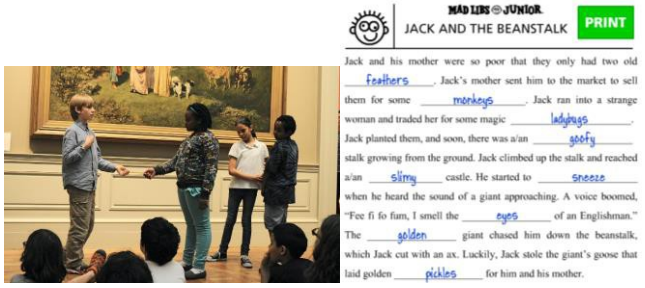


Figure 3: These images show the example of the games (left) Tableau and (right) Mad Libs. Images downloaded from www.artmuseumteaching.com/2012/12/06/tableaux-vivant-history-and-practice/ and www.thedigitalscoop.com/the_digital_scoop/2011/06/ma d-libs-junior.html respectively in September 2018

5 Conclusion and Future Work

This research presents CUOVO, a mobile augmented reality application with user-defined virtual elements. Using this AR application, the users can create their virtual elements either by their sketches or taken from an existing image and overlaythem in the real world. Additionally, the users can create multiple virtual objects, display them simultaneously, and interact with them by moving them to a different location or changing their sizes.

To address the current limitations of the application, we need to explore better image segmentation techniques and have a feature that would help minimize the identified issues. We have selected the Collage with Scavenger hunt-like application suggested by the Ateneo Art Gallery as the use case, and we are currently improving the CUOVO to fit the specific application. We are also planning to conduct a user and expert evaluation to measure the usability and determine effectiveness in visual literacy.

ACKNOWLEDGMENTS

We would like to express our gratitude to the Commission on Higher Education (CHED) for granting a scholarship to the student researcher to make this study happen. We thank the people from Ateneo Grade School, Ateneo Art Gallery, and the Fine Arts Department, who have been the stakeholders for being a part of the development of this study. We would also like to express thanks to Areté, Ateneo Laboratory for Learning Sciences (ALLS), and the Department of Information Systems and Computer Science (DISCS) for their excellent support and cooperation for this study.

REFERENCES

[1] Adrian Clark, Andreas Dünser, and Raphaël Grasset. 2011. An interactive augmented reality coloring book. 2011 10th IEEE International Symposium on Mixed and Augmented Reality (2011). DOI:http://dx.doi.org/10.1109/ismar.2011.6143487.

[2] Anders Henrysson, Mark Billinghurst, and Mark Ollila. 2005. Face-to-face collaborative AR on mobile phones. Fourth IEEE and ACM International Symposium on Mixed and Augmented Reality (ISMAR05) (2005). DOI:http://dx.doi.org/10.1109/ismar.2005.32.

[3] Andreas Dünser, Raphaël Grasset, and Mark Billinghurst. 2008. A survey of evaluation techniques used in augmented reality studies. ACM SIGGRAPH ASIA 2008 courses on - SIGGRAPH Asia 08 (2008). DOI:http://dx.doi.org/10.1145/1508044.1508049.

[4] ARCore. ARCore – Google Developer | ARCore | Google Developers. Retrieved from https://developers.google.com/ar/.

[5] Azfar Bin Tomi and Dayang Rohaya Awang Rambli. 2013. An Interactive Mobile Augmented Reality Magical Playbook: Learning Number with the Thirsty Crow. Procedia Computer Science 25 (2013), 123–130. DOI:http://dx.doi.org/10.1016/j.procs.2013.11.015.

[6] Carlos Wilkes. 2015. Lean Touch. Retrieved September 10, 2018, from https://assetstore.unity.com/packages/tools/input-management/lean-touch-30111.

[7] Cheolil Lim and Taejung Park. 2011. Exploring the educational use of an augmented reality books. In Proceedings of the Annual Convention of the Association for Educational Communications and Technology (pp. 172-182).

[8] Chromville. Chromville. https://chromville.com/.

[9] Daniel Wagner and Dieter Schmalstieg. First steps towards handheld augmented reality. Seventh IEEE International Symposium on Wearable Computers, 2003. Proceedings. DOI:http://dx.doi.org/10.1109/iswc.2003.1241402.

[10] David Farmer. 2014. Tableaux. Retrieved September 10, 2018, from https://dramaresource.com/tableaux/.

[11] Dayang Rohaya, Awang Rambli, Wannisa Matcha, Suziah Sulaiman, and Mohd Yunus Nayan. 2012. Design and Development of an Interactive Augmented Reality Edutainment Storybook for Preschool. IERI Procedia 2 (2012), 802–807. DOI:http://dx.doi.org/10.1016/j.ieri.2012.06.174.

[12] Edirlei Soares De Lima, Bruno Feijó, Simone D.j. Barbosa, Antonio L. Furtado, Angelo E.m. Ciarlini, and Cesar T. Pozzer. 2014. Draw your own story: Paper and pencil interactive storytelling. Entertainment Computing 5, 1 (2014), 33–41. DOI:http://dx.doi.org/10.1016/j.entcom.2013.06.004.

[13] J. Edward Swan, and Joseph Gabbard. 2005. Survey of user-based experimentation in augmented reality. In Proceedings of 1st International Conference on Virtual Reality (pp. 1-9).

[14] J. Ferrer-Torregrosa, J. Torralba, M.A. Jimenez, S. García, and J.M. Barcia. 2014. ARBOOK: Development and Assessment of a Tool Based on Augmented Reality for Anatomy. Journal of Science Education and Technology 24, 1 (2014), 119–124. DOI:http://dx.doi.org/10.1007/s10956-014-9526-4.

[15] Lele Feng, Xubo Yang, and Shuangjiu Xiao. 2017. MagicToon: A 2D-to-3D creative cartoon modeling system with mobile AR. 2017 IEEE Virtual Reality (VR) (2017). DOI:http://dx.doi.org/10.1109/vr.2017.7892247.

[16] Mark Billinghurst, Adrian Clark, and Gun Lee. 2014. A Survey of Augmented Reality. Foundations and Trends® in Human-Computer Interaction 8, 2-3 (2014), 73–272. DOI:http://dx.doi.org/10.1561/1100000049.

[17] Mark Billinghurst, Hirokazu Kato, and Ivan Poupyrev. 2001. MagicBook: transitioning between reality and virtuality. In CHI '01 Extended Abstracts on Human Factors in Computing Systems (CHI EA '01). ACM, New York, NY, USA, 25-26. DOI: https://doi.org/10.1145/634067.634087.

[18] Min Xin, Ehud Sharlin, and Mario Costa Sousa. 2008. Napkin sketch. Proceedings of the 2008 ACM symposium on Virtual reality software and technology - VRST 08 (2008). DOI:http://dx.doi.org/10.1145/1450579.1450627.

[19] Murat Akçayır and Gökçe Akçayır. 2017. Advantages and challenges associated with augmented reality for education: A systematic review of the literature. Educational Research Review, 20 (2017), 1–11. DOI:http://dx.doi.org/10.1016/j.edurev.2016.11.002.

[20] Nate Hagbi, Raphael Grasset, Oriel Bergig, Mark Billinghurst, and Jihad El-Sana. 2010. In-Place Sketching for content authoring in Augmented Reality games. 2010 IEEE Virtual Reality Conference (VR) (2010). DOI:http://dx.doi.org/10.1109/vr.2010.5444806.

[21] Nobuko Taketa, Kenichi Hayashi, Hirokazu Kato, and Shogo Noshida. Virtual Pop-Up Book Based on Augmented Reality. Lecture Notes in Computer Science Human Interface and the Management of Information. Interacting in Information Environments, 475–484. DOI:http://dx.doi.org/10.1007/978-3-540-73354-6_52.

[22] OpenCV. OpenCV Library. Retrieved from https://opencv.org/.

[23] Oriel Bergig, Nate Hagbi, Jihad El-Sana, and Mark Billinghurst. 2009. In-place 3D sketching for authoring and augmenting mechanical systems. 2009 8th IEEE International Symposium on Mixed and Augmented Reality (2009). DOI:http://dx.doi.org/10.1109/ismar.2009.5336490.

[24] Poonsri Vate-U-Lan. 2012. An Augmented Reality 3D Pop-Up Book: The Development of a Multimedia Project for English Language Teaching. 2012 IEEE International Conference on Multimedia and Expo (2012). DOI:http://dx.doi.org/10.1109/icme.2012.79.

[25] QuiverVision. Quiver 3D Augmented Reality Coloring Apps. Retrieved from http://www.quivervision.com/.

[26] Raphaël Grasset, Andreas Dünser, and Mark Billinghurst. 2008. Edutainment with a mixed reality book. Proceedings of the 2008 International Conference in Advances on Computer Entertainment Technology - ACE 08 (2008). DOI:http://dx.doi.org/10.1145/1501750.1501819.

[27] Robert Laganière. 2017. OpenCV 3 computer vision application programming cookbook: recipes to help you build computer vision applications that make the most of the popular C library OpenCV 3, Birmingham, UK: Packt Publishing Ltd.

[28] Roger Price and Leonard Stern. 2017. The Original# 1 Mad Libs. Penguin.

[29] Ronald Azuma. 1997. A survey of augmented reality. In Presence: Teleoperators and Virtual Environments, 6 (4), 355–385.

[30] Tomoki Saso, Kenji Iguchi, and Masa Inakage. 2003. Little red. Proceedings of the SIGGRAPH 2003 conference on Sketches & applications in conjunction with the 30th annual conference on Computer graphics and interactive techniques - GRAPH 03 (2003). DOI:http://dx.doi.org/10.1145/965572.965573.

[31] Tosti Chiang, Stephen Yang, Gwo-Jen Hwang. 2014. Students' online interactive patterns in augmented reality-based inquiry activities. Computers & Education, 78, 97-108. DOI: https://doi.org/10.1016/j.compedu.2014.05.006.

[32] Unity3D. Unity. Retrieved from https://unity3d.com/.

Philippine Information Technology Journal, Vol. 11, No. 2, July - December 2018

A Review on Process-Oriented Approaches for Analyzing Novice Solutions to Programming Problems

Maureen M. Villamor
University of Southeastern Philippines
Iñigo Street, Bo. Obrero
Davao City
maui@usep.edu.ph

ABSTRACT

Introductory programming courses are both difficult for students to learn and for instructors to teach. High attrition and dropout rates are common in these courses. One of the reasons students drop out is the loss of motivation to pursue their programming courses due to the lack of feedback and proper assessment of their progress. A process-oriented approach in assessing programming progress entails examining and measuring students' compilation behaviors by analyzing their online protocols, which is a collection of source codes written and submitted by the students to the compiler while coming up with a solution to a programming problem. In this paper, the elements of a process-oriented approach are discussed as well as previous studies that used this approach. Specific metrics covered are Jadud's Error Quotient, the Watwin Score, Probabilistic Distance to Solution, Normalized Programming State Model, and the Repeated Error Density. These metrics help in deriving meaningful information from collected data.

CCS Concepts

• General and Reference→General literature • Applied Computing→Education→ Computer-managed instruction.

Keywords

Process-oriented approach; novice programming; online protocol analysis; data-driven metrics

1. INTRODUCTION

Programming is essentially a difficult subject to learn. This is evident in the increasing failure and dropout rates among novice programmers in their introductory programming courses [1, 2, 3, 4, 5]. Several studies had been conducted to pinpoint what could have contributed to these high attrition rates [6, 7, 8, 9, 10, 11, 12, 13, 14, 15] and one of the prominent reasons is the lack of problem-solving skills [16, 17]. Results also showed that time management strategies, effort, lack of design and programming ability were among the primary reasons for their struggles in programming [15]. Most novice programmers go into coding immediately without clearly defining the specific tasks needed to be done. They engage in trial and error [16, 18] and rely mostly from the compiler's feedback for error correction [16, 19]. When students tend to depend solely from the compiler, they could easily get discouraged whenever they hit a roadblock and cannot get themselves "unstuck" from an error after exhausting all the strategies they are familiar with. Students who portray this

This scenario involving novice programmers is common not only during actual programming exercises in school but also at the comforts of their home when attempting to work on solutions to their programming assignments. This is very frustrating to students causing them to possibly disengage from what they are trying to solve. Their lack of interest to carry on with their solutions could persist making them less inspired to learn further about programming. This is seen as a likely threat to programming students who are under pressure and those who have become less motivated because studies have shown that there is a link between student motivation and success in learning to program [16].

Programming teachers, on the other hand, have little or no way of knowing the real hardships students face and the progress that they make along the way since the teachers only see the final outputs submitted to them. Teachers possibly do not even realize the extent of knowledge and capacity their students possess to write correct programs [19]. This is the problem when students are being assessed traditionally. Giving students performance-based conventional assessments such as take-home programming assignments, examinations (e.g. requiring students to come up with code fragments), and charettes (e.g. short programming assignments carried out in a fixed-time laboratory sessions) [21] and grading only their final outputs do not take into account the students' intentions [22] and the process that led to their final submissions [23]. Programming should be regarded as not just merely about code generation but also as a means to reflect how students think, decompose, and solve problems [26].

It is for this reason that it is important that students' intermediate results be likewise considered rather than just the completed output. In order to gain meaningful insights about how students move towards a solution and be able to judge their programming proficiency as well as provide the necessary remedial actions, a process-oriented approach [27] is strongly suggested so that the students' programming process can also be analyzed. Students would appreciate if their teacher would inform them how their work would be assessed, provide them with intermediate feedback to encourage them more [28], and award them the grade that would truly measure their progress. If no forms of instructional interventions are afforded, students would be left clueless as to where they have gone wrong and consequently would probably choose to drop out when they get discouraged.

Once of the challenges of novice student instruction in Computer Science and other related disciplines, therefore, is how to keep the students' interest in programming. The key to keep them motivated is to help them succeed in their early programming courses. Students must not only possess the skills but also the motivation to succeed in learning [29]. Hence, if teachers can help keep their students' interest in their early programming

courses at bay, then there is a greater chance that attrition could be reduced. One way to do this is to use a process-oriented style of gauging their work.

The goal of this paper is to discuss the following (1) process-oriented approach in programming and its potential benefits, (2) elements of a process-oriented approach, (3) studies conducted using this approach in the context of novice programming with focus on data collection, and (4) some metrics used in this approach.

2. A PROCESS-ORIENTED APPROACH IN PROGRAMMING

Programming in itself is a process, and hence, the process that students undertake in developing a solution to a computer problem must also be evaluated. A completed source code weakly represents the student's knowledge and skills needed to create it [23] because when a student's final work is solely graded, it is not possible to ascertain he student's level of ability since that single grade is a combination of several factors [24]. A final grade alone as a metric is neither valid nor reliable to deduce about the student's ability to write codes [30]. As such, it is proposed that using a more complex metrics is more suitable in assessing students' work in the process of coming up with the final solution.

These metrics, which are based on students' progress, code or behavior, are called data-driven metrics [31]. Examples of these metrics are the *Intention-Based Score (IBS)* [23] also known as "degree of closeness [29]", *Error Quotient (EQ)* [25], *Watwin Score (WS)* [32], *Probabilistic Distance to Solution (PDS)* [33], and the more recent *Normalized Programming State Model (NPSM)* [34] and *Repeated Error Density (RED)*[35].

A process-oriented method of assessing open-ended problems [36], such as in programming, is about capturing students' intermediate works and using a suitable metric to account for students' learning and behavior. In this approach, the process of producing the final output is examined in an attempt to see the students' progression while solving the problem. The data that can be captured in the course of program development can be turned into meaningful information to help teachers see if students understand the programming concepts used [37]. With this chronological data progression, it is possible to recreate the problem-solving steps the student makes [18], giving a glimpse of how each student advances towards a solution.

Patterns can also be observed from this kind of data and analyzing such patterns can give teachers and researchers alike an indication of students' behavior and learning process, which can be a fertile ground for research. Examples of these patterns that can be extracted are programming behaviors patterns such as debugging [38], compilation and coding [18, 39]; patterns depicting practices common to successful students but not to struggling students [40]; problem solving strategies [41, 42] and anomalies [43]; submission patterns [17, 44]; and code evolutions [45, 46]. The possibility of cheating and the effect of starting late on projects can also be traced from these patterns [47] as well as distinguishing experienced students from poorly performing students [48, 49]. The data collected can also be explored to predict student performance identifying at-risk students in programming [17, 32, 50, 51, 52] with the aim of providing them tailored remedial instructions. Capturing and analyzing these patterns as a result of using a process-oriented approach is indeed important as it can provide significant help to teachers to vary their strategies for the benefit of the students.

3. ELEMENTS OF A PROCESS-ORIENTED APPROACH AND STUDIES CONDUCTED

Using a process-oriented approach of assessing students' work entails a decision on how data will be collected, how much data will be captured, and how frequent the data will be stored. For this approach to effectively work, data must be acquired automatically from students during their actual programming sessions using some tools or programming environment [19]. All source codes produced, together with other pertinent related information, are gathered. The term *online protocol* is used to refer to this sort of data. Precisely, an online protocol is a collection of files submitted to a compiler that gives a chain of snapshots [23] representing the programmers' in-between thought processes, and hence, has the capability to capture not only the final submissions but also the entire history of program development.

In developing an online protocol, the granularity of data must be decided. Different data granularity level were identified [52]: (1) *submission-level data* – stores submitted source codes; (2) *snapshot-level data* – stores file saves, compilations, and executions; and (3) *keystroke-level data* – stores data every time a student makes changes in the source code. These levels of data granularity are ordered as follows from the finest to coarsest [54]: *keystrokes, line-level edits, file saves, compilations, executions*, and *submissions*.

It is important to select the level of granularity that could provide adequate and reasonable amount of data depending on the analysis need. If data is collected at the smallest level of detail (e.g. mouse events or keystrokes), then there should be a means to store and handle large volumes of data. Some of these details might not be even relevant anymore. On the other hand, if data is collected at a larger granularity (e.g. assignment submissions), then extracting the necessary information might not be possible. Comparisons of these levels are found in [53].

Tools that could unobtrusively automate data collection must also be considered. The common tools utilized by researchers that could provide the means for data to be collected systematically are the automated grading systems, IDE instrumentation, version control systems (VCS), and key logging [54]. The granularity of the data is dependent on these tools. For instance, automated grading systems like Web-CAT [55] produce data at the granularity of submissions. Web-CAT is capable of collecting file submissions of the same task repeatedly. Examples of IDE instrumentation tools, which collect data as snapshots are called third generation approaches [40] like Hackystat (file modifications) [56], Jadud's work (compilation events) [25], Marmoset (file saves) [57], and ClockIt (compilation events) [40]. VCS analyze source code snapshots at commit points only, whereas key logging offers data collection down to the smallest level of detail such as individual keystroke events.

There is plenty of literature using a process-oriented approach. Table 1 lists studies conducted ordered chronologically by year from 2006-2016 using this approach in the context of novice programming. The table shows what has been done, the programming language and tools used along with the data granularity.

Of the 42 studies listed, majority of the data collection was done in Java (26), followed by Python (6), and C++ (2). Five of the studies did not mention the programming language used, while two of the studies were conduction using multiple programming

languages (C/Python and C/Python/Java). Since Java was the preferred programming language, it is assumed that this is the reason why BlueJ was the most preferred tool for capturing data. Compilations (11) took the lead when it comes to data granularity followed by a combination of varying levels of granularity with submissions (9) coming in third.

4. METRICS

In a process-oriented approach, the use of a proper metric in analyzing online protocol is important if the aim is to derive meaningful information from the data collected. The studies listed in Table 1 utilized a combination of different metrics, which is dependent on the information need of the researcher. For the purpose of discussing how these metrics were derived and computed, only five of these metrics are discussed. These metrics are Jadud's Error Quotient (EQ) [25], Watwin Score (WS) [31], Probabilistic Distance to Solution (PDS) [33], Normalized Programming State Model (NPSM) [34], and Repeated Error Density (RED) [35].

4.1 Error Quotient

EQ is a metric that measures the number of syntax errors a student encounters in a single programming session (i.e. a sequence of compiles from one class period). This measurement is a based on several criteria, such as the (1) number of syntax errors made by a student, (2) the frequency of repeating those syntax errors, and the (3) error's source location in subsequent compilations. A high EQ is given to students who commit many syntax errors and fail to fix them in between compilations. On the other hand, a low EQ is awarded to students who encounter a small number of syntax errors or who manage to quickly fix the syntax error encountered. The use of EQ permits us to compare student sessions easily.

Given a session of compilation events e_1 through e_n, the EQ is calculated as follows [25]:

Collate — Create consecutive pairs from the events in the session, e.g. (e_1, e_2), (e_2, e_3), (e_3, e_4) up to (e_{n-1}, e_n).

Calculate — Score each pair according to the algorithm presented in Figure 1.

Normalize — Divide the score assigned to each pair by 9 (the maximum value possible for each pair).

Average — Sum the scores and divide by the number of pairs. This average (in the range 0-1) is taken as the EQ for the session.

To show how the scoring is done, consider the tabular visualization in Figure 2. The table in this figure is an example of a novice's compilation session with four events. Each row in the table signifies the change between a pair of successive compilation events. The meanings of the table columns are as follow:

Err Type — Denoted with a "★" if a compilation pair was error-free; otherwise, it shows a number representing a known syntax error

⊿T — Represents the amount of time (in seconds) that passed between compilations (reduced to five bins: 0-10 seconds, 20-30 seconds, 30-60 seconds, 60-120 seconds, and more than two minutes

⊿Ch — Tells how many characters were added or removed from one compilation to the next

Location — Represents the extent of student's program file (left side: start, right side: furthest point a student edited their program, colored rectangle: location of the syntax error reported, black dot: location of the change made to fix the syntax error)

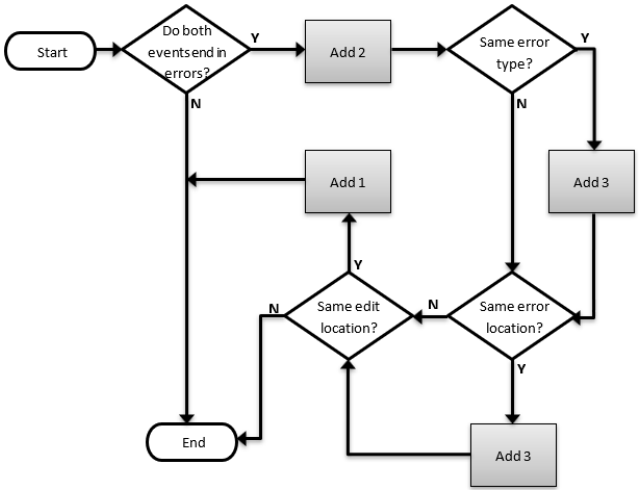


Figure 1. EQ algorithm flowchart.

#	Err Type	ΔT	ΔCh	Location	SCORE
1	4		2		
2	★		13		0
3	1		-27		0
4	1		-2		8

Figure 2. An example of a four-event session [25].

Scoring begins by taking a pair of events one by one. Referring to Figure 2, these event pairs are (1, 2), (2, 3), and (3, 4). Taking the first pair and tracing using the flowchart in Figure 1, it can be verified that both events end in a syntax error. Since the second event is error-free, this pair of events is given a score of zero. This is repeated on the second pair, which also receives zero since the first event incurred no errors. As for the third pair, both events end in a syntax error (add 2), both have the same error type (add 3), and both have the same error location (add 3) yielding a total of 8. Hence, the three pairs have scores 0, 0, and 8, respectively. The scores are then normalized to 0, 0, and 0.88 resulting to an average penalty of 0.29 (0.88/3) to the pairs. This average over all the pairs is now the EQ of this programming session.

Per Jadud's experiment, EQ appears to be inversely correlated with grades, which means that students who have low EQ scores are likely to have higher assignment and exam grades in programming than those students with high EQ scores. Further, EQ offers also a possibility as a predictor for the students' academic performance.

4.2 Watwin Score

The Watwin algorithm attempted to address the flaws of EQ as the latter only assumes that students either work only on a single

source file, or linearly work on several files. Watson's et al. [31] found, however, that students can probably work on multiple files. This algorithm also offers considerable improvement by increasing its predictive accuracy in predicting student performance based on the way a student tackles different types of errors when compared to peers by proposing resolve time as a predictor. Students receive penalty based on the amount of time they take to fix an error relative to their peers, which is distributed normally. The algorithm is performed as follows:

Input: A set of student programming logs (compilation and invocation) for all files compiled during a session

1. *Prepare* a set of successive compilation-event pairings using the process shown in Figure 3.
2. *Quantify Programming Behavior*
   - *Score* all compilation pairings generated using the scoring algorithm in Figure 4.
   - *Normalize* each score by dividing by 35 (maximum possible score for each pairing).
   - *Average* the normalized scores of all pairings.

Output: The average of all pairings (in the range 0-1) is taken as the student's WS for the session. A score of zero denotes that the student's session is error-free, whereas a score of 1 means that all compilations ended in errors. A score approaching zero indicates that a student is more experienced in programming.

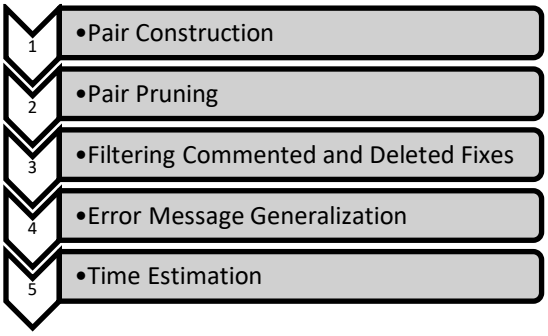


Figure 3. Steps in preparing a set of compilation pairings.

Step 1 in Figure 3 builds pairs of compilation events $\{\{e_1, e_2\}, \{e_2, e_3\}, \ldots, \{e_{n-1}, e_n\}\}$ ordered by timestamp on a per file basis. In building such pairs, it considers the possibility where a student works concurrently on multiple files. In step 2, all pairings $\{e_f, e_t\}$, whose code snapshots are the same are identified and removed as these can bloat the total number of compilation pairings. Pairings with a successful event type e_f are likewise removed. Deletion and commented fixes are detected in step 3 as these do not reflect a student's ability in fixing errors. Deletion fixes are spotted by getting the diff ratio between the snapshots of e_f and e_t. The pair $\{e_f, e_t\}$ is removed if the number of insertions and alterations is zero, and if deletes is greater than zero. Commented fixes are spotted and removed using a regex expression. In step 4, error messages generated in every compilation event pairs are generalized to build a profile for different classes of errors. Finally, in step 5, the amount of time spent on each compilation pairing $\{e_f, e_t\}$ is estimated. This is accomplished by creating a combined sequence of invocation and compilation events $\{h_1, h_2, \ldots, h_{k-1}, h_k\}$ for all files in a session, sorted by timestamp. For every $\{e_f, e_t\}$, if there occurs an h_i, such that the timestamp of $e_f > h_i > e_t$, the estimated time spent on $\{e_f, e_t\}$ is computed as the difference between the timestamps of e_t and h_i.

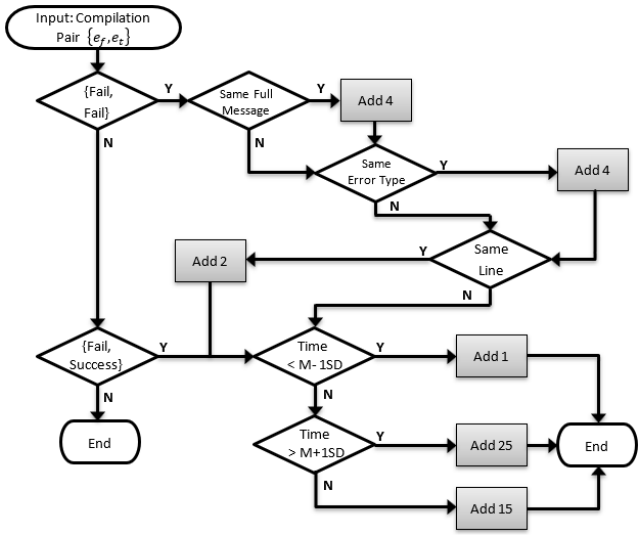


Figure 4. Watwin score algorithm.

In this approach, resolve time to fix errors is used as a predictor in the scoring model for two reasons: (1) this has not been explored in previous studies prior to the conduct of this research, and (2) the researchers found a strong significant correlation in students' mean resolve time vis-à-vis performance. As illustrated in the scoring algorithm in Figure 4, students are penalized if their resolve time is more than one deviation above or below the mean. A low penalty means that students have fixed an error quickly than their peers, and a high penalty signifies that students have fixed an error much slower than their peers. After scoring all pairings, the scores are normalized, and the computed average becomes the WS. The components included and the penalties assigned in the algorithm were not products of random guesswork but were based on previous studies conducted using regression models and cross-validation, hence making this algorithm applicable to independent datasets.

Using a students' WS as the independent variable, and the students' overall coursework mark as the dependent variable, a linear regression can be performed to test the predictive power of this algorithm on students' performance in programming. Moreover, WS can also be used as a classifier of student performance.

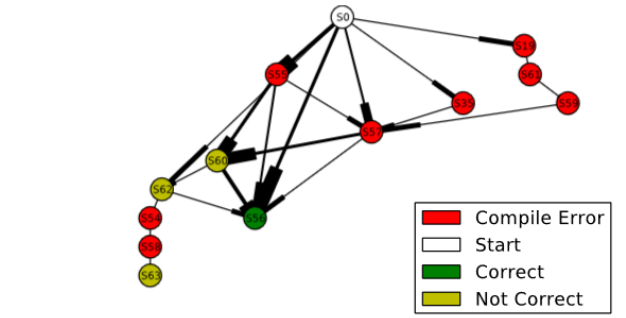
4.3 Probabilistic Distance to Solution

Traditional metrics of measuring student performance, such as those that depend on the number of submissions or deal with the time spent to correctly finish a programming problem do not actually reflect student's understanding and misconceptions about the problem they are solving. In fact, these are considered too coarse-grained to make a distinction between conceptual misunderstanding and syntactical or parsing mistakes made by the students. The PDS metrics gives a glimpse of the entire history of problem-solving steps a student takes starting from the first attempt of solving the problem up to the completed solution while providing additional insight into misconceptions and problem-solving paths by focusing on individual algorithmic components within the larger algorithm student are constructing. These

algorithmic components, which are used to evaluate student progress, provide a way to determine if a student is making a productive edit (e.g. moving closer to a solution) or an unproductive edit (e.g. does not fix an incorrect algorithmic component of the code), whether engaging in a shotgun debugging or pursuing a misconception.

The idea is, for each submission, the student's code is converted to a vector of binary features called an alignment vector (AV), which encompasses the algorithmic components (e.g. looping structure, decision structure, use of the loop control in an array access, and inclusion of a return statement), correctness outcomes, and compilation success. The model of required algorithmic elements was based on the work of Sudol [77]. A value of 1 and 0 in the AV represents the appearance and absence of each algorithmic component, respectively. The generated AVs are represented as program states in a network. The model used is useful because it can be seen which individual concepts students struggle and is also useful for tracking and evaluating students' progress across multiple submissions.

Figure 5 shows a snapshot of a network constructed given a particular problem. The network nodes $S_1 \ldots S_{n-1}$ are possible program states with an end state node *F*, and the edges represent the observed transitions between states. In the table, columns *Fx* are the binary program features that correspond to the required algorithmic components for a given problem. Row *Sx* indicate the presence (green box) or absence (white box) of those features. *S*0 is the starting state (empty starter code), and *S*56 represents the correct final state. In the transition graph, the thickness of the edges indicates the number of edits the transition was observed all over the students' submission, whereas the lengths are arbitrary and do not have something to do with the model. All lines are single directional, indicating a move from the thin side of the line to the thick side.



	F2	F4	F6	F7	F8	F9	F10	F11	F12	F13	F14	P(distance)
S0												3.67
S19												7.78
S35												4.78
S54												4.00
S55												2.99
S56												0.18
S57												4.43
S58												3.00
S59												4.78
S60												2.17
S61												6.78
S62												3.09
S63												-0.00

Figure 5. Example of student program states [34].

Consider, for example, the path S0, S55, S60, and S56. It can be seen from the AV (table in Figure 5) that from the initial state students submitted a code with a missing component (F8), and then a code with a correct return statement that did not compile before transitioning to the final state. For each node, a Maximum Likelihood Estimate (MLE) transition probability to every other node using the observed transitions was computed. Given the number of observed transitions from state *x* to state *y* ($T_{x,y}$), the probability of being in state *y* at time *t* is estimated with the MLE:

$$\hat{p}\left(S_y(t)\right) = P\left(S_y(t)\middle|S_x(t-1)|\right) = \frac{T_{x,y}}{\sum_i T_{x,i}}$$

This is equivalent to a Markov chain estimate with a 1-state history.

A classic problem in Computer Science, which is the random walk problem in networks, is applied to the calculation of a PDS particularly referring to the number of likely edits it would take a student from any given node (AV state) to a final state (complete solution). A mean distance from each state to the final state is calculated using a set of linear equations. This is done by modeling each edge as a transition probability and single unit of distance between states.

The PDS metric, along with the transition graph (TG) as seen in Figure 5, can provide substantial information about the paths followed by students in their attempt to arrive at a correct solution. By looking at the PDS and the TG, it is possible to identify if a student has made productive edits or just guessing to fix compiler errors. This is because the AV was generated based on the instructional goals or learning objectives. Hence, its representation and visualization can draw generalizations how program states are parallel to student performance and other latent factors such as learning. For example, in the table in Figure 5, one student's first submission is observed as *S*55 with PDS of 2.99. and the next state it transitioned to is *S*57 with PDS 4.43. This means that the edit made was less productive since it resulted in a shift to a state with a higher probabilistic number of submits needed to arrive at a correct solution.

4.4 Normalized Programming State Model

NPSM differs from EQ and WS in terms of predicting student performance because not only it determines syntactic correctness, but also semantic correctness of a programming solution given at any point in time. Hence, it offers a more holistic predictive model of student performance compared to EQ and WS. NPSM's approach deals with programming data stream and requires that a student's current programming solution be mapped into one of the four states in the 2x2 space shown in Figure 6. The syntactic correctness of a program is determined based on whether the last compilation attempt results in an error. On the other hand, since semantic correctness is not immediately obvious, it cannot be determined right away. The only way to go around this is to look for the presence or absence of runtime exceptions in the last execution attempt. A program is semantically correct if it produces a runtime exception in the last execution attempt. It is semantically unknown if the last execution attempt does not produce a runtime exception.

		Syntax	
		Correct	*Incorrect*
Semantics	*Incorrect*	Syntactically correct/ Semantically incorrect	Syntactically incorrect/ Semantically incorrect
	Unknown	Syntactically correct/ Semantically unknown	Syntactically incorrect/ Semantically unknown

Figure 6. Dimensions of program correctness.

A state-transition diagram (found in [34]) is used to maps this model to the stream of programming log data provided by the IDE used in this study. The 11 states in the transition diagram along with their codes, for quick viewing, are shown in Table 2.

Table 2. The 11 states in the transition diagram

State	Code
Unknown (Start) State	UU
Idle	
Editing Syntactically Incorrect, Last Debug Successful	NU
Editing Syntactically Incorrect, Last Debug Unsuccessful	NN
Editing Syntactically Correct, Last Debug Successful	YU
Editing Syntactically Correct, Last Debug Unsuccessful	YN
Execute without Debug (following NU)	R/
Execute without Debug (following YN)	RN
Execute without Debug (following YU)	RU
Debugging (following YN)	DN
Debugging (following YU)	DU

The editing states in the model, namely: YN, YU, NU, and NN, serve as a rough proxy representing both the syntactic and semantic status of the program being edited as shown in Figure 6. It is inferred that students who spend more time in syntactically correct and semantically unknown states will likely to perform better than students spending more time in syntactically and semantically incorrect states. In the RN and DN states, students would seem to ask the question, "*Why does not my program work?*" and in the RU and DU states, they would be inclined to ask the question, "*Does my program work?*" Among these states, DN and DU are in a more powerful position to ask these questions since they are using the debugger. Lastly, students who spend time in R/ state may imply that a student is struggling. Overall, the relationship between these five execution states vis-à-vis student performance is not clear.

A student's programming activity is mapped to one of the 11 states in this model. To correlate the use of this method with student performance, the amount of time a student spent in each state is recorded and the recorded times relative to the total time the student spent programming is normalized. In the preliminary analysis of the data gathered using this model, it was observed that the *Idle* state was the dominating state of the students, which means that students are likely to spend most of their day not doing any programming activity. Since the focus is on the time students spent in programming, the *Idle* state was eliminated from the normalization process. One variable was added in the model, which is the time-on-task, leaving a total of 11 data points per students. These 11 data points form the NPSM.

A predictive measure was derived using NPSM to predict performance. Based on previous studies conducted and given that NPSM has 11 predictors, the ideal sample size is estimated at 220 students to achieve full statistical power when deriving the predictive measure. Four states were considered in the development of the predictive model, namely: UU, NR, RU, and RN, and multivariate regressions was performed using these states. RN and RU are found to be positive contributors to student success, while UU and NU are found to be negative contributors. This suggests that dealing with a program's runtime behavior, notwithstanding the semantic correctness, is a successful programming approach. On the other hand, writing large chunks of code and not compiling it (UU) does not contribute to programming success, and when students compile, they will be trapped in NU state, which is also found to be negatively correlated with performance. YN and NN are not significant predictors so they were dropped in this predictive model.

4.5 Repeated Error Density

RED is a metric used to quantify repeated errors. It is considered more advantageous compared to Jadud's EQ since it is less context dependent and is suitable for short sessions. RED has properties making it capable to answer Jadud's questions, such as: (1) "*If one student fails to correct an 'illegal start of expression' error over the course of three compilations, and another over 10, is one student [about] three times worse than the other?",* and (2) "*What if the other student deals with a non-stop string of errors, and the repetition of this particular error is just one of many?*" Dissecting these questions would mean that when EQ is used as a metric, both students in these cases would have an EQ of 1.

These cases can be demonstrated in the succeeding scenarios. Take students *A*, *B*, and *C*; an error type *x*; and a repeated error *r*. Take student *A* who logs two consecutive errors of type *x* resulting in one repeated error *r*. Is it fair to say that student *B* who logs three consecutive errors of type *x* resulting in two repeated errors *2r* is struggling with that error more by a factor of two? What about student *C* who also logs *2r* but not all in one string *s* (say across *2s*, by logging two consecutive errors of type *x* followed by some successful compiles, followed by two more consecutive errors of type *x*)? Considering all these cases and given the assumption that no other errors are made, and all other variables among the three students are equal, the EQ will be equal to 1 in all cases. Table 3 summarizes these three scenarios.

Table 3. Summary of the three scenarios

Student	Seq. *S*	*x*	*r*	*s*	EQ
A	... *x x* ...	2	1	1	1
B	... *x x x* ...	3	2	1	1
C	... *x x*... *x x* ...	4	2	2	1

Table 3 shows that EQ is not of any help. Hence, RED came into the picture to quantify repeated errors by both looking at the amount of repeated error strings a student encounters and looking at the length of these strings. This metric involves summing a submetric calculated on each repeated error string encountered in a compilation event sequence. This submetric is $r_i^2/|s_i|$ where $|s_i|$ is the length of the string s_i containing r_i repeated errors. This can be expressed completely in terms of r_i as $r_i^2/(r_i+1)$, since the length of a string s_i containing r_i consecutive errors is always equal to $r_i + 1$.

The value of RED for a given sequence *S* of *n* repeated error strings for each s_i in *S* is shown in the equation below:

$$RED = \sum_{i=1}^{n} \frac{r_i^2}{r_i + 1}$$

where r_i is the number of repeated error *r* as a pair of events where each event results in the same error. Table 4 shows the RED values for all sequence combinations for which a total of 0-4 repeated errors are encountered.

Table 4. RED values for all combinations of 0-4 repeated errors *r* (*A, B, C* correspond to students in Table 3)

No.	Sequence *S*	*r*	RED
1	*x*	0	0
2	... *x* ... *x* ...		0
3 (*A*)	... *x x* ...	1	0.5
4 (*C*)	... *x x* ... *x x* ...	2	1
5 (*B*)	... *x x x* ...		$1.\overline{3}$
6	... *x x* ... *x x* ... *x x* ...	3	1.5
7	... *x x x* ... *x x* ...		$1.8\overline{3}$
8	... *x x x x* ...		2.25
9	... *x x* ... *x x* ... *x x* ... *x x* ...	4	2
10	... *x x* ... *x x* ... *x x x* ...		$2.\overline{3}$
11	... *x x x* ... *x x x* ...		$2.\overline{6}$
12	... *x x* ... *x x x x* ...		2.75
13	... *x x x x x* ...		3.2

In Table 4, sequences 1 and 2 mean that a single occurrence of error *x*, or two occurrences separated by other activity, do not comprise repeated errors, and hence, has a RED value of 0. Sequences 4 and 5 illustrate two ways that error *x* can be repeated twice successively, but sequence 4 is considered the "natural" choice for showing that a student struggles twice of that the student in sequence 3 as evidenced by the RED values of sequence 4 (1) and sequence 3 (0.5) giving a ratio of 2:1. The same can also be observed for sequences 11 and 5. With this, Jadud's first question was addressed.

For Jadud's second question, this is addressed by inspecting sequences 5 and 4. Sequence 5 contains one long string of two repeated errors with RED = $1.\overline{3}$, and sequence 4 also has two repeated errors but over two shorter strings and has RED = 1. In this case, a penalty of 1/3 is incurred for a sequence that encounters two repeated errors in one long string. Therefore, it can be said that the student in sequence 5 (*B*) is struggling more with error *x* than the student in sequence 4 (*C*) as reflected by the former's higher RED score.

5. SUMMARY AND CONCLUSION

This paper discusses what is a process-oriented approach of analyzing students' programming solutions. This approach not only takes into account the final submissions of the students but also captures the progress the students make in writing programs. In using a process-oriented approach, online protocols (e.g., sequences of automatically collected source codes written by students while writing a solution to a programming problem) must be first be collected by deciding the level of data granularity to use. These levels of data granularity are ordered as follows from the finest to coarsest are the following: keystrokes, line-level edits, file saves, compilations, executions, and submissions. In order to automatically and inconspicuously collect data, different tools (e.g., instrumented BlueJ, Marmoset, ClockIt, Web-CAT, etc.) must also be utilized depending on the level of data granularity used.

Once the online protocols are in place, the next step is to employ some data-driven metric that can quantitatively analyze these online protocols. This paper covers some of these metrics, which prove to be helpful in deriving meaningful information about the students' behavior while writing a program. The metrics discussed include Jadud's Error Quotient (EQ), Watwin Score (WS), Probabilistic Distance to Solution (PDS), Normalized Programming State Model (NPSM), and Repeated Error Density (RED).

EQ measures how many syntax errors a student encounters during a single programming session and how successive compilation failures in a session compare in terms of error message, location, and edit location. EQ is proven to correlate inversely with grades, and hence, can be used to predict student performance. It can also be used as an indicator how well or poorly a student is progressing. On the other hand, EQ suffers from some disadvantages: (1) it is dependent on parameter values, (2) it has not explored the amount of time which a student takes to resolve an error, (3) it assumes that students only work on a single source file or work on multiple files in a linear manner that is considered flawed when creating a set of consecutive compilation pairings, and (4) it varies across groups, environments, and context.

WS uses time as a predictor to predict performance based on how a student responds to different types of error compared to their peers. WS was able to address the shortcomings of EQ by constructing pairings on a per-filename basis and can be used to predict student performance even early in the course. Overall, WS outperforms EQ as a predictive measure. However, with WS, no measures are taken to check superficial changes made to source code, which can be incorrectly flagged as semantic changes.

The PDS metric measures probabilistic distance between an observed student solution and a correct solution using a Markov model. The good thing about PDS is that it can determine if an edit or student path is (a) typical of students who have mastered content, and (b) productive in progression toward a solution. PDS, however, is difficult to implement because it requires that a model of required algorithmic components must be identified first. Likewise, the student evaluation is also constrained based on the model used.

NPSM characterizes students' programming activity in terms of the dynamically-changing syntactic and semantic correctness of their programs. It can be used to predict student performance by considering the programming process more holistically. NPSM is substantially a better predictor than EQ and WS. Although, NPSM includes semantic correctness, it provides however only a rough proxy for determining semantic correctness by checking for the presence or absence of runtime exceptions in the last execution attempt.

RED quantifies repeated errors by looking at the amount of repeated error strings a student encounters, and also the length of these strings. RED, unlike EQ, is more suitable for shorter sequences or sessions. It is more attractive than EQ as it can be applied in any context independent of the parameter value selection. However, RED's unbounded nature brings some questions when a large number of data points are involved. It is also prone to outliers and has not been validated if it correlates with student performance.

It can be observed that Jadud's EQ has become the basis of the creation of other metrics. Newer metrics had been developed to address the shortcoming of the older metrics. As data evolve and as the need for answers are being sought, more suitable and better metrics will come to exist in the near future.

Table 1. Studies conducted using a process-oriented approach in the context of novice programming

Study	Year	What has been done?	PL	Tools	Granularity
[57]	2006	Provided controlled feedback and instructional monitoring on students' progress for them to start working early	Java	Marmoset IDE Instrumentation with optional CVS synchronization	File saves
[58]	2006	Quantified how much or how little student struggles with syntax errors in terms of EQ	Java	Instrumented BlueJ	Compilations
[19]	2006	Correlated EQ with exam marks	Java	Instrumented BlueJ	Compilations
[59]	2006	Determined the typical errors made and guess the programming intentions	Eiffel	Eiffel Studio	Compilations
[60]	2008	Implemented a reporting mechanism and derived a correlation on the following: no. of submissions vs. final score, code complexity vs. final score, and early vs. late testing	Java	Web-CAT and Business Info. Reporting Tool (BIRT)	Submission
[40]	2008	Determined the practices that make students successful S/W developers and what practices do not	Java	ClockIt (BlueJ data logger/ visualizer)	Compilations
[50]	2008	Identified at-risk students by computing EQ	Java	Instrumented BlueJ	Compilations
[47]	2009	Identified some higher-level patterns of novice student programming behaviors (e.g. potential cheating, impact of starting projects late, etc.)	Java	ClockIt	Compilations
[61]	2009	Measured which concepts were difficult to students, how well the different programming structures were understood, and traced student learning	Python	Virtual learning environment	Submissions
[62]	2009	Used a combination of human observation, midterm score, and online protocol to study which observable affective states and behaviors can be used to predict student achievement	Java	Instrumented BlueJ and JCreator	Compilations
[63]	2009	Attempted to automatically detect student frustration	Java	Instrumented BlueJ	Compilations
[64]	2009	Analyzed novice programmer behavior (mover, stopper, and extreme movers) using EQ scores	Java	BlueJ Browser	Compilations
[44]	2010	Used frequent episode mining to find common patterns of behavior (well-performing vs. poorly performing, early vs. late, etc.)	Java	Web-CAT	Submissions
[46]	2011	Explored and compared student coding strategies and identified coding profiles (copy-pasters, mixed-mode self-sufficients)	Java	NetLogo	Keystrokes, mouse events, compilations
[51]	2011	Determined error, compilation, and EQ profiles of students and attempted to accurately identify at-risk novice programmers	Java	Instrumented BlueJ	Compilations
[45]	2012	Predicted student performance using machine learning on student development traces and trajectories	Java	Eclipse	Compilations
[37]	2012	Proposed a process to capture intermediate versions of students' programs during development and analyzed the data using milestone markers	Python	Google documents editor	File saves
[17]	2012	Tracked a students' overall assignment submission rate (e.g. timeliness) to identify students who are at-risk of performing poorly in class	(not mentioned)	Web Submission System, SVN repository	Submissions
[43]	2012	Visualized the solution paths using an interactive graph to explore patterns and anomalies	Python	JS-Parsons tool	File modifications
[65]	2012	Explored and analyzed mistakes in student-submitted solutions to study novice's misconceptions of programming	Python	UUhistle Program Visual. System	Submissions
[33]	2012	Proposed a metric to measure the probabilistic distance between an observed student solution and a correct solution as applied to a transition graph	(not mentioned)	Online tutor	Submissions
[66]	2013	Revealed student work patterns (e.g. what hours of day students work, how much work is done before and the day of the deadline, total amount of time spent coding)	Java	Marmoset using an Eclipse Plugin	File saves with unit tests

		and examined the use of release tests in detail that provides feedback to students			
[27]	2013	Attempted to detect the evolution of students programming strategies and knowledge focusing on "tinkering" and "planning" episodes	(not mentioned)	(not mentioned)	Line-level edits and keystrokes
[67]	2013	Proposed a tool for observing and recording the programming process to apply Personal Software Process (PSP) in the classroom and to enable learners to conduct PSP analysis by themselves	Java	Developed Eclipse logger, Programming Process Visualizer	Keystrokes (minimum)
[32]	2013	Predicted performance using a time as a predictor based upon how a student responds to different types of error compared to their peers (Watwin Algorithm)	Java	Instrumented BlueJ	Compilations
[68]	2013	Presented and demonstrated tool for collecting, viewing detailed data about Python programming sessions and analyzing students' activities (e.g. testing, use of automatic feedback)	Python	Web-based Python programming environment	Submission, file modifications, compilations, executions
[69]	2014	Used a browser-side snapshot analysis tool capable of providing functionality for annotating and visualizing codes snapshots to seek differences between novice programmer that passed or failed	(not mentioned)	CodeBrowser	File saves, line-level edits
[42]	2014	Examined programming behavior found along student problem paths that exhibit patters as Builders, Massagers, Reducers, and Strugglers	Java	JavaParser	File modifications and saves, executions, submissions
[70]	2014	Characterized students based on their intention-based score patterns and "update characteristics" of their code compilations	Java	Instrumented BlueJ	Compilations
[71]	2014	Explored factors that may influence the difficulty of programming assignments by applying a recursive partitioning to construct a decision tree of assignment difficulty, and metrics for automatically assessing those factors	Java	(not mentioned)	Keystroke events
[72]	2014	Used machine learning techniques to discover programming behavioral patterns, correlated them with students' assignment and exam grades, and transformed code snapshots of map of states that show the progress of the students' work	Java	Instrumented Eclipse	File saves, file modifications
[73]	2014	Investigated the things that programmers with no previous programming experience struggle with and how their behavior changes over a short period of time	Java	Test My code (NetBeans plugin)	Keystroke events
[39]	2015	Explored correlation on students' effort and success on exercises vs. final exam score, attempted to detect "flailing" students early enough, and tried to answer whether students improve during the semester	C, Python	CloudCoder	Submissions, test results, file modifications
[74]	2015	Considered the incremental changes students make and correlating score between sequential submissions using the following metrics: source lines of code, cyclomatic (McCabe) complexity, state space, and the 6 Halstead measure of complexity of the program	C++	Athene online automated system	Submissions
[75]	2015	Used the Blackbox data set to analyze the frequency, time-to-fix, and spread of errors among users	Java	BlueJ	Compilations
[76]	2015	Detected high- and low-performing students after the very first week of the programming course to provide better support for them using Error Quotient, Watwin score, and standard machine learning techniques	Java	Test My Code	Keystroke events
[34]	2015	Proposed and derived NPSM to predict students programming performance, and compared its explanatory power against EQ and WS	C++	OSBIDE	All programming activities
[31]	2015	Evaluated the performance of EQ in multiple contexts (different data sets, different languages, working practices, and student backgrounds)	C, Python, Java	CloudCoder, PCRS, Test My Code	Submissions with code changes only
[52]	2015	Attempted to predict accurately failing and passing students in the middle of the semester using a decision	(not mentioned)	PASTA (automatic marking and	Submissions

		tree classifier		feedback system)	
[48]	2015	Aimed to automatically analyze students' solving process in programming exercises using visualization	Python	Thonny	All actions performed by user
[49]	2016	Correlated keystroke latency with exam performance and programming experience	Java	Test My Code	Keystrokes
[35]	2016	Introduced RED to quantify errors, and compared it with EQ	Java	Custom Java editor	Compilations

6. REFERENCES

[1] Jens Bennedsen and Michael E. Caspersen. 2007. Failure rates in introductory programming. *ACM SIGCSE Bulletin*39, 2 (2007), 32. DOI:http://dx.doi.org/10.1145/1272848.1272879

[2] Colleen Lewis. 2010. Attrition in Introductory Computer Science at the University of California, Berkeley. *Berkeley: University of California, Berkeley*.

[3] Aharon Yadin. 2011. Reducing the dropout rate in an introductory programming course. *ACM Inroads*2, 4 (2011), 71. DOI:http://dx.doi.org/10.1145/2038876.2038894

[4] Christopher Watson and Frederick W.b. Li. 2014. Failure rates in introductory programming revisited. *Proceedings of the 2014 conference on Innovation & technology in computer science education - ITiCSE 14*(2014). DOI:http://dx.doi.org/10.1145/2591708.2591749

[5] Diane Horton and Michelle Craig. 2015. Drop, Fail, Pass, Continue. *Proceedings of the 46th ACM Technical Symposium on Computer Science Education - SIGCSE 15*(2015). DOI:http://dx.doi.org/10.1145/2676723.2677273

[6] Kulli Kori et al.2015. First-year dropout in ICT studies. *2015 IEEE Global Engineering Education Conference (EDUCON)*(2015). DOI:http://dx.doi.org/10.1109/educon.2015.7096008

[7] Beth Simon, Raymond Lister, and Sally Fincher. 2006. Multi-Institutional Computer Science Education Research: A Review of Recent Studies of Novice Understanding. *Proceedings. Frontiers in Education. 36th Annual Conference*(2006). DOI:http://dx.doi.org/10.1109/fie.2006.322291

[8] Päivi Kinnunen and Lauri Malmi. 2006. Why students drop out CS1 course? *Proceedings of the 2006 international workshop on Computing education research - ICER 06*(2006). DOI:http://dx.doi.org/10.1145/1151588.1151604

[9] Anabela Gomes and António José Mendes. 2007. Learning to program-difficulties and solutions. *International Conference on Engineering Education–ICEE* (2007).

[10] Maureen Biggers, Anne Brauer, and Tuba Yilmaz. 2008. Student perceptions of computer science. *Proceedings of the 39th SIGCSE technical symposium on Computer science education - SIGCSE 08*(2008). DOI:http://dx.doi.org/10.1145/1352135.1352274

[11] Jan Erik Moström. 2011. A study of student problems in learning to program. Dissertation. *Umeå University, Department of Computing Science*.

[12] Mahmoud Mhashi and A. L. I. M. Alakeel. 2013. Difficulties facing students in learning computer programming skills at Tabuk University. *Proceedings of the 12th International Conference on Education and Educational Technology (EDU'13),* Iwate, Japan.

[13] Beth Simon, Julian Parris, and Jaime Spacco. 2013. How we teach impacts student learning. *Proceeding of the 44th ACM technical symposium on Computer science education - SIGCSE 13*(2013). DOI:http://dx.doi.org/10.1145/2445196.2445215

[14] Kofi Adu-Manusarpong, John Kingsley Arthur, and Prince Yaw Owusu Amoako. 2013. Causes of Failure of Students in Computer Programming Courses: The Teacher Learner Perspective. *International Journal of Computer Applications*77, 12 (2013), 27–32. DOI:http://dx.doi.org/10.5120/13448-1311

[15] Rebecca Vivian, Katrina Falkner, and Nickolas Falkner. 2013. Computer science students causal attributions for successful and unsuccessful outcomes in programming assignments. *Proceedings of the 13th Koli Calling International Conference on Computing Education Research - Koli Calling 13*(2013). DOI:http://dx.doi.org/10.1145/2526968.2526982

[16] Angela Carbone, John Hurst, Ian Mitchell, and Dick Gunstone. 2009. An exploration of internal factors influencing student learning of programming. *Proceedings of the Eleventh Australasian Conference on Computing Education*, 95, 25-34. Australian Computer Society, Inc.

[17] Nickolas J.g. Falkner and Katrina E. Falkner. 2012. A fast measure for identifying at-risk students in computer science. *Proceedings of the ninth annual international conference on International computing education research - ICER 12*(2012). DOI:http://dx.doi.org/10.1145/2361276.2361288

[18] M. H. N. C Vee, Bertrand Meyer, and Keith L. Mannock. 2006. Understanding novice errors and error paths in object-oriented programming through log analysis." In *Proceedings of workshop on educational data mining at the 8th international conference on intelligent tutoring systems (ITS 2006)*,13-20.

[19] Matthew C. Jadud. 2005. A First Look at Novice Compilation Behaviour Using BlueJ. *Computer Science Education*15, 1 (2005), 25–40. DOI:http://dx.doi.org/10.1080/08993400500056530

[20] D.N. Perkins, Chris Hancock, Renee Hobbs, Fay Martin,

and Rebecca Simmons. 1986. Conditions of Learning in Novice Programmers. *Journal of Educational Computing Research*2, 1 (1986), 37–55. DOI:http://dx.doi.org/10.2190/gujt-jcbj-q6qu-q9pl

[21] Michael Mccracken et al.2001. A multi-national, multi-institutional study of assessment of programming skills of first-year CS students. *Working group reports from ITiCSE on Innovation and technology in computer science education - ITiCSE-WGR 01*(2001). DOI:http://dx.doi.org/10.1145/572133.572137

[22] W. Lewis Johnson-Elliot Soloway and W. L. Johnson. 1984. Intention-based diagnosis of programming errors. *Proceedings of the 5th National Conference on Artificial Intelligence, Austin, Texas*.

[23] H.Chad Lane and Kurt Vanlehn. 2005. Intention-based Scoring: an approach to measuring success at solving the composition problem. *Proceedings of the 36th SIGCSE technical symposium on Computer science education - SIGCSE 05*(2005). DOI:http://dx.doi.org/10.1145/1047344.1047471

[24] Titus Winters and Tom Payne. 2005. What do students know?: an outcomes-based assessment system. *Proceedings of the 2005 international workshop on Computing education research - ICER 05*(2005). DOI:http://dx.doi.org/10.1145/1089786.1089802

[25] Matthew Jadud. 2006. An exploration of novice compilation behaviour in BlueJ (Doctoral dissertation, University of Kent).

[26] Bill Marion, John Impagliazzo, Caroline St. Clair, Barry Soroka, and Deborah Whitfield. 2007. Assessing computer science programs: What we have learned. *Proceedinds of the 38th SIGCSE technical symposium on Computer science education - SIGCSE 07*(2007). DOI:http://dx.doi.org/10.1145/1227310.1227358

[27] Marcelo Worsley and Paulo Blikstein. 2013. Programming Pathways: A Technique for Analyzing Novice Programmers' Learning Trajectories. *Lecture Notes in Computer Science Artificial Intelligence in Education*(2013), 844–847. DOI:http://dx.doi.org/10.1007/978-3-642-39112-5_127

[28] John Hattie and Helen Timperley. 2007.The power of feedback. *Review of educational research* 77, 1, 81-112.

[29] Sue Helme and David Clarke. 2001. Identifying cognitive engagement in the mathematics classroom. *Mathematics Education Research Journal* 13, 2 (2001), 133–153. DOI:http://dx.doi.org/10.1007/bf03217103

[30] Rachel Cardell-Oliver. 2011. How can software metrics help novice programmers?. I*Proceedings of the Thirteenth Australasian Computing Education Conference-Volume* 114, 55-62. Australian Computer Society, Inc.

[31] Andrew Petersen, Jaime Spacco, and Arto Vihavainen. 2015. An exploration of error quotient in multiple contexts. *Proceedings of the 15th Koli Calling Conference on Computing Education Research - Koli Calling 15*(2015). DOI:http://dx.doi.org/10.1145/2828959.2828966

[32] Christopher Watson, Frederick W.b. Li, and Jamie L. Godwin. 2013. Predicting Performance in an Introductory Programming Course by Logging and Analyzing Student Programming Behavior. *2013 IEEE 13th International Conference on Advanced Learning Technologies*(2013). DOI:http://dx.doi.org/10.1109/icalt.2013.99

[33] Leigh Ann Sudol, Kelly Rivers, and Thomas K. Harris. 2012. Calculating Probabilistic Distance to Solution in a Complex Problem Solving Domain. *International Educational Data Mining Society* (2012*)*.

[34] Adam S. Carter, Christopher D. Hundhausen, and Olusola Adesope. 2015. The Normalized Programming State Model: Predicting Student Performance in Computing Courses based on Programming Behavior. *Proceedings of the eleventh annual International Conference on International Computing Education Research - ICER 15*(2015). DOI:http://dx.doi.org/10.1145/2787622.2787710

[35] Brett A. Becker. 2016. A New Metric to Quantify Repeated Compiler Errors for Novice Programmers. *Proceedings of the 2016 ACM Conference on Innovation and Technology in Computer Science Education - ITiCSE 16*(2016). DOI:http://dx.doi.org/10.1145/2899415.2899463

[36] Matthew Johnson, Michael Eagle, John Stamper, and Tiffany Barnes. 2013. An algorithm for reducing the complexity of interaction networks. *Educational Data Mining 2013*.

[37] Raymond Pettit, Ryan Clements, and Susan Mengel. 2012. A Process for Collecting and Analyzing Chronological Data for CS1 Programming Assignments. *Proceedings of the International Conference on Frontiers in Education: Computer Science and Computer Engineering (FECS)*, 1.

[38] Marzieh Ahmadzadeh, Dave Elliman, and Colin Higgins. 2005. An analysis of patterns of debugging among novice computer science students. *Proceedings of the 10th annual SIGCSE conference on Innovation and technology in computer science education - ITiCSE 05*(2005). DOI:http://dx.doi.org/10.1145/1067445.1067472

[39] Jaime Spacco et al.2015. Analyzing Student Work Patterns Using Programming Exercise Data. *Proceedings of the 46th ACM Technical Symposium on Computer Science Education - SIGCSE 15*(2015). DOI:http://dx.doi.org/10.1145/2676723.2677297

[40] Cindy Norris, Frank Barry, James B.Fenwick Jr., Kathryn Reid, and Josh Rountree. 2008. ClockIt: collecting quantitative data on how beginning software developers really work. *ACM SIGCSE Bulletin*40, 3 (2008), 37. DOI:http://dx.doi.org/10.1145/1597849.1384284

[41] Ulrich Kiesmueller, Sebastian Sossalla, Torsten Brinda, and Korbinian Riedhammer. 2010. Online identification of learner problem solving strategies using pattern recognition methods. *Proceedings of the fifteenth annual conference on Innovation and technology in computer science education - ITiCSE 10*(2010).

DOI:http://dx.doi.org/10.1145/1822090.1822167

[42] Roya Hosseini, Arto Vihavainen, and Peter Brusilovsky. 2014. Exploring problem solving paths in a Java programming course, 65-76.

[43] Juha Helminen, Petri Ihantola, Ville Karavirta, and Lauri Malmi. 2012. How do students solve parsons programming problems? an analysis of interaction traces. *Proceedings of the ninth annual international conference on International computing education research - ICER 12*(2012). DOI:http://dx.doi.org/10.1145/2361276.2361300

[44] Anthony Allevato and Stephen H. Edwards. 2010. Discovering patterns in student activity on programming assignments. *2010 ASEE Southeastern Section Annual Conference and Meeting*.

[45] Chris Piech, Mehran Sahami, Daphne Koller, Steve Cooper, and Paulo Blikstein. 2012. Modeling how students learn to program. In Proceedings of the 43rd ACM technical symposium on Computer Science Education (SIGCSE '12). ACM, New York, NY, USA, 153-160. DOI: https://doi.org/10.1145/2157136.2157182

[46] Paulo Blikstein. 2011. Using learning analytics to assess students behavior in open-ended programming tasks. *Proceedings of the 1st International Conference on Learning Analytics and Knowledge - LAK 11*(2011). DOI:http://dx.doi.org/10.1145/2090116.2090132

[47] James B. Fenwick, Cindy Norris, Frank E. Barry, Josh Rountree, Cole J. Spicer, and Scott D. Cheek. 2009. Another look at the behaviors of novice programmers. *Proceedings of the 40th ACM technical symposium on Computer science education - SIGCSE 09*(2009). DOI:http://dx.doi.org/10.1145/1508865.1508973

[48] Aivar Annamaa, Annika Hansalu, and Eno Tonisson. 2015. Automatic analysis of students' solving process in programming exercises. *IFIP TC3 Working Conference "A New Culture of Learning: Computing and next Generations"*, 11.

[49] Juho Leinonen, Krista Longi, Arto Klami, and Arto Vihavainen. 2016. Automatic Inference of Programming Performance and Experience from Typing Patterns. *Proceedings of the 47th ACM Technical Symposium on Computing Science Education - SIGCSE 16*(2016). DOI:http://dx.doi.org/10.1145/2839509.2844612

[50] Emily S.Tabanao, Ma Mercedes T. Rodrigo, and Matthew C. Jadud. 2008. Identifying at-risk novice java programmers through the analysis of online protocols." In *Philippine Computing Science Congress*, 1-8.

[51] Emily S. Tabanao, Ma. Mercedes T. Rodrigo, and Matthew C. Jadud. 2011. Predicting at-risk novice Java programmers through the analysis of online protocols. *Proceedings of the seventh international workshop on Computing education research - ICER 11*(2011). DOI:http://dx.doi.org/10.1145/2016911.2016930

[52] Irena Koprinska, Joshua Stretton, and Kalina Yacef. 2015. Students at Risk: Detection and Remediation. *Educational Data Mining*, 512-515.

[53] Arto Vihavainen, Matti Luukkainen, and Petri Ihantola. 2014. Analysis of source code snapshot granularity levels. *Proceedings of the 15th Annual Conference on Information technology education - SIGITE 14*(2014). DOI:http://dx.doi.org/10.1145/2656450.2656473

[54] Petri Ihantola, Arto Vihavainen, Alireza Ahadi, Matthew Butler, Jürgen Börstler, Stephen H. Edwards, Essi Isohanni, Ari Korhonen, Andrew Petersen, Kelly Rivers, Miguel Ángel Rubio, Judy Sheard, Bronius Skupas, Jaime Spacco, Claudia Szabo, and Daniel Toll. 2015. Educational Data Mining and Learning Analytics in Programming: Literature Review and Case Studies. In Proceedings of the 2015 ITiCSE on Working Group Reports (ITICSE-WGR '15). ACM, New York, NY, USA, 41-63. DOI: https://doi.org/10.1145/2858796.2858798

[55] Stephen H. Edwards and Manuel A. Perez-Quinones. 2008. Web-CAT: automatically grading programming assignments. *ACM SIGCSE Bulletin*40, 3 (2008), 328. DOI:http://dx.doi.org/10.1145/1597849.1384371

[56] Philip Johnson et al.2003. Beyond the Personal Software Process: Metrics collection and analysis for the differently disciplined. *25th International Conference on Software Engineering, 2003. Proceedings.*(2003). DOI:http://dx.doi.org/10.1109/icse.2003.1201249

[57] Jaime Spacco, David Hovemeyer, William Pugh, Fawzi Emad, Jeffrey K. Hollingsworth, and Nelson Padua-Perez. 2006. Experiences with marmoset: designing and using an advanced submission and testing system for programming courses. *Proceedings of the 11th annual SIGCSE conference on Innovation and technology in computer science education - ITICSE 06*(2006). DOI:http://dx.doi.org/10.1145/1140124.1140131

[58] Matthew C. Jadud. 2006. Methods and tools for exploring novice compilation behaviour. *Proceedings of the 2006 international workshop on Computing education research - ICER 06*(2006). DOI:http://dx.doi.org/10.1145/1151588.1151600

[59] MH Ng Cheong Vee, Bertrand Meyer, and Keith L. Mannock. 2006. Empirical study of novice errors and error paths in object oriented programming. *Proceedings of the 7th Annual HEA-ICS Conference*.

[60] Anthony Allevato, Matthew Thornton, Stephen Edwards, and Manuel Perez-Quinones. 2008. Mining data from an automated grading and testing system by adding rich reporting capabilities. *Educational Data Mining 2008*.

[61] Jussi Kasurinen and Uolevi Nikula. 2009. Estimating programming knowledge with Bayesian knowledge tracing. *ACM SIGCSE Bulletin*41, 3 (2009), 313. DOI:http://dx.doi.org/10.1145/1595496.1562972

[62] Ma. Mercedes T. Rodrigo et al.2009. Affective and behavioral predictors of novice programmer achievement. *Proceedings of the 14th annual ACM SIGCSE conference on Innovation and technology in computer science education - ITiCSE 09*(2009).

DOI:http://dx.doi.org/10.1145/1562877.1562929

[63] Ma. Mercedes T. Rodrigo and Ryan S.j.d. Baker. 2009. Coarse-grained detection of student frustration in an introductory programming course. *Proceedings of the fifth international workshop on Computing education research workshop - ICER 09*(2009). DOI:http://dx.doi.org/10.1145/1584322.1584332

[64] Maria Mercedes T.Rodrigo, Emily Tabanao, Ma Beatriz E. Lahoz, and Matthew C. Jadud. 2009. Analyzing online protocols to characterize novice java programmers. *Philippine Journal of Science*138, 2(2009), 177-190.

[65] Teemu Sirkiä and Juha Sorva. 2012. Exploring programming misconceptions: an analysis of student mistakes in visual program simulation exercises. *Proceedings of the 12th Koli Calling International Conference on Computing Education Research - Koli Calling 12*(2012). DOI:http://dx.doi.org/10.1145/2401796.2401799

[66] Jaime Spacco, Davide Fossati, John Stamper, and Kelly Rivers. 2013. Towards improving programming habits to create better computer science course outcomes. *Proceedings of the 18th ACM conference on Innovation and technology in computer science education - ITiCSE 13*(2013). DOI:http://dx.doi.org/10.1145/2462476.2465594

[67] Yoshiaki Matsuzawa, Ken Okada, and Sanshiro Sakai. 2013. Programming process visualizer: a proposal of the tool for students to observe their programming process. *Proceedings of the 18th ACM conference on Innovation and technology in computer science education - ITiCSE 13*(2013). DOI:http://dx.doi.org/10.1145/2462476.2462493

[68] Juha Helminen, Petri Ihantola, and Ville Karavirta. 2013. Recording and analyzing in-browser programming sessions. *Proceedings of the 13th Koli Calling International Conference on Computing Education Research - Koli Calling 13*(2013). DOI:http://dx.doi.org/10.1145/2526968.2526970

[69] Kenny Heinonen, Kasper Hirvikoski, Matti Luukkainen, and Arto Vihavainen. 2014. Using CodeBrowser to seek differences between novice programmers. *Proceedings of the 45th ACM technical symposium on Computer science education - SIGCSE 14*(2014). DOI:http://dx.doi.org/10.1145/2538862.2538981

[70] Emily Gatchalian. 2014. A Comparative Analysis of Intention-Based Score Trajectories and Update Characteristics Trajectories Among Novice Programming Students. Master Thesis. Ateneo de Manila University.

[71] Petri Ihantola, Juha Sorva, and Arto Vihavainen. 2014. Automatically detectable indicators of programming assignment difficulty. *Proceedings of the 15th Annual Conference on Information technology education - SIGITE 14*(2014). DOI:http://dx.doi.org/10.1145/2656450.2656476

[72] Paulo Blikstein, Marcelo Worsley, Chris Piech, Mehran Sahami, Steven Cooper, and Daphne Koller. 2014. Programming Pluralism: Using Learning Analytics to Detect Patterns in the Learning of Computer Programming. *Journal of the Learning Sciences*23, 4 (2014), 561–599. DOI:http://dx.doi.org/10.1080/10508406.2014.954750

[73] Arto Vihavainen, Juha Helminen, and Petri Ihantola. 2014. How novices tackle their first lines of code in an IDE: an analysis of programming session traces. *Proceedings of the 14th Koli Calling International Conference on Computing Education Research - Koli Calling 14*(2014). DOI:http://dx.doi.org/10.1145/2674683.2674692

[74] Raymond Pettit, John Homer, Roger Gee, Susan Mengel, and Adam Starbuck. 2015. An Empirical Study of Iterative Improvement in Programming Assignments. *Proceedings of the 46th ACM Technical Symposium on Computer Science Education - SIGCSE 15*(2015). DOI:http://dx.doi.org/10.1145/2676723.2677279

[75] Amjad Altadmri and Neil C.c. Brown. 2015. 37 Million Compilations: Investigating Novice Programming Mistakes in Large-Scale Student Data. *Proceedings of the 46th ACM Technical Symposium on Computer Science Education - SIGCSE 15*(2015). DOI:http://dx.doi.org/10.1145/2676723.2677258

[76] Alireza Ahadi, Raymond Lister, Heikki Haapala, and Arto Vihavainen. 2015. Exploring Machine Learning Methods to Automatically Identify Students in Need of Assistance. *Proceedings of the eleventh annual International Conference on International Computing Education Research - ICER 15*(2015). DOI:http://dx.doi.org/10.1145/2787622.2787717

[77] Leigh Ann Sudol-DeLyser. 2014. AbstractTutor: Increasing Algorithm Implementation Expertise for Novices Through Algorithmic Feedback. PhD Dissertation, Carnegie Mellon University.

Philippine Information Technology Journal, Vol. 11, No. 2, July - December 2018

Automatic Recognition of Music Emotion on the Lyrics using Multinomial Naïve Bayes

Courtney Love A. Peñaflorida
University of San Jose- Recoletos
Magallanes St., Cebu City
clpenaflorida@gmail.com
0998-967-9734

Leah Joy L. Ylaya
University of San Jose- Recoletos
Magallanes St., Cebu City
ljylaya1996@gmail.com
0932-376-2937

Eric V. Magto
University of San Jose- Recoletos
Magallanes St., Cebu City
evm@usjr.edu.ph
0943-838-9183

Gregg Victor D. Gabison
University of San Jose- Recoletos
Magallanes St., Cebu City
gregg@usjr.edu.ph
0917-328-4941

Abstract

This paper will present the application of Multinomial Naïve Bayes to determine the mood of the songs by analysing its lyrics. A dataset which comprises 350 songs with its respective lyrics, evenly divided into 14 moods (25 songs per mood) has been used during the training phase. The results shows 54% accuracy given that out of 140 songs that served as the test set, only 76 were correctly labelled using the algorithm.

1. Introduction

Music has always been an interesting matter to deal with especially in the field of computer science. And with the large amount of Music Streaming Services on the web, users has access to large collection of music. Users want to customize the music playlist they want to listen. Thus, they make ways to find for music that they preferred to listen.

Various properties which are present on the music such as genre, year as well as the number of times the song has been played (play count) are used to implement music recommendation or automatic generation of playlist [1]. The idea is to find songs which are similar to each other based on the various information present on the music.

Mood is one of the properties which are also considered by users in searching for songs or creating a playlist. A number of researches had been attempted to determine which mood is associated with the song. Music mood recognition is the process where emotions are identified by means of analysing audio or examining the lyric content of the song. There are a number of studies that presents mood classification be means of analysing features that are extracted by audio signals. Usually mood of the song is determined by analysing audio and lyrics of the song. However it is believed that even if these information are related to each other, text information is still processed by the brain in independent manner. Furthermore, text does not only convey informative contents, but also expresses emotional states [2]. Thus, evaluating the lyrics of the song to classify its mood may be a good opportunity to develop the progress of music analysis.

The group of Xiao Hu et al [3] compared the performance of using various features to classify the mood of the song. They examined the impact of lyric features on music mood classification by comparing lyric features, audio features hybrid (lyrics and audio with features selection and hybrid features generated by three feature selection methods. They made use of Support Vector Machine (SVM) to classify 5,585 songs in 18 mood categories. Their study showed that evaluated lyric features outperformed the audio features on cases where the semantic meanings taken from lyrics tied well to the mood category. A study from Zaanen et al [1] made use of the lyrics to classify the mood of the songs as well. They made use of a two-dimensional plane with four areas, dividing the plane into 4 parts which represents Thayer's arousal and valence plane. The features were described using TF*IDF to tell the importance of a word to the respective classes (mood).

This study aims to determine the mood of the song by analysing its lyrics with the application of Multinomial Naïve Bayes as the classifier. This paper is organized with the following sections: Section 2 presents the list of classes (mood) being used for the research as well as machine learning process. Section 3 covers the results and analysis. Section 4 presents to the summary of findings of the study.

2. Methodology

This section will present the conceptual framework, the classes and the machine learning that is being used in the study.

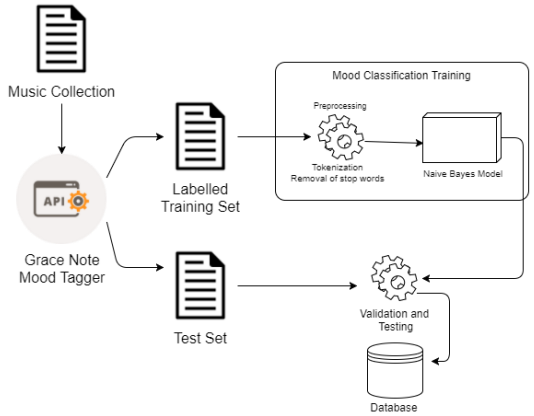


Figure 1: Conceptual Framework

2.1 Collection of Data

The machine learning process relies on the song title, mood and the lyrics to perform. The list of songs which will be used as the dataset and the test sets are taken from the Million Song Dataset [4]. Each song is labelled with its respective lyrics using SongLyrics.com [5] and tagged with the corresponding mood using GraceNote Web API [6].

mood	count
aggressive	25
brooding	25
cool	25
defiant	25
easygoing	25
empowering	25
fiery	25
lively	25
romantic	25
rowdy	25
sensual	25
somber	25
sophisticated	25
urgent	25

Figure 2: List of moods and its respective number of songs.

Grace Note API has a rich set of music metadata by which developers can look up into. This study made use of Grace Note as the expert system that labels each music with the corresponding mood. During the collection process, 350 songs (25 songs per mood) are manually chosen to serve as the training set and another 140 songs to serve as the test set during the testing phase. The number of songs available for each mood are evenly controlled to ensure that there will be no potential bias that may happen during the machine learning process. Furthermore, the study has chosen songs that were labelled by the Grace Note to single mood only and did not fall to another category.

2.3 Machine Learning Model

This study made use of Multinomial Naïve Bayes, of supervised machine learning algorithm that predicts class membership probabilities, such as the probability that a given sample belongs to a particular class [7]. The lyrics underwent pre-process (removing stop words and stemming) before creating the model.

$$c_{MAP} = \text{argmax}_c \hat{P}(c) \prod_i \hat{P}(x_i \mid c)$$

$$P(c) = \frac{N_c}{N} \qquad P(w \mid c) = \frac{\text{count}(w,c) + 1}{\text{count}(c) + |v|}$$

Figure 3: Formula in choosing the mood, with the priors and conditional probabilities.

The figure above shows the formula used in choosing the mood, where *C* is the mood to be evaluated, *P(c)* is the prior of mood *C and P* $(x_i|c)$ are the conditional probabilities of mood *C* from the words on the song *X*. In getting the prior for each of the mood, the number of songs (N_c) of mood *c* will be divided by *N*, which is the total number of moods on the training set. In getting the conditional property for every word (*w*) in every mood (*c*), the count of word *w* which is under the mood [8] c will be divided by the total number of words present under mood *c* added with the number of distinct words $|V|$. The count of word *w* on mood c is always added by 1. This serves as the Laplace smoothing method to avoid instances of getting zero as a result [8].

	Song	Lyrics	Mood
Training	1	friend talk friend	empowering
	2	friend friend need	empowering
	3	friend day	empowering
	4	pain eye friend	defiant
Test	5	friend friend friend pain eye	???

$$P(\text{empowering}) = \frac{3}{4} \qquad P(\text{friend} \mid \text{empowering}) = \frac{6+1}{8+6}$$

$$P(\text{defiant}) = \frac{1}{4} \qquad P(\text{pain} \mid \text{empowering}) = \frac{1+1}{8+6}$$

$$P(\text{eye} \mid \text{empowering}) = \frac{1+1}{8+6}$$

$$P(\text{friend} \mid \text{defiant}) = \frac{1+1}{3+6}$$

$$P(\text{pain} \mid \text{defiant}) = \frac{1+1}{3+6}$$

$$P(\text{eye} \mid \text{defiant}) = \frac{1+1}{3+6}$$

$$P(\text{empowering} \mid \text{song 5}) = 3/4 * (3/7) * 1/14 * 1/14 = 0.0003$$

$$P(\text{defiant} \mid \text{song 5}) = 1/4 * (2/9) * 2/9 * 2/9 = 0.0001$$

Figure 4: Choosing the appropriate mood for "song 5".

Figure 4 above shows an application of the Multinomial Naïve Bayes formula. The lyrics of the song serves as features that defines the song. The priors and the conditional probabilities make up the model for

the classification. In choosing for the mood of the song, calculate the probability of the mood given the song by multiplying all conditional probabilities of the words found as well as the song's prior. The song will be labelled the largest calculated mood value.

$$c_{NB} = \underset{c_j \in C}{\operatorname{argmax}} \log P(c_j) + \sum_{i \in positions} \log P(x_i \mid c_j)$$

Figure 5: Formula for choosing a mood that uses logarithm to avoid floating underflow.

As shown in figure 4, many conditional probabilities are multiplied. This may result in floating underflow. In programming, a floating underflow occurs happens when the calculation gets smaller than the computer can actually store in the memory [8]. This results to a NaN or a non-representable data type kind of a value. To address this issue, adding logarithms of probabilities is being performed instead of multiplying probabilities. Finding the maximum value from the added log weights will determine the mood of the song [8].

3. Results and Discussion

3.1 Accuracy Testing

		Predicted															
	Number of songs ⬇ 140	1 Aggressive	2 Brooding	3 cool	4 defiant	5 easygoing	6 empowering	7 fiery	8 lively	9 romantic	10 rowdy	11 sensual	12 somber	13 sophisticated	14 urgent	precision	recall
1	Aggressive	10														71.43%	100%
2	Brooding	0	9									1				81.82%	90%
3	cool	1		6	1		1					1				37.50%	60%
4	defiant	1	1	1	4		2						1			44.44%	40%
5	easygoing	0				8	1			1						33.33%	80%
6	empowering		1	1	1	1	4			1		1				19.05%	40%
7	fiery				1			7		1				1		100.00%	70%
8	lively			3					7							77.78%	70%
9	romantic						1			8	1					61.54%	80%
10	rowdy			1			2		1		2	1	1	2		50.00%	20%
11	sensual			2	1	1	5						1			0.00%	0%
12	somber	2			1	2	2			2	1					0.00%	0%
13	sophisticated													10		76.92%	100%
14	urgent			2			3		1			2	1		1	100.00%	10%
	Total Predicted	14	11	16	9	12	21	7	9	13	4	6	4	13	1		
	Accuracy	54%															

Actual

Figure 6: Results of the accuracy test represented using confusion matrix.

The figure above shows the results of the testing conducted. 140 songs, evenly distributed among 14 moods (10 songs per mood) are used as test sets to match with the results from mood classification performed by the system. This shows that the classifier performed 54% accuracy. As shown on the table, the classifier were biased towards Empowering and Cool. Furthermore, although the classifier achieved 100% recall in labelling Aggressive and Sophisticated, Sensual and Somber achieved zero in both precision and recall.

4. Summary of Findings

Over the course of this research there are limitations that are discovered, especially in analysing the lyrics of the song. When it comes to text analysis, number of words on a given document is crucial during the computation process. On analysing a song on the other hand, there are cases that a song does not have a definite ending (eg. continuous chorus until fade). This may affect the results during the computation process. Furthermore, lyrics may tend to have words that are not included to the dictionary, yet cannot be captured as a stop words (eg. woooo, yeeaaaaah). Nevertheless, during the tests that involves text analysis, the results yield such results.

Over the course of testing the mood classification, it is observed that accuracy rating is 54%. It is observed that the amount of songs that are used as training set may affect the accuracy of the classification process. The decision of converting the words into its base form (stemming) as well as removing the stop words may have an effect of the results as well. Although the researchers are yet to compare the results coming from this research and to the same experiment but not removing stop words and ignoring the stemming during the pre-processing phase.

Bibliography

[1] M. v. Zaanen and P. Kanters, "Automatic Mood Classification Using TF*IDF Based on Lyrics," *International Society for Music Information Retrieval*, p. 1, 2010.

[2] C. O. Alm, D. Roth and R. Sproat, "Emotions from text: machine learning for text-based emotion prediction," *Proceedings of Human Language Technology Conference and Conference on Empirical Methods in Natural Language*, p. 579–586, 2005.

[3] X. Hu, S. J. Downie and A. F. Ehmann, "Lyric Text Mining in Music Mood Classification," in *10th International Society for Music Information Retrieval Conference (ISMIR 2009)*, International Music Information Retrieval Systems Evaluation Laboratory, University of Illinois at Urbana-Champaign, 2009.

[4] "The Million Song Dataset," [Online]. Available: https://labrosa.ee.columbia.edu/millionsong/.

[5] "SongLyrics," [Online]. Available: http://www.songlyrics.com/.

[6] "Grace Note Web API," [Online]. Available: https://developer.gracenote.com/web-api.

[7] K. M. Leung, "Naive Bayesian Classifier," 2007.

[8] D. Jurafsky, "CS 124: From Languages to Information," [Online]. Available: https://web.stanford.edu/class/cs124/lec/naivebayes.pdf. [Accessed 2017].

[9] A. M. Kibriya, E. Frank, B. Pfahringer and G. Holmes, "Multinomial Naive Bayes for Text Categorization Revisited," vol. Vol. 3339, pp. pp. 488-499, Decemver 2004.

DAGIT

An All-in-One Directory, Navigation, Traffic, and Parking

Advisory Mobile Application for Dumaguete City

Joy M. Dy
College of Computer Studies
Silliman University
Dumaguete City, Philippines
joymdy@su.edu.ph

Justin Carlos A. Malana
College of Computer Studies
Silliman University
Dumaguete City, Philippines
justinamalana@su.edu.ph

Dave E. Marcial
College of Computer Studies
Silliman University
Dumaguete City, Philippines
demarcial@su.edu.ph

Alyssa L. Palencia
College of Computer Studies
Silliman University
Dumaguete City, Philippines
alyssalpalencia@su.edu.ph

Clarice Pocot
College of Computer Studies
Silliman University
Dumaguete City, Philippines
claricepocot@su.edu.ph

ABSTRACT

This research aimed at developing an application that will aid the traffic management system of Dumaguete City called DAGIT. Two cross-platform mobile applications for Android and iOS were developed for the General Users and OnField Traffic Management Officers. A web application was developed for the Traffic Management Office (TMO). The General User mobile application allows them to access a map of the city, a directory of social and emergency services, receive notifications and announcements, and send messages and accident, violation reports to the Desk TMO. The On-Field TMO mobile application allows them to rate the traffic condition and update the parking availability at their assigned location. The Desk TMO web application allows them to access a map of the city, manage TMO accounts, manage the directory, publish notifications, receive accident, violation reports, and reply to messages sent by the General Users.

After development, testing was conducted using three test cases with 26 respondents consisting of an office employee from the TMO, five on-field officers, and 20 random respondents. All critical functions passed. Thus, DAGIT is a reliable all-in-one directory, navigation guide, traffic, and parking advisory mobile application for Dumaguete City. It is recommended that DAGIT will be fully implemented.

CCS CONCEPTS

• Human-centered computing → Ubiquitous and mobile computing → Ubiquitous and mobile devices → Smartphones

KEYWORDS

Traffic Management, Mobile Development, Dumaguete City

1 INTRODUCTION

Today's economy is fast leaping forward both locally and globally. As more platforms for economic growth continue to arise, comes with it are great opportunities for economies, both small and big, to expand.

Dumaguete, ranked fifth on Forbes' list of best retirement cities in the world, is becoming an economically vibrant city in the country. The city's growing industry is vastly diversified, thus facilitating its economic growth.

With economic development comes the issue of road congestion due to traffic. According to Julius Caesar Parreñas [1], coordinator of the Asia-Pacific Infrastructure Partnership and Asia-Pacific Financial Forum, traffic is a symptom of a growing economy. Specifically, traffic causes massive delays in productivity, wastage of time, energy, and financial resources. An average Metro Manila resident spends 1,000 hours on congested roads annually, summing up to around 28,000 hours of his economic life. [2] In addition, a report by the Japan International Cooperation Agency showed that the cost of traffic in the Philippines would continue to rise and, despite economic growth, stands to lose up to ₱6B a day by 2030 if adequate solutions will not be implemented. [3]

Road traffic in Dumaguete is becoming a challenge to its development. The city's narrow streets are being used by all types

of vehicles numbering more than 660,184 in just a span of 12 hours daily, particularly during rush hours. [4] With the city's continuous progress, the number of vehicles would most likely increase. Moreover, the current road network in the city is incapable of meeting the requirements of the increasing number of vehicles, thus leading to the congestion problem. Additionally, the city's narrow streets are affected by issues regarding sidewalk vendors, incorrect parking, traffic accidents, construction works, and organized public gatherings that are critical reasons for the congestion problem. [5] Leaving the traffic congestion problem unmanaged might create a negative impact to the city's economic growth, hence the need for effective traffic management is necessary.

Having good and efficient traffic management has the potential to benefit road users, businesses, and the community. Apart from saving financial, time, and energy resources, productivity is also uninterrupted. Moreover, having good traffic management facilitates faster emergency response for the public. As the city's economy continues to boom, it is essential to leave a good impression for possible investors.

With the advancement of today's technology, often it has been advantageous in improving traffic management in many places. There are existing applications that are widely used in monitoring road conditions. While many people already have access to smartphones, using this technology to improve traffic management would most likely be efficient. With this, the group came up with DAGIT, an all-in-one mobile application that would serve as a directory, navigation guide, traffic, and parking advisory exclusively for Dumaguete City. The application's name was taken from the term dagit, by which the name of the city was coined from, meaning "to snatch." Through the application, drivers, commuters, and the public can quickly snatch information regarding traffic conditions and other navigation information needed to traverse the city. Unlike other mobile traffic applications that use crowdsourcing to update their data, DAGIT has a separate interface exclusively for Traffic Management Officers to access and update traffic conditions and parking availability in the city. Hence, data manipulation and update are easy to control. Additionally, information broadcasted by the application is reliable. With the coordination of the public and city authorities, this innovation hopes to improve and bring efficiency to the city's traffic management.

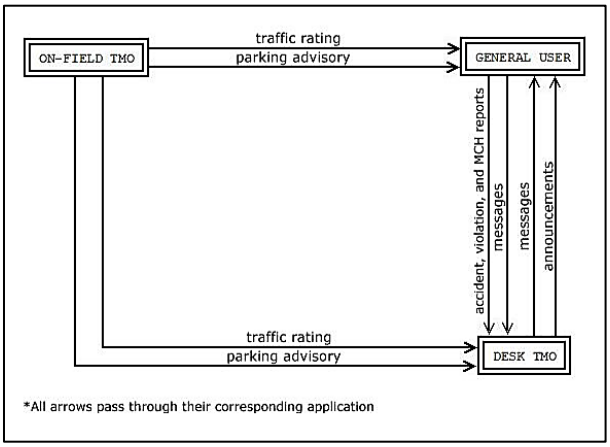


Figure 1. Application's Flow of Information

2 REVIEW OF RELATED SYSTEMS

Waze. Waze is a GPS-based mobile application with traffic monitoring and navigation as its main functionalities. The application, with the help of active community users, shares realtime traffic and road information. Like Waze, DAGIT functions as a traffic monitoring and navigation application. However, DAGIT uses traffic enforcers as the primary medium for sharing real-time information rather than primarily gathering information form General Users.

MMDA Traffic Navigator v2.0. MMDA Traffic Navigator v2.0 is a mobile application that updates users with the latest advisories about Metro Manila in real-time. The application provides information on the current traffic situations of various roads whether they are Light, Medium, or Heavy. It can also locate the nearest possible police stations, hospitals, and fire stations in the major cities and municipalities of Metro Manila. Like MMDA Traffic Navigator v2.0, DAGIT provides General Users with realtime traffic information and a directory that contains the address, operating hours, and contact information of hospitals, fire stations, police stations, and terminals in Dumaguete.

3 METHODOLOGY

This section contains the design details of the application.

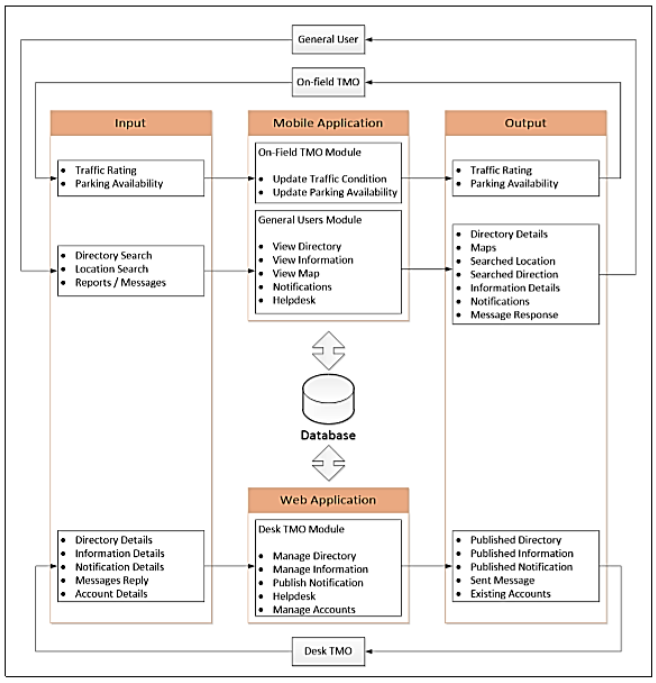


Figure 2. Conceptual Diagram

As shown in Figure 2, the Desk TMO creates and inputs the account details of the On-Field TMO in the web application to register their accounts into the system. The Desk TMO can add, edit, delete, enable, and disable an On-Field TMO account that is stored in the database. This is done through a module called Manage Account. Once an On-Field TMO account is created, the On-Field TMO logs in to the OnField TMO mobile application with the credentials set by the Desk TMO. Once logged in, the On-Field TMO can update the traffic rating and parking availability at their assigned street or intersection. Once the On-Field TMO updates the traffic rating and/or parking availability, the General User can view these advisories through the General User mobile application. Meanwhile, the General User registers using an email account through the General User mobile application and the account details are stored in the database. Once the account is set-up, the DAGIT: An All-in-One Directory, Navigation Guide, Traffic, and Parking Advisory Mobile Application for Dumaguete City March, 2018, Dumaguete City, Negros Oriental General User can access the application features. The General User can view the directory, search locations and directions on the map, and send messages and reports to the Desk TMO. Additionally, the Desk TMO can manage the directory. The Desk TMO can add, edit, and delete information found in the directory. Like the directory, the Desk TMO can manage information that can be viewed by the General User. This information can also be added, edited, and deleted by the Desk TMO. Furthermore, the Desk TMO can receive messages and reports from the General User through the helpdesk feature. The Desk TMO can respond to messages sent by the General User. Lastly, the Desk TMO can publish notifications that are received by the General User through notifications.

3.2 Software Development Model

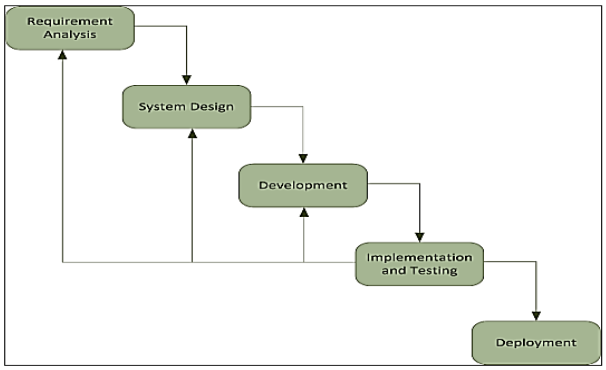


Figure 3. Software Development Model

As shown in Figure 3, the software development model used in the project was the waterfall model. Through this, the team ensured that each phase was completed before proceeding to the next step of the development process. The development process was divided into Requirements Analysis, System Design, Development, Implementation and Testing, and Deployment. In detail, the activities in each phase are as follows:

Phase 1. Requirements Analysis included the gathering of facts from the end users to identify the needs and expectations to be implemented in the project. Specifically, the team researched traffic in Dumaguete City. After having enough knowledge in the matter, the team presented the research and system proposal to Atty. Cordova. He confirmed that the causes of traffic stated in this paper are true. Moreover, he added that the city has been exerting effort to continue addressing the issue, specifically the implementation of the new traffic scheme and the soon-to-beimplemented parking boxes on the side of the streets. He expressed his wholehearted support for the project as well as suggestions to improve it. All these were then analyzed and documented in a requirement specifications document.

Phase 2. System Design took place after analyzing facts in the requirement specifications. In the system design, the developers worked on the data flow diagram, database design, and screen layouts to have an initial idea of the overall system architecture to be developed.

Phase 3. Development proceeded after the system architecture was laid out. In this phase, the design was translated into codes. Development was done by the module. The developed modules met the requirements specified in the requirement specifications document.

Phase 4. Implementation and Testing followed after coding all modules. In this phase, the requirement specifications document

was used as a guide to check whether all specifications were well integrated into the developed system. Testing was divided into sub-phases namely the Alpha Stage, Beta Stage, and Live Stage. In the Alpha Stage, the developed project underwent Functional Testing and Integration Testing. In Functional Testing, it was ensured that the specified functionalities stated in the system requirements worked. In Integration Testing, it was ensured that the interaction between software and hardware was tested. In the Beta Stage, the developed project underwent Usability Testing and Acceptance Testing. In Usability Testing, the TMO was able to evaluate the user interface of the developed project. Acceptance Testing was done with the TMO to ensure the developed project met the requirements and worked as expected. Lastly, in the Live Stage, the developed project underwent Performance Testing and System Testing. In Performance Testing, the speed and effectiveness of the developed project were assessed to ensure it was generating results within a specified time as stated in the performance requirements. In System Testing, the developed project was tested as a full system in an implementation environment. Errors and bugs detected during the different testing stages were corrected.

Phase 5. The deployment took place after testing. The application passed all critical functions and is now ready to be deployed to end users for actual use.

3.4 Screen Layouts

DAGIT composes of three applications, a cross-platform mobile application to be used by the on-field TMO, another cross-platform mobile application to be used by the general users, and a web application to be used by the officer in the traffic management office.

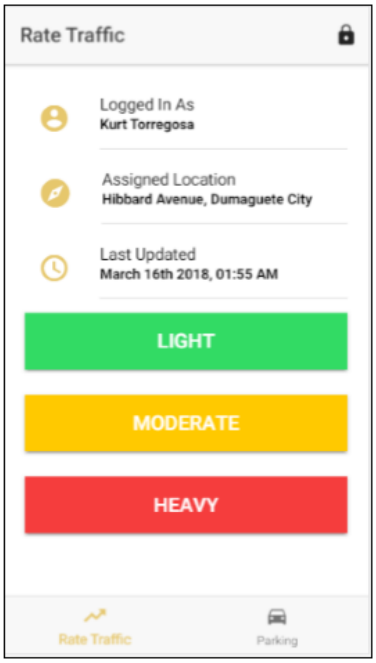


Figure 4. Rate Traffic Screen

Figure 4 shows the rate traffic screen of the on-field TMO mobile application. The currently logged in On-Field TMO's name, assigned location, and the date and time the traffic rating was last updated are displayed on the upper part of the screen. They can update the traffic condition as Light, Moderate, or Heavy by tapping the corresponding button. They can tap the "Parking" option on the bottom to go the Update Parking Availability screen.

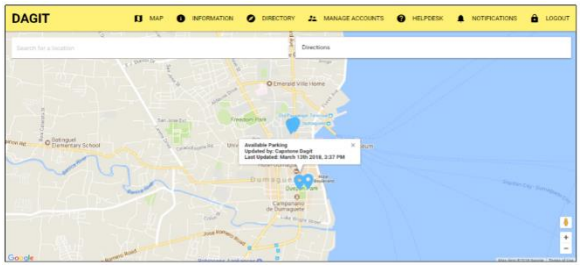


Figure 5. Map for General User Mobile App

As shown on Figure 5, the web application for the Desk TMO composes of six main menu options; map for accessing the maps, information for managing the information page of the mobile application for general users, directory for managing the directory of social and emergency services, manage accounts for managing the accounts of the Traffic Management Officers using the applications, helpdesk for viewing reports and messages sent by the general users, and notifications for managing the notifications published.

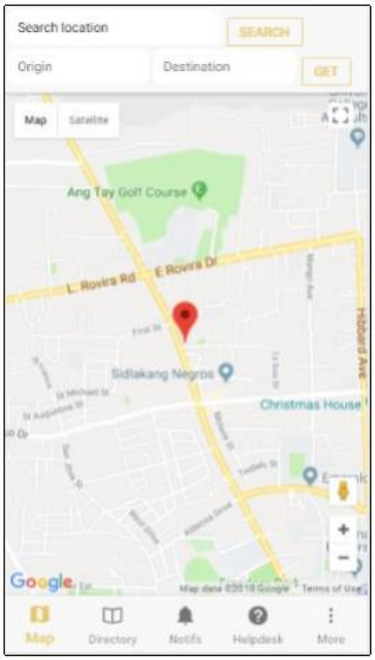


Figure 6. Map for Desk TMO Web App

As shown in Figure 6, the mobile application for the General Users composes of five main menu options; map for accessing the maps, directory for viewing the directory of social and emergency services, notifs for viewing announcement, traffic advisories, and parking advisories, helpdesk for sending reports and messages to the Traffic Management Office, and more for account settings and FAQs.

4 TESTING AND EVALUATION

The test cases were administered in the span of two days to a total of 26 respondents comprised of an employee and five traffic officers from the TMO and 20 random respondents from different departments in Silliman University.

Based on the test case results, the Desk TMO, On-Field TMO, and General User modules all passed testing. The On-Field TMO mobile application passed with all its modules functioning as expected. However, the Desk TMO web application and General User mobile application had modules that failed testing.

For the Desk TMO test case, an error message did not display in the Get Directions of Places with No Existing Details, Search On-Field TMO Account by Location with No Input, and Search On-Field TMO Account by Location with Non-Existing Location modules. The error message simply did not display but did not affect the other modules. The developers rectified the error by ensuring the error message displayed properly. Other than that, the rest of its modules passed testing.

For the General User test case, errors were encountered in the Register with Mismatched Password Confirmation, Get Directions and Distance with Invalid Input, and Report Violation with Complete Input modules. In the Register with Mismatched Password Confirmation module, one out of 20 users was able to register with mismatched passwords inputted in the required password fields when an error message should have displayed. The developers verified and ran the same test three times with the same test data and the expected result was met, similar to the 19 other respondents. In the Get Directions and Distance with Invalid Input module, eight out of 20 users noted that the Map did not turn blank when invalid locations were inputted in the Origin or Destination fields. This error occurred when inputted data from the respondents who tested the module at the Portal Office and SUSG Office at OH was not submitted to Google Maps due to slow Internet connectivity. The developers verified and ran the same test three times with the same test data with stable Internet connectivity and the expected result was met, similar to the 12 other respondents who tested the module. In the Report Violation with Complete Input module, one out of 20 users was not able to successfully send an accident report as a result of closing the application while image uploading was ongoing. Slow Internet connectivity slowed down image uploading, and the respondent thought the application stopped functioning which led to the application being closed. The developers verified and ran the same test three times with the same test data and the expected result was met, similar to the 19 other respondents. Meanwhile, the rest of the modules functioned as expected and passed testing.

Nonetheless, the Get Directions of Places with No Existing Details, Search On-Field TMO Account by Location with No Input, and Search On-Field TMO Account by Location with Non-Existing Location modules for the Desk TMO web application and the Register with Mismatched Password Confirmation, Get Directions and Distance with Invalid Input, and Report Violation with Complete Input modules for the General User mobile application do not affect the overall performance of the mentioned applications.

5 CONCLUSIONS AND RECOMMENDATIONS

5.1 Conclusion

Based on the test results, the development of web, Android, and iOS-based applications for DAGIT was successful. Thus, DAGIT is a reliable all-in-one directory, navigation guide, traffic, and parking advisory application for drivers, commuters, and establishment owners. The General User mobile application offers a navigation guide, an informative directory of emergency and social services, traffic advisory, parking advisory, announcements, and a helpdesk where they can send messages and accident, violation, and

MCH reports to the Desk TMO. Likewise, the On-Field TMO mobile application can easily update the current traffic condition and parking availability at the On-Field TMO's assigned location. Meanwhile, the Desk TMO web application efficiently allows them to manage TMO accounts, manage the directory, publish traffic advisory, parking advisory, and announcements, and reply to messages sent by General Users.

5.2 Recommendations

For further enhancements and improvements of the system, the optimization of quicker image uploading in the General User mobile application is recommended. As supported by the TMO, it is also recommended to add more violations as mandated by the TMO in the Report MCH module.

Furthermore, it is recommended to integrate descriptive statistics that will generate a summary of accident, violation, and MCH reports sent by General Users as this will be presented during city council meetings. Since the project passed all critical functions, it is recommended to fully implement DAGIT.

ACKNOWLEDGMENTS

We acknowledge the BPI Foundation and DOST for awarding and acknowledging this project for the BPI-DOST Science Awards 2018.

REFERENCES

[1] Reymar Latoza, Traffic congestion 'symptom of growing economy', http://www.iloilometropolitantimes.com/trafficcongestion-symptom-ofgrowing-economy/.
[2] Ordinario, Cai, Pinoys waste 28,000 hours in 'traffic', http://www.businessmirror.com.ph/pinoys-waste-28000-hoursin-traffic/.
[3] Francisco, Katerina, Fix traffic or PH can lose P6B daily by 2030 – JICA, http://www.rappler.com/business/economy-watch/51824-ph-cost-traffic-jica.
[4] Demecillo, Dems Rey, UNBELIEVABLE! 660,184 vehicles jam city streets, https://negroschronicle.com/unbelievable-660184-vehicles-jam-city-streets/.
[5] Ablong, William E. 2016. Need for a traffic management plan, http://dumaguetemetropost.com/need-for-a-trafficmanagement-plan-p6883-635.htm.
[6] SunStar Dumaguete, Dumaguete bridge to be closed for widening, http://www.sunstar.com.ph/dumaguete/localnews/2017/02/03/dumaguete-bridge-be-closed-widening-523705.
[7] Philippine News Agency, Dumaguete City to test one-way traffic scheme this Weekend, http://news.mb.com.ph/2017/02/18/dumaguete-city-to-test-one-way-traffic-schemethis-weekend/.
[8] SunStar Dumaguete, Dumaguete City eyes new scheme to cut traffic jam by 50%, http://www.sunstar.com.ph/dumaguete/local-news/2017/01/30/dumaguete-city-eyesnew-scheme-cut-traffic-jam-50-522903.
[9] SunStar Dumaguete, Mayor negotiates with 'recalcitrant' homeowners on road Project, http://www.sunstar.com.ph/dumaguete/local-news/2017/03/20/mayor-negotiatesrecalcitrant-homeowners-road-project-532032.
[10] The Negros Chronicle, New bridge to close for P29M expansion, https://negroschronicle.com/new-bridge-closep29m-expansion/.
[11] SunStar Dumaguete, Dumaguete re-routing ready prior to bridge closure, http://www.sunstar.com.ph/dumaguete/local-news/2017/02/08/dumaguete-rerouting-ready-prior-bridge-closure-524648.
[12] Rouse, Margaret, Mobile Application Development, http://searchmicroservices.techtarget.com/definition/mobileapplicationdevelopment.
[13] Statista, Number of apps available in leading app stores as of March 2017, https://www.statista.com/statistics/276623/number-of-apps-available-in-leading-appstores/.
[14] Statista, Average daily time spent online via mobile by internet users worldwide as of 2nd quarter 2016, by generation (in minutes), https://www.statista.com/statistics/428425/daily-time-spentonline-mobile-age/.
[15] Rouse, Margaret, Web App Development, http://searchcloudapplications.techtarget.com/definition/Web-appdevelopment.
[16] Kohan, Bernard, Guide to Web Application Development, http://www.comentum.com/guide-to-web-applicationdevelopment.html.
[17] Simbla, The Importance of Website Development, https://www.simbla.com/importance-of-website-development.
[18] Simbla, Why Is Website Development So Important?, https://www.simbla.com/why-is-website-development-important.

Portable Java-based Client/Server Application of Classical Item Analysis Theory

Arjay F. Abio
San Jose Campus
Partido State University
Philippines
abio.arjay@gmail.com

ABSTRACT

This paper describes the Test and Item Analysis System (TIAS) designed as a Portable Java-based Client/Server Application of Classical Item Analysis Theory. It was developed through the descriptive developmental approach of research concerning software design, development, and testing. The product is mainly intended for the school teachers of the different educational institutions in the Philippines and other organizations to facilitate their evaluation and item analysis activities. It can perform analysis to both dichotomous and polytomous item test types. It automates the process of determining the difficulty level, discriminating power, and recommendation for an item and measures the test's reliability by applying the Cronbach's alpha, Split-Half Method, Kuder-Richardson Formula 20, and Kuder-Richardson Formula 21. Trials revealed that the system provided 100% accurate computational results in test evaluation and earned a 90.91 score or Excellent remark in the system usability measure. It was found effective also, in facilitating the procedure, by offering the opportunity to finish the process as fast as possible. These results show that the system can perform its purpose when used by its intended users without much concern regarding its operations.

CCS CONCEPTS

• Computing Education → Technology for Teaching and Learning; Instruction Support Tools

KEYWORDS

Item analysis system, item analysis software, item analysis program, test analysis, test evaluation, difficulty level, discriminating power, options analysis, test reliability

1 INTRODUCTION

It is customary in the classroom setting that after instructions, the teacher gives an assessment or achievement test to gather evidence on the effectiveness of the teaching-learning process [14]. The scores earned by the students from the test mean many things that teachers use in their evaluation and decision-making. The scores contribute to the grades that serve as the basis for determining the level of skills or learning acquired [15]. It is, therefore, significant to ensure that the instruments for evaluating the students' performance are effective, reliable, and valid. Thus, the teachers must also check their test instruments to measure the quality and effectiveness in performing its purpose. This process is called test and item analysis.

The item analysis is the process of determining the performance of each item in a test in measuring its objectives based on the organized and statistically treated responses of students [17]. It also inspects the quality of the test instrument or assessment tool as a whole by determining its reliability and validity measures [14]. With this, the teacher will be able to review both the employed teaching strategy and the test instrument; and it helps the teacher measure the effectiveness of the test items and determine those items needing revisions or replacements to enhance the quality of the test for the next administration [10].

The importance of evaluating assessment tools through the test evaluation and item analysis procedure is given attention by educational institutions. By incorporating this topic into the curriculum of education courses, teachers are expected to apply this as part of their teaching practice. But because the process consists of mixed computations and statistical applications [5] that make it complicated, the activity is often ignored [2].

Although there are already computer programs designed for stand-alone computers to automate the computational processes in the procedure, like the TAP: Test Analysis Program [8-9] and SAS [11], the manual encoding of students' responses in a test is still a tedious and tiring part. To address this problem, other software packages like Test Checker and Item Analyzer with Statistics (TCIAS) [2-3] and SPSS [20] allow the use of a scanner to read and process the responses from its required answer sheets. However, this solution is applicable only for those tests where questions are answerable by shading the responses on an answer sheet like the multiple-choice type of test. Hence, the solution does not support other types of tests such as fill in the blanks, identification, and problem-solving that are usually part of the examination.

In this study, the researcher proposed the Test and Item Analysis System (TIAS) designed as a Portable Java-based Client/Server Application of Classical Item Analysis Theory. It is a software tool for teachers of the different educational institutions in the Philippines and other organizations to facilitate their test evaluation and item analysis activities. It specifically automates the computational processes in the procedure and runs through the client/server system to allow multiple users to access the server computer simultaneously in conducting the activity. Through this design, two or more encoders can process the data entry of students' responses concurrently, which offers to finish the procedure as fast as possible.

1.2 Objectives of the Study

The general objective of this study is to design and develop a Portable Java-based Client/Server Application of Classical Item Analysis Theory for school teachers and other organizations to facilitate their test evaluation and item analysis activities. Specifically, it aimed to:

1. Develop a system that would automate the computational processes in the test evaluation and item analysis procedure.
2. Develop a system that would run through the client/server architecture allowing multiple users in conducting test evaluation and item analysis activities simultaneously.
3. Determine the accuracy percentage of the system in terms of test evaluation.
4. Determine the usability measure of the system.
5. Evaluate the effectiveness of the system in facilitating the test evaluation and item analysis procedure.

1.3 The Significance of the Project

This system would be a great help to the teachers who wish to improve their test instruments. It could help them improve their skills in test construction and evaluate their teaching strategies. Since the activity will be easy and faster using the system, compared to the traditional manual approach, their time and effort can be more saved, thus can be devoted to other school-related activities. The school administration will also benefit because this system speeds up the procedure without requiring teachers to prepare or purchase specialized answer sheets and acquire additional hardware. Additionally, considering that the purpose of the activity will also contribute to the quality of instruction, this system would help the students to learn more, thereby would also help the school to achieve its mission. Furthermore, once the activity becomes a habit of the teachers, the student's performance and achievement are evaluated fairly, and the grades will become more justifiable. It will also make the Education Students or would-be teachers appreciate much the relevance of item analysis and test evaluation process, thus, makes this a habit to ensure that the tests given to the students are reliable and valid.

For other practitioners or other researchers and other organizations, this system will be helpful also because they can apply this in their test instruments. To the researchers, the test instruments that they used often should be reliable and valid. Hence, an evaluation process is needed, which will be easy and fast with this system. This benefit would also apply to other organizations or companies who conduct evaluation processes to their testing materials used to examine their job applicants and for other purposes.

2 RELATED WORKS

Spreadsheet software like Microsoft Excel has been one of the commonly used software for recording and organizing data. It allows the performing of calculations through its built-in functions or the user-defined formula. This software automates any computations that it can handle, like some of the features involved in item analysis. However, in using this software for item analysis, the user must first devote sufficient time to format the spreadsheet and enter the required formula or functions to achieve the desired output. Then after these initial efforts, the user can already encode the required data, which is another time-consuming task.

To at least minimize the formatting effort and eliminate the manual insertion of formula in an Excel spreadsheet, some software packages like the TVAP: Test Validation & Analysis Program version 7.0 of Biddle Consulting Group [4] utilized the programming environment of Excel through its *macro* feature. This feature allows the writing of program code using Visual Basic language to instruct Excel to perform the tasks that are usually not included in its built-in functions. This feature is used by TVAP in its workbooks to accept data from its user and to generate computational results. It is a better advantage than using Excel from scratch. However, since it is dependent on the *macro* feature of Excel, it will not work with other spreadsheet software.

Another computer program dependent on Microsoft Excel is the ezIA version 1.0 of Delta Sigma Soft [12]. The ezIA is a classical item analysis program integrated into the Excel environment through the add-in feature. With this add-in program, the user can select the data from the worksheet and execute the command to perform the computations for item analysis. It also provides a mechanism for automatic conversion of students' responses into its equivalent point value. That is "1" for the correct answer and

"0" for the wrong answer. It shows that this program supports only those tests where items are all dichotomous question types. It is another limitation of this program aside from being dependent on the Microsoft Excel environment. An item is said to be dichotomous when the response to its question will yield a correct or wrong result only when checked like the multiple-choice, true or false, identification, and fill in the blanks. On the other hand, an item is said to be polytomous when it allows partial point marking such as multiple responses, definitions, and problem-solving.

For a stand-alone application of classical item analysis that supports dichotomous items and allows partial credit scoring for multiple-choice questions, the TAP: Test Analysis Program version 14.7.4 of Gordon P. Brooks [8] bears these characteristics and capabilities. However, the entering of data in this application is quite complicated and prone to errors. Entering the answer key for each item is by encoding any valid character representing the key inside the horizontal textbox. Above this text box are the labels of item numbers. Another textbox allows multiple lines of inputs for the responses of students. In each line of this text box, the user manually encodes the unique identification code of students, then the series of valid characters representing the responses to each item. This data entry mechanism is not interactive and does not provide a feedbacking system, making it difficult to notice any possible incorrect data entered.

As a solution, the Test Checker & Item Analyzer with Statistics (TCIAS) [3] provides an interactive user interface for the manual data entry of students' responses in a test. This software does not support polytomous types of tests. However, it provides a feedbacking system for the user with automatic displaying of the remarks of each encoded response plus the update of the computed total score of the student in a multiple-choice type of test. In this way, the user can immediately review the accuracy of the encoded data. Nevertheless, maybe because the manual data encoding of students' responses in a test is still a tiring and time-consuming part of the test and item analysis procedure, the TCIAS design allows reading responses from the shaded answer sheets through its required scanning device.

The use of scanning devices to facilitate the organization of the response data is currently the employed solution by some of the software packages for item analysis. Aside from TCIAS, the SPSS also supports this approach. The SPSS White Paper [20] entitled "Using SPSS for item analysis" presented that the software can perform item analysis for multiple-choice questions. The paper also presented some products for creating test forms so that the response data will be read accurately by the scanner and immediately become ready for the data analysis [20]. Both the TCIAS and SPSS deserve recognition for their contributions to technological advancement and innovation. At least, when the multiple-choice examination administered using their required answer sheets is now can be instantly analyzed. However, the problem remains for other types of tests and even in the multiple-choice when administered in a usual classroom evaluation setting which does not use shaded answer sheets during the examination. This situation shows that manual data encoding is inevitable and that there is a need for another alternative solution to speed up the process.

In this study, the Test and Item Analysis System (TIAS) was developed. Like the TVAP [4] and ezIA [12], it can read data from an Excel worksheet and automate the computational processes. However, unlike the two software packages that both dependent on Microsoft Excel, the TIAS will run even without an Excel program. The TIAS is a stand-alone application like the TAP [8], TCIAS [3], and SPSS [20]. But considering the aim to ensure the accuracy of the entered data, the TIAS is like TCIAS [3] that provides a feedbacking system to notice the error in input data. With this feature, however, the TIAS provided an additional function. It can filter the entered responses according to its remarks for fast searching of any incorrect data entered. Then when it comes to generating results, both the TCIAS [3] and the TIAS provide verbal interpretations of their computational results. However, TCIAS [3] supports dichotomous items only, while the TIAS handles both dichotomous and polytomous items. Aside from the mentioned advantages of TIAS over other existing systems, the presence of TIAS is to serve as an alternative solution to the time-consuming data encoding process in test evaluation and item analysis activity. In this study, the TIAS as the proposed system presents the client/server approach as its unique feature that is not available to the currently existing systems for item analysis designed to process the data even from the manual testing administration. The application of this approach is allowing multiple users to help encode the students' responses in a test. This feature aims to finish the process as fast as possible depending on the number of encoders concurrently processing the data. Fast data entry to the system makes the procedure more efficient as all computations are performed automatedly by the system in just less than a minute.

3 OVERVIEW OF THE PROPOSED SYSTEM

This study focused on developing software, specifically an automated system intended to facilitate the test evaluation and item analysis procedure of the classical item analysis theory. The application of this theory in the system is, based on the idea that although there already existed the item response theory as the modern approach, the classical test theory was still the most frequently used method for item analysis [11, 19].

In item analysis, the classical test theory is the approach for determining item characteristics such as the difficulty level and discriminating power, including the effectiveness of the distractors in multiple-choice items and the reliability of the whole test based on examinees' responses [15]. In this approach, the generated information about the test items and the entire instrument is dependent on the ability of the particular group of students who took the test [15]. Thus, through responsible interpretation of its results, the approach does not only scrutinize the test and its items but may alert the teacher on the effectiveness

of the employed teaching strategy on a particular group of students. It shows that the classical test theory serves as a guide for test construction and also helps improve teaching pedagogy. Through this study, some of the commonly used functions of the classical test theory are in the Test and Item Analysis System.

The system performs analysis of both dichotomous and polytomous items. It determines the level of difficulty and discriminating power of these items, and it gives a recommendation of which item shall be retained, revised, or rejected. The system also measures the effectiveness of the options in a multiple-choice question, and it gives a remark on whether the option is good or poor. These verbal interpretations serve as the references by the teacher in improving their test instrument, spotting the topics that need remedial instruction, and enhancing the classroom instruction. See the sample item analysis result in Figure 1.

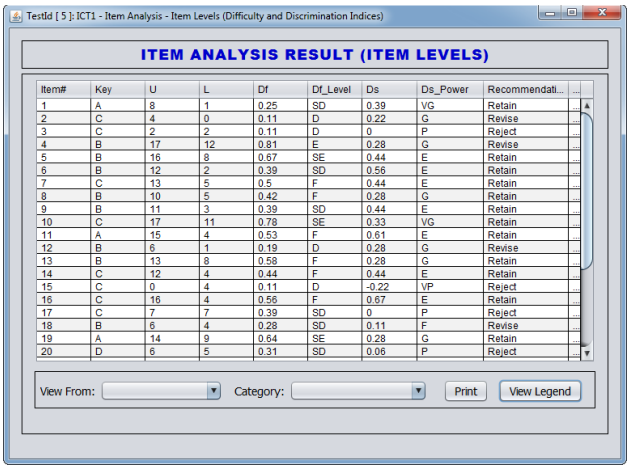


Figure 1: The sample item analysis result.

The system evaluates the test as a whole. It measures the test's reliability by applying Cronbach's alpha, Split-Half method, Kuder-Richardson Formula 20, and Kuder-Richardson Formula 21. The Cronbach's alpha applies to both tests containing dichotomous and polytomous items. The other three approaches apply to tests containing dichotomous items only. Based on these reliability measures, the system also computes the standard error of measurements. See the sample test reliability result using Cronbach's alpha in Figure 2.

Other supporting measurements for test analysis that the system generates are measures of central tendencies (mean, median, and mode) and the measures of dispersions (highest, lowest, variance, and standard deviation) based on the scores of students. The system accepts tests for evaluation that have 10 to 5000 items taken by 10 to 10,000 examinees. For multiple-choice and multiple-response questions, the system supports items with 3 to 5 options. The system also handles some scoring algorithms for multiple-response questions and allows inputting of individual point values for each option.

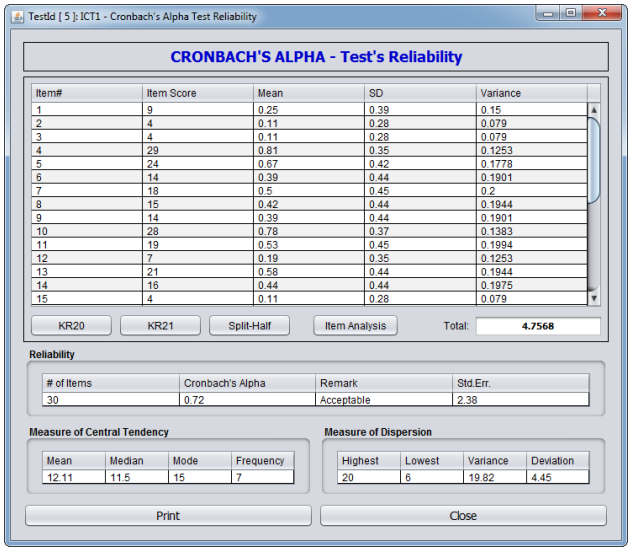


Figure 2: The sample test reliability result using Cronbach's alpha.

The system is designed to receive data directly from the keyboard. However, it also allows uploading an Excel file containing students' responses and the test's key to correction. This feature enables the system to process those data retrieved from other systems when organized into its required Excel input file. The system also contains *restore* and *backup* features of the database for security and maintenance purposes. And using the database system and networking system, the system runs on a stand-alone computer for a single user or networked computers for multiple users forming a client/server architecture.

The system is a Java-based application. It successfully ran on a Windows-based environment. However, although we can expect that it will run on different operating systems as Java has its virtual machine, it was not yet tried to run on other operating systems. Nevertheless, it is open for a future configuration to support other platforms if needed. It also has room for improvement for adding more features or enhancing its current design.

3.1 Architectural Design

There are two ways on how to use the system. Since each copy of the software package has a copy of the database file, a single user can use it on a stand-alone computer. Each copy also of the application can connect to the database of other applications in a network. It means that multiple users can use the system in a network via connecting to its desired database. Facilitating the data entry of students' responses to the system is through simultaneous access by multiple encoders using client computers connected via a network. It is the application of the client/server system. This design also allows the users to conduct test evaluation and item analysis procedures for one or more tests concurrently. The main components of the system in this design are the database, the Java DB 10.10 [18], the host computer or

server, the network, the client computer, and the TIAS application. Figure 3 shows the graphical representation of the system's architectural design using the client/server model.

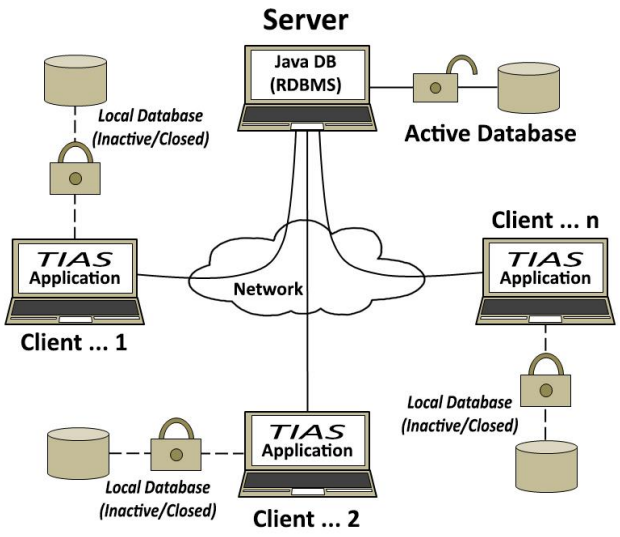


Figure 3: The system's architectural design.

The database contains the data received and processed by the system like the user accounts, test details, the key to correction, and the evaluation results. The Java DB is the Relational Database Management System [18] designed to manage the data inside the database. It is responsible for responding to the requests like retrieving, modifying, and deleting the data. Additionally, it also implements restrictions to secure the data. Meanwhile, the host computer holds the database and the Java DB. Through the functions of the Java DB, this machine acts as the database server that sends and receives data from client computers connected to the network. The network is the physical or logical connection between the server computer and client computers used for the data transmissions. The client computer is the machine that is running the TIAS application connected to the database in the server computer. Connecting to the server computer is through specifying its IP address during the login process using the login form of the TIAS application. The TIAS application is the core program of the system that performs the entire evaluation process.

3.2 Process Flow

The process flow shown in Figure 4 shows the sequence of the processes required to complete the test evaluation using the system. The user must log in first to the system to be able to perform or execute each operation. The entering of the test's details is the first process to complete before entering the test's key to correction. Providing the data, such as the test's total number of items, allows the system to prepare the form for the data entry of the test's key to correction. Each predecessor process is the prerequisite of its succeeding process. Thus, before allowing the user to enter the students' responses, the user must first complete the entering of the test's key to correction. It is for the system to determine whether response data is correct, partially correct, or wrong. Through this automatic determination of the response's remark, the system can compute the earned points in each item and the total score. These are the requirements of the system to perform the computations. A single click of the button that calls the function for this process finishes all the calculations involved in the analysis that finally allows the user to generate or view the results.

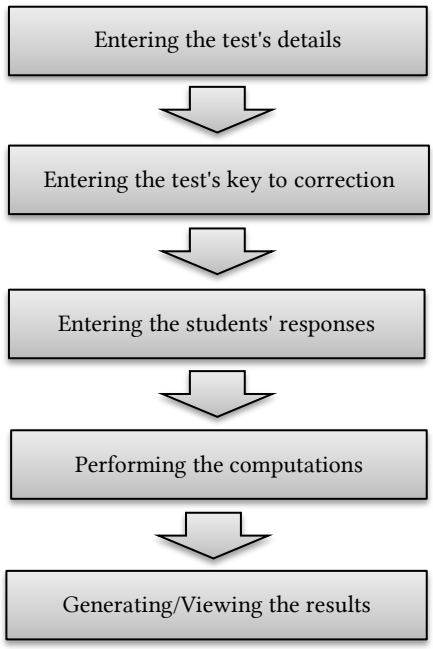


Figure 4: The Process Flow in evaluating a test using the system

4 METHODOLOGY

In this study, the researcher applied the descriptive developmental approach of research concerning software design, development, and testing. The concept of the developmental method was used during the software development process where the product is the new proposed system, and the application of the descriptive method is the reporting of quantitative results and analysis of its testing phase. This section discusses the development process of the system and its testing phase.

4.1 System's Development

The researcher studied the classical item analysis procedure. In this process, the researcher observed how the teachers in their university prepare the test papers for evaluation. The researcher also consulted some of the teachers who have enough knowledge about the procedure. For a deeper understanding of the concept, including the computations involved, other information was

collected from the different textbooks, web resources, journals, and other references. These helped the researcher conceptualized the design of the system.

The planned design of the system was to make it a portable client/server application using Java language. The researcher used the Java Platform Standard Edition (Java SE) 8 from the package of the Java Development Kit (JDK) 8 [21] and used the Netbeans IDE 8.1 [23]. The JDK 8 already contains the Java DB [18] Relational Database Management System (RDBMS) software for managing databases. Thus, the Java DB, used for storing and processing the data of test evaluation and item analysis procedure in using the system. It realizes the application of the client/server approach to have centralized database storage accessible simultaneously by multiple encoders from client machines. It contributes to the portability of the system since it no longer requires an installation of other RDBMS.

The system also allows uploading an Excel input file that contains the data of a test, such as the key to correction and students' responses. This additional feature enables the system to process the data retrieved from other systems compiled into an Excel file. However, since Java has no predefined functions for this purpose, the researcher used Java Excel API [22] in realizing this feature. It is the same with the printing reports feature wherein the researcher used the DynamicReports version 4.1.0 [16], which allows the researcher to create reports without even using the visual report designer. Because of this reporting tool, the system can already generate printable and exportable reports of the test and item analysis results.

During the compilation of the system's software package, all files of the required tools or APIs imported to NetBeans IDE were automatically placed in the generated folder by NetBeans after running the building process. NetBeans also created a copy of the database file in its default target location, but the researcher manually transferred this to the generated folder.

To simplify the process of running the database server and at the same time pointing the database file inside the folder, a batch file with the required commands was created and placed in the same folder. This setup realizes the portability of the system. The entire folder can already be copied and transferred across Windows-based computers with installed Java, and it is already ready to run.

4.2 Testing

While the researcher was in the development process, the researcher performed tests on every part of the written code to check the appropriateness of its used logic, algorithms, and formula. Then after the development of the whole system, the researcher thoroughly checked the functionality of its different features and then evaluated its accuracy and usability aspects. The researcher also observed the effectiveness of the system in facilitating the procedure.

The accuracy of the system in evaluating a test was computed in terms of percent accuracy. It was determined by comparing its reliability results to the results generated by Microsoft Excel and by TCIAS [3] software packages. The researcher used Microsoft Excel as several scholars [5, 17, 19] already used it in analyzing the data in their studies. And the TCIAS [3] was also used because of being proven accurate software [2]. Thus, the results of these two software packages served as the ground truth in this process.

The purpose of this process was to make sure that the system produces accurate computational results. The results of TIAS must be equal to the results of the two software up to the hundredth place. In this process, the comparison of the computational results was up to the hundredth place since the system generates this format as required for system generation of verbal interpretations.

In terms of system usability, the TIAS was evaluated using the System Usability Scale (SUS) of John Brooke [6]. It was used because of its simplicity, yet reliable and valid survey questionnaire for evaluating the usability of a system [7]. It contains 10 item questions with alternating positively worded questions (items 1, 3, 5, 7, and 9) and negatively worded questions (items 2, 4, 6, 8, and 10) [6-7]. In using this instrument, the respondents who tried to use the system were asked to rate its usability by selecting their desired response to each question from the scale of 1 (Strongly Disagree) to 5 (Strongly Agree). Then, the responses of all respondents were organized and computed following the required process of computations for this instrument.

And lastly, the researcher observed the effectiveness of the system in facilitating the test evaluation and item analysis procedure by recording and comparing each time consumed in processing the same test data with a different number of encoders in each trial using the client/server approach. In this process, the researcher used a manual timer using a stopwatch application in mobile phones. The researcher expected that as the number of encoders increases, the processing time to finish the procedure also decreases. It will prove and evidenced the advantage of the system over other systems or programs that the proposed system will offer to finish the activity as fast as possible, even without using a scanner and answer sheets.

5 RESULTS AND DISCUSSION

5.1 Accuracy

During the implementation of the accuracy test, the researcher used four sets of test data. These test data were the classroom tests given by the researcher to his students at the Partido State University - San Jose Campus. The first test was the 20 items multiple-choice quiz with four options in the subject ICT1 - Information and Communication Technology taken by 31 BSTM1 (Bachelor of Science in Tourism Management) students during the first-semester academic year 2015-2016. Table 1 shows the test evaluation results generated by the TCIAS and Excel and by the TIAS applying the Cronbach's alpha, Split-Half Method, Kuder-Richardson Formula 20, and the Kuder-Richardson Formula 21 test reliability. In this process, the results of TCIAS and Excel served as the ground truth, and as expected, generated the same values

in computing each reliability method. The table shows that the TIAS results in each test reliability method are just the same as the results generated by the two software making the TIAS software 100% accurate in this test in computing the four reliability methods.

Table 1: The Accuracy of TIAS in generating test reliability results of the first test

Test Reliability	TCIAS & Excel	TIAS	% Accuracy
Cronbach's alpha	0.73	0.73	100%
Split-Half method	0.77	0.77	100%
KR 20	0.74	0.74	100%
KR 21	0.67	0.67	100%

The same method was performed in the second test that was 30 items multiple-choice test with four options during the final examination in the subject ICT1 taken by 67 students from the mixed classes of BSTM1 and BSHRM1A (Bachelor of Science in Hotel and Restaurant Management) during the first-semester academic year 2016-2017. The results showed in Table 2 show that again the TIAS is 100% accurate software in computing the four test reliability methods.

Table 2: The Accuracy of TIAS in generating test reliability results of the second test

Test Reliability	TCIAS & Excel	TIAS	% Accuracy
Cronbach's alpha	0.72	0.72	100%
Split-Half method	0.87	0.87	100%
KR 20	0.72	0.72	100%
KR 21	0.66	0.66	100%

In the first and second tests, the four reliability methods were included in evaluation as all of these are applicable for tests containing dichotomous items like a multiple-choice test. For tests containing polytomous items which allow partial credit scoring like multiple response types, Cronbach's alpha is the only applicable method for measuring the test reliability using the TIAS. Thus, during the third and fourth tests where both multiple response types, Cronbach's alpha was the only test reliability generated by the TIAS. And these were compared to the results generated by Excel to determine the percent accuracy. In this case, the TCIAS was no longer used because it has no feature to evaluate multiple response tests.

The third test was the 20 items multiple response quiz with four options in the subject ICT1 taken by 46 students from the mixed classes of BSTM1 and BSHRM1A during the first-semester academic year 2016-2017. And the fourth test was the 12 items multiple response midterm examination with four options in the subject MIS - Management Information System taken by 47 examinees from the mixed classes of BSHRM3A and BSHRM3B during the first-semester academic year 2016-2017. Table 3 shows the Cronbach's alpha reliability results generated by TIAS and the percent accuracy upon comparison to the results generated by Excel. As shown in the table, the reliability measures of TIAS to both third and fourth tests are 100% accurate. These results concluded that the overall percent accuracy of TIAS in terms of test evaluation is 100%.

Table 3: The Accuracy of TIAS in generating test reliability results of third and fourth tests

Subject Code	Test Reliability	Excel	TIAS	% Accuracy
ICT1	Cronbach's alpha	0.72	0.72	100%
MIS	Cronbach's alpha	0.87	0.87	100%

5.2. Usability

During this evaluation process, 11 faculty members of the Partido State University who had tried the system were asked to answer the SUS questionnaire [6] right after they experienced using the system. Their responses to each item of SUS, including the computed SUS scores and the average SUS score, are shown in Table 4. Each respondent's SUS score shown in the last column was earned by subtracting 1 to the responded scale position to all odd number items and subtracting the responded scale position from 5 to all even number items and the sum of the resulting values of all 10 item questions was multiplied by 2.5 [6-7].

Table 4: SUS results of TIAS

Respondents	Question Number										SUS Score
	1	2	3	4	5	6	7	8	9	10	
R1	5	3	4	2	4	2	5	1	4	1	82.5
R2	5	1	5	1	5	1	5	1	5	1	100
R3	5	2	4	2	5	1	4	1	5	4	82.5
R4	4	1	4	1	4	1	5	1	5	1	92.5
R5	5	2	3	3	5	2	4	2	4	3	72.5
R6	5	2	5	1	4	2	5	1	4	2	87.5
R7	5	1	5	1	5	1	5	1	5	1	100
R8	5	1	5	1	5	1	5	1	5	1	100
R9	5	1	5	1	5	1	5	1	5	1	100
R10	5	2	5	1	5	1	5	1	5	1	97.5
R11	4	3	5	2	5	2	5	1	4	1	85
										Average (Overall SUS Score):	90.91

In SUS, the range of scores is from 0 to 100 [6]. To interpret the result, the higher the overall SUS score, the higher the perceived usability of the system [13]. In this study, the average SUS score

representing the overall usability measure of the system is 90.91. According to Sauro, as cited by Brooke [7], the average SUS score is 68. It shows that the system has a high usability measure being far from the average. To better understand this result, applying the adjective rating scale for SUS score [1], the system reached the "Excellent" level of usability. It means that the system is capable of performing its purpose when used by its intended users without much concern regarding its operation.

5.3. Processing Time

The heart of this study is the presentation of this system using a client/server approach aiming to facilitate the entire test evaluation process by allowing multiple encoders to encode the students' responses on a test. It makes the system immediately execute the automation of the computational process and instantly generate the results. In this system, we can consider the procedure as finished when it received all the required data. Thus, the processing time in this section refers only to the length of time spent in entering the students' responses.

Using the same test data in the first accuracy test and with the assistance of three volunteer encoders, the effectiveness of the system in facilitating the procedure was tested. It was done inside the computer room using three desktop computers connected via a local area network (LAN). All of these computers were running Microsoft Windows 10 with Intel CoreI5 microprocessor and 4 GB RAM. One of these computers served as the server as it hosted the active database, and at the same time, served as a client as it runs the TIAS application connected to the database. It was also used during the encoding process. The other two computers served as clients only.

In testing the processing time based on the number of encoders, the system was first used by one encoder only; the second was with the two encoders already accessing the system concurrently; the last was with the three encoders already simultaneously encoding the students' responses. Based on the results, in the first test, with a single encoder, it took 10 minutes and 47 seconds to finish the procedure. While in the second test, the two encoders took 5 minutes and 32 seconds only, half of the processing time incurred by the single encoder. And the last test, the three encoders, took only 3 minutes and 19 seconds. These results are summarized in Table 5.

Table 5: Summary of the Processing Time

Number of encoders	Processing Time
1	00:10:47
2	00:05:32
3	00:03:19

As expected, the results show that as the number of encoders increases, the processing time of the procedure decreases. In this test, the revealed results after evaluating a test with 20 items taken by 31 students that took only 3 minutes and 19 seconds to process shows that the system made the procedure very fast. Because if this was done manually (using pen and paper), it would take a day or more, especially when the teacher who will do the evaluation is not very familiar with the computations used in the procedure. Unlike using the system wherein the calculations are no longer concerns of the teacher, the teachers will no longer worry about the time-consuming computations. They will only need to enter the data in the fastest way, which can be done now by multiple encoders encoding the students' responses to the system simultaneously. These encoders could be their students, co-teachers, or other persons who could help them encode students' responses.

6 CONCLUSION AND RECOMMENDATION

In this study, the Test and Item Analysis System (TIAS) was developed. It was designed as a Portable Java-based Client/Server Application of Classical Item Analysis Theory intended for the school teachers of the different educational institutions in the Philippines. It facilitates the test evaluation and item analysis activities. Other organizations may also adopt this system in their workplace.

This system automates the computational processes in test evaluation and item analysis procedures. It relieves the teachers from performing on their own the series of computations of the process that is very tiring, prone to errors, and time-consuming tasks. The outputs generated by this system that has verbal interpretations already guide the teachers in their decision-making about improving test instruments, improving teaching strategies, dealing with the students, and improving other external systems in their environment related to the teaching profession and research.

Like other item analysis software, this system also works using a stand-alone machine. But this system has its new approach, which is a unique feature to facilitate the entire test evaluation activity. This system applies client/server architecture using a database system and computer networking systems. This design allows the teachers or the school administration to set up the system to multiple computers and assign one server to host the database that is accessible by other computers. It allows the teachers to help each other by encoding the students' responses on a test, and it offers to finish the activity as fast as possible.

This system was proven accurate and usable. The tests made to this system proved its accuracy and evidenced its high usability measure described as excellent in terms of its usability aspect. It also showed its effectiveness in facilitating the entire test evaluation process. However, the purpose of this system will be effectively realized when it is already accessible by its intended users with awareness regarding its application.

In this regard, the researcher recommends distributing this system to the various educational institutions in the Philippines, especially in schools with no dedicated software for item analysis. And a seminar and workshop, also be conducted to teach them about its application and its operations.

REFERENCES

[1] Aaron Bangor, Philip Kortum, and James A. Miller. 2009. Determining What Individual SUS Scores Mean: Adding an Adjective Rating Scale. *JUS* 4, 3 (May 2009), 114–123.

[2] Cesar B. Bermundo and Alex B. Bermundo. 2007. Test Checker and Item Analyzer with Statistics. In *10th National Convention on Statistics (NCS)*. EDSA Shangri-La Hotel. Retrieved from http://nap.psa.gov.ph/ncs/10thNCS/papers/contributed%20papers/cps-13/cps13-01.pdf

[3] Cesar B. Bermundo, Alex B. Bermundo, and Rex C. Ballester. 2004. *TCIAS: Test Checker & Item Analyzer with Statistics*. Philippines.

[4] Biddle Consulting Group, Inc. 2010. *TVAP: Test Validation & Analysis Program*. 193 Blue Ravine, Suite 270 Folsom, CA 95630 / 916.294.4250 ext. 113. Retrieved from http://www.biddle.com/tvap/index.htm

[5] C. Boopathiraj and K. Chellamani. 2013. Analysis of test items on difficulty level and discrimination index in the test for research in education. *IRJC* 2, 2 (February 2013), 189–192.

[6] John Brooke. 1996. SUS: 'a quick and dirty' usability scale. In *Usability Evaluation in Industry*. Taylor and Francis, London, 189–194. (Also see https://hell.meiert.org/core/pdf/sus.pdf)

[7] John Brooke. 2013. SUS: A Retrospective. *JUS* 8, 2 (February 2013), 29–40.

[8] Gordon P. Brooks. 2014. *TAP: Test Analysis Program*. Retrieved from http://www.ohiouniversityfaculty.com/brooksg/#TAP

[9] Gordon P. Brooks and George A. Johanson. 2003. TAP: Test Analysis Program. *Applied Psychological Measurement* 27, 4 (July 2003), 303–304.

[10] Steven J. Burton, Richard R. Sudweeks, Paul F. Merrill, and Bud Wood. 1991. How to Prepare Better Multiple-Choice Test Items: Guidelines for University Faculty. Retrieved from https://testing.byu.edu/handbooks/betteritems.pdf

[11] Chong Ho Yu and Josephine Wai-chi Wong. Using SAS for Classical Item Analysis and Option Analysis. Retrieved from https://www.researchgate.net/publication/253705144_Using_SAS_for_Classical_Item_Analysis_and_Option_Analysis

[12] Delta Sigma Soft, Inc. 2008. *ezIA*. Retrieved from www.DeltaSigmaSoft.com

[13] Iman Dianat, Zahra Ghanbari, and Mohammad AsghariJafarabadi. 2014. Psychometric Properties of the Persian Language Version of the System Usability Scale. *Health Promotion Perspectives* 4, 1 (2014), 82–89. DOI:https://doi.org/10.5681/hpp.2014.011

[14] Heba Bakr Khoshaim and Saima Rashid. 2016. Assessment of the Assessment Tool: Analysis of Items in a Non-MCQ Mathematics Exam. *International Journal of Instruction* 9, 1 (January 2016), 119–132. DOI:https://doi.org/10.12973/iji.2016.9110a

[15] Carlo Magno and Jerome A. Ouano. 2010. *Designing Written Assessment for Student Learning*. Phoenix Publishing House, Manila, QC. Retrieved from https://www.researchgate.net/publication/277405724

[16] Ricardo Mariaca. 2010. *DynamicReports*. Retrieved from http://www.dynamicreports.org

[17] N K Mitra, H S Nagaraja, G Ponnudurai, and J P Judson. 2009. The Levels Of Difficulty And Discrimination Indices In Type A Multiple Choice Questions Of Pre-clinical Semester 1 Multidisciplinary Summative Tests. *International E-Journal of Science, Medicine and Education* 3, 1 (2009), 2–7.

[18] Oracle. 2014. Getting Started with Java DB Version 10.10. Retrieved from https://docs.oracle.com/javadb/10.10.1.2/getstart/getstartderby.pdf

[19] Pradip Chauhan, Girish Rameshbhai Chauhan, Bhoomika Rameshbhai Chauhan, Jayesh Vrajlal Vaza, and Suresh Punjabhai Rathod. 2015. Relationship between difficulty index and distracter effectiveness in single best-answer stem type multiple choice questions. *Int J Anat Res* 3, 4 (2015), 1607–1610. DOI: http://dx.doi.org/10.16965/ijar.2015.299

[20] SPSS Inc. 1998. SPSS white paper: Using SPSS for item analysis. Retrieved from http://www.spsstools.net/Syntax/ItemAnalysis/UsingSPSSforItemAnalysis.pdf

[21] *Java Development Kit*. Retrieved from http://filehippo.com/download_java_development_kit/64/

[22] *Java Excel API*. Retrieved from http://jexcelapi.sourceforge.net/

[23] *NetBeans IDE*. Retrieved from https://netbeans.org/community/releases/81/

Philippine Information Technology Journal, Vol. 11, No. 2, July - December 2018

Wheel-spinning in a Novice Programming Context within a Meaningful Gamification System

Rejeenald M. Flores†
Ateneo de Manila University
Quezon City, Philippines
flores.rejeenald@gmail.com

Ma. Mercedes T. Rodrigo
Ateneo de Manila University
Quezon City, Philippines
mrodrigo@ateneo.edu

Jenilyn L. Agapito
Ateneo de Naga University
Naga City, Philippines
jen.agapito@gmail.com

ABSTRACT

Wheel-spinning (WS) refers to failure to master a skill in a timely manner or after a considerable number of practice opportunities. Past studies have shown that WS exists within some Intelligent Tutoring Systems (ITS) and game-based learning environment systems for Mathematics and Physics. In this study, we determine whether wheel-spinning exists in novice programmers using an existing database of a Meaningful Gamification System (MGS), and on which of the identified programming skills within MGS do novice programmers show more incidence of wheel-spinning. We found that wheel-spinning exists in a novice programming context. We also found out that novice programmers wheel-spin in program generation skill, and skills involving tracing and variables.

CCS CONCEPTS

• Applied computing~Education

KEYWORDS

Wheel-spinning, Mastery Learning, Student Modeling, Meaningful Gamification System, Novice Programmers

ACM Reference format:

Rejeenald M. Flores, Ma. Mercedes T. Rodrigo, and Jenilyn L. Agapito. 2018. Wheel-spinning in a Novice Programming Context within. In *Proceedings of Information and Computing Education Conference (ICE 2018), Cebu City, Philippines, 9 pages.*

1 Introduction

Wheel-spinning refers to failure to master a skill in a timely manner, despite effort [17][10][9][24][23][8]. Students experience wheel-spinning due to a knowledge deficit [22][9], and poor acquisition of prerequisite skills [8]. Previous studies have examined data from several educational tutoring systems (Physics Playground [22], ScatterPlot Tutor [9], Cognitive Algebra Tutor and ASSISTments [10][8][24][23]) to study the wheel-spinning phenomenon. While all of these tutoring systems are built for its purpose to help students master the skills by applying the concept of mastery learning, some students inevitably get stuck at some point in time, and wheel-spin.

In this paper, we study the incidence of wheel spinning among 141 novice programmers from Ateneo de Manila University (ADMU) and Central Bicol State University and Agriculture (CBSUA), using an existing database of Meaningful Gamification System (MGS) [12], and attempt to identify the programming skills that novice programmers are showing more incidence of wheel-spinning. Unlike the systems used in the previous studies which are built to teach Mathematics and Physics, MGS is designed to help students in their introductory programming course.

2 Review of Related Literature

This research builds on two foundational concepts – novice programmer education and wheel-spinning. In this section, we discussed each one. Also, we discussed the system used in this study, the Meaningful Gamification System (MGS).

Learning programming involves two main tasks, program comprehension and program generation [19]. Mastering these tasks is inherently difficult to accomplish. Studies have shown that novices struggle in concepts [18], programming language syntax [6][18], program planning [15][6][5], tracing [6][18][21][1], and debugging [15][6][18][7][2]. Novices meet a lot of challenges and make many mistakes to gain expertise in programming [13] and spend too much time correcting errors [6][7]. Gaining expertise means mastering all the skills involved, especially in programming. Since studies have showed that novice programmers struggle for each of the programming fundamental skill, then they are not exempt to experience wheel-spinning.

An active area of study within computer science education research is the quantification of students' ability to cope with errors. To this end, several metrics have been developed. These are the Error Quotient (EQ) [14], the Watwin Algorithm [3], and the Repeated Error Density (RED) [19]. In past studies, students with higher scores in EQ [6][7][14][16][4], Watwin [4][3], or RED [19][4] are the ones who struggles to correct the errors they encountered while doing the programming tasks. These scores may be indicative of wheel-spinning. However, these metrics only apply to program generation skills of the students, and not on the conceptual and the program comprehension skills. Therefore, we cannot use these metrics in this study of wheel-spinning.

*Article Title Footnote needs to be captured as Title Note
†Author Footnote to be captured as Author Note
Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
ICE2018, October 2018, Cebu City, Philippines
© 2018 Copyright held by the owner/author(s).

The problem of wheel-spinning had been studied within some educational software, to investigate or detect wheel-spinning, such as ScatterPlot Tutor [9], Physics Playground [22], Cognitive Algebra Tutor (CAT) and ASSISTments [10][8][24][23]. Beck & Rodrigo (2014) [9] found out that wheel-spinning exists in a non-western population using ScatterPlot Tutor. Palaoag and her team (2016) [22] found out that wheel-spinning also exists in Physics Playground where 30 – 40% of their student participants require additional intervention to help them towards mastery. Using ASSISTments, Wan & Beck (2015) [8] determined that the lack of the prerequisite knowledge contributes to wheel-spinning, while Gong et al (2016) [24] found out that there was a big allocation of students' time (i.e. 28%) being wasted. In addition, Gong and Beck used CAT together with ASSISTments to detect wheel-spinning. They found out in their earlier study [10] that there were less than 40% of students who did not master the skill or probably wheel-spinning. They also found out that students who struggle to master the skill, will probably never master the skill at all. In their later study [23], they showed that those students who wheel-spin were those who committed less number of correct responses, and provided more incorrect responses and attempted more problems on a skill without mastery.

These past studies have successfully studied wheel-spinning in those educational software. This means that wheel-spinning exists in other systems. However, none of them have studied the existence of wheel-spinning in the context of programming.

2.1 Meaningful Gamification System

In this study, we used a Meaningful Gamification System designed to facilitate activities such as quizzes and programming exercises which are common to introductory courses in programming. This system was built and used by Agapito & Rodrigo to study on Meaningful Gamification, which uses game elements to create a gameful and playful learning environment and builds internal motivation for a long-term change and engage people in a specific context [11][20][12].

The system allowed the students to program with C++ or Python during the experiment of past studies [11][12]. The researchers added the questions into the system by encoding them through the system's user interface, and they tagged each question with one of the default skills in the system. These skills were *"Concepts", "Tracing", "Programming",* and *"Debugging"*. Some questions for program generation, though, were tagged with both *"Programming"* and *"Debugging"* skills.

According to the experts' evaluation about their system [11], the embedded game design elements in the system supported mastery and autonomy. Mastery in the system was supported by allowing students to fail and learn from mistakes while autonomy was supported by giving students the feeling of control such as choosing from the extra items to answer which they feel they were good at. Furthermore, they showed in their usability test [12] that the control, freedom to fail coupled with a meaningful feedback were helpful to novices because they got better scores.

The implementation of the system's game design elements especially mastery was helpful for a better achievement of students. However, this could mean that wheel-spinning may also existed during the learning process as most students utilized the freedom to fail feature of the system. The system was built recently and the researchers had not yet used the same data set to discover the existence of wheel-spinning. Using the same data set, this study is interested in doing so.

3 Methodology

The goal of the study is to determine the existence of wheel-spinning in a novice programming context. To do that, the methods are discussed in this section. The first part discusses about the data extracted from the data set that are found useful from the MGS database. The second part is the method used to identify whether wheel-spinning exists in a novice programming context. The last part explains how the skills with more incidence of wheel-spinning are identified.

3.1 Data

Prior to addressing the research questions, the useful data had to be identified and extracted from the existing database of the system. Some tables in the database were selected which were found useful and some data in these tables were removed such as the data of those dummy accounts created by the admin, and the data with missing values. However, it was made sure that those removed data are carefully evaluated and confirmed by the creator/admin of the system.

In preparing the data, important software tools were used, namely the *Visual Paradigm* for the visual review of the database structure in order for us to know how we could retrieve the needed data through queries, *PostgreSQL* for us to access the existing database as it is a Postgres database, *Jupyter Notebook* for us to build Python program functions for data cleaning and manipulation, and *Tableau* for data visualization.

3.1.1 Demographics. There are 141 total number of first year students from both Universities who used the system: 71 students enrolled in a CS1 subject were from Ateneo de Manila University (ADMU) and 70 students taught only with computing fundamentals using C++ were from Central Bicol State University and Agriculture (CBSUA). ADMU students used Python while CBSUA used C++ as their programming language.

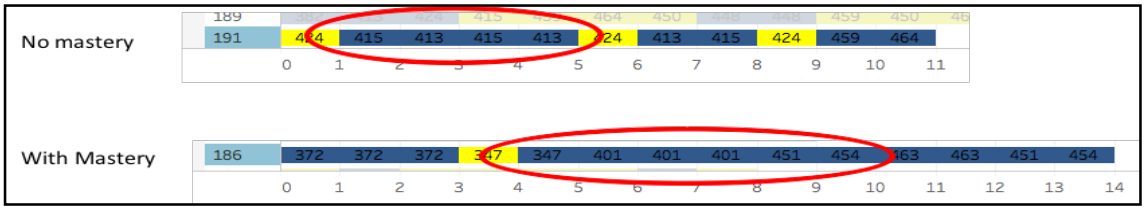


Figure 1 Instances with and without mastery

In the existing database, records of students were grouped into two, the experimental and the control groups. Only the experimental groups were considered since all the control groups were limited to single attempt per activity while the students in experimental groups were not. Thus, there were only 60 students left for observation: 33 from ADMU and 27 from CBSUA.

3.1.2 Student Task. Students were tasked to answer questions through the given activities within the system. Each submission earned them a final score by the number of correct answers over the total number of items in the activity. They can reattempt to answer all the items in each activity after submission. The experimental group of ADMU had unlimited attempts while the experimental group of CBSUA had at most three attempts. During the experiment, they were given 2 short quizzes, 2 programming exercises, and 1 long quiz.

3.2 Identifying Existence of Wheel-spinning

The Percentage of Students with Mastery (PSM) helps identify whether students are progressing towards mastery or not (possibly wheel-spinning) on a particular skill. To compute the PSM per skill, three prerequisite stages are performed beforehand. First, the identification of skills. Second, the identification of instances of mastery. Last, the validation of identified skills. These prerequisite stages are iterative until the correct skills involved are identified.

An instance identified whether it is an instance of mastery or not is in student-skill pairings. It is basically just the set of all responses of a student per skill. Each response serves an attempt of student towards mastering the skill.

3.2.1 Identifying Skill per Question. Since instances are in student-skill pairings, it is necessary to identify what skill a student is trying to master or what skill a particular student response corresponds to. Since the default set of skills of MGS are coarse grained, it may be necessary to identify a more specific set of skills. The tagging of skill per question is done manually by properly reviewing each question and identify what skill is really involved.

3.2.2 Identifying Instances of Mastery. Identifying if there are instances with mastery is significant. If ever there are no instances with mastery per identified skill, then that could possibly mean that the identified skill is not the right skill as no one can master it, and the students just often commit mistakes.

There are 4,840 total responses to 204 given questions from students of both Universities. Table 1 shows the breakdown of this numbers where there are 112 questions given to CBSUA experimental group and have provided 2,105 responses to these questions while the ADMU experimental group are given 92 questions and have provided 2,735 responses to these questions as well.

Table 1 Experimental Groups Question and Response Count

	CBSUA experimental group	ADMU experimental group	Total
Questions	112	92	204
Responses	2105	2735	4840

At this stage, questions are already tagged with skills. All those responses are used to generate the instances per skill. Knowing which skill a particular response maps to are based on its corresponding question's tagged skill. After the instances are generated per skill, these instances are identified whether it is an instance of mastery or not. The mastery criterion is discussed in the following part. Moreover, only the instances of mastery are considered in the computation of PSM.

3.2.2.1 Describing Mastery. Following the criteria from prior studies [10][8][24][23], we say that a student has mastered a skill when he/she is able to give three consecutive correct responses in a row. Any responses after the mastery criterion has been fulfilled are ignored. Responses corresponding to questions correctly answered in the previous attempts are also ignored.

The reason for the additional rule is that whenever the students reattempt an activity, the same set of question has to be answered again. Students reattempt because they want to correct their answers on questions they provided with an incorrect response, and not on the questions with the correct response. However, they still need to answer all the question to earn higher score points.

Figure 1 shows as an example in identifying an instance of mastery and an instance no mastery. Each tile represents a response. Yellow and Blue tiles represent an incorrect and a correct response, respectively. The numbers in Yellow and Blue tiles are the question ids, while the numbers in Light Blue tiles are the student ids. Responses are in ordered from the first response to the last response of a student. Each response corresponds to the series of numbers below which are the number of Response Opportunities (RO) or attempts.

Looking at the part encircled with red, the first instance is showing no mastery as both responses to questions 415 and 413 repeat and are correct in the previous RO. This is equivalent to just 2 consecutive correct responses instead of 4. On the other hand, the second instance is considered to have shown mastery at the 9th RO instead of the 3rd or the 7th RO because questions 372 and 401 are correctly answered in the previous attempts as well.

3.2.3 Validating Identified Skills. To validate the identified skills, we make use of a Learning Curve through the Error Rate computation. Then, we use a software tool called Tableau to visualize the instances per skill and see whether instances of students are still committing several consecutive incorrect responses. This is to point out questions with frequent incorrect responses among students for another skill review.

3.2.3.1 Generating Learning Curve per Skill. A learning curve is a graphical representation of the performance of students on a particular skill. It depicts the relationship of the error rate on the y axis to the number of practice opportunities on the x axis. Ideally, the error rate of students at the first practice opportunity begins at 100% as all students are expected to commit mistake in the first attempt, and gradually decreases approaching zero (Figure 2 – green curve).

When a learning curve fluctuates and does not decrease, it means that the students continue to commit mistakes despite opportunities to practice the skill (Figure 2 – red curve). When

this happens, the questions have to be reviewed again as the current skill might be mislabeled.

In this first validation method, the learning curve is generated through the computation of students' Error Rate (ER) per Response Opportunity (RO). The ER is computed using the following formula:

Error_rate = (Total Incorrect Responses / Total Responses) * 100

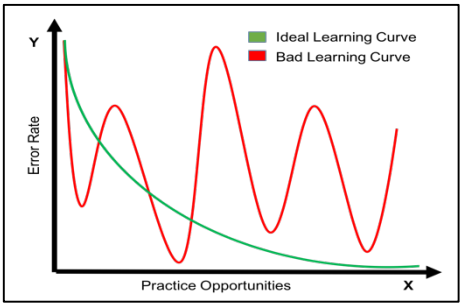


Figure 2 –Learning Curves that represents a good and a bad performance

However, there are considerations in the computation of ER in order to put it into the context of the system. First, only the instances with mastery are considered, otherwise, the learning curve fluctuates and does not decrease. Second, the responses after mastery are ignored.

3.2.3.2 Visualizing Instances. The instances with mastery are fed as input to the Tableau for visualization. Visualizing the instances in Tableau allows us to immediately see whether some students are still committing several consecutive mistakes per skill. In the case that the instances are seen with several consecutive mistakes in Tableau, then the questions answered by the students with a high number of incorrect responses are reviewed again.

3.2.4 Computing the Cumulative Percentage of Students with Mastery (PSM). In this study, the cumulative Percentage of Students with Mastery (PSM) per skill are calculated to answer the first research question using the following formula.

PSM = (Total number students with mastery / Total number of students) * 100 + current_percentage

In theory, considering the criteria for mastery, the graph for the percentage of students who master the skill gradually rises from 0 to 100 percent after the 3rd attempt. No one should show mastery on the first and second attempts (Figure 3 – green curve).

The computation of PSM is performed for every identified skill. Every PSM value is plotted in a graph to see whether the graph follows the same graph in the theory of PSM or whether the graph eventually flattens which means that the students are possibly wheel-spinning on the identified skill (Figure 3 – blue curve). In the case, that the graph never flattens and eventually reaches 100 percent, then there are no instances of wheel-spinning, otherwise, wheel-spinning exists.

3.3 Identifying Skills with Instances of Wheel-spinning

Students who fail to master the skill by the 10th RO or attempt are regarded as wheel-spinning. This is the same in the past studies [10][8][9][24][23] where students are identified as wheel-spinning after the 10th attempt. However, it is possible that students make several consecutive mistakes. Thus, in this study, the students who make at least 4 consecutive incorrect responses with no mastery are also considered to be wheel-spinning regardless of whether they reach the 10th RO or not.

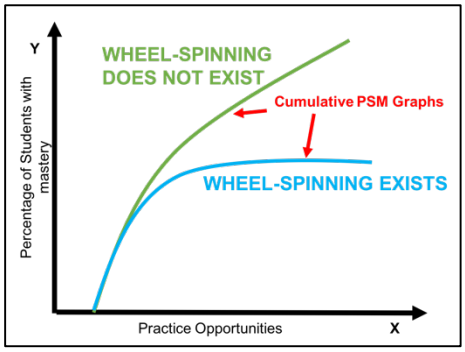


Figure 3 – Graphs with and without Wheel-spinning

3.3.1 Retrieving Instances with more instances of Wheel-Spinning. Upon reaching this stage, the correct skills involved are already identified. A number of steps are performed to identify the skills with more instances of wheel-spinning:

1. Instances of "Wheel-spinning" are identified, removing those instances of mastery and indeterminate instances.
2. Skills involved in these WS instances are identified.
3. The number of instances per identified skill are counted.
4. Skills are sorted in descending order according to the number of the instances of WS, putting the skill with the highest count on top.

4 Results and Discussion

4.1 Existence of Wheel-spinning

We were able to do few iterations of the prerequisite stages for the computation of the cumulative PSM as some of the tagged skills on the previous iterations failed to pass the skill validation and as students were also committing several mistakes.

4.1.1 Initial Iteration. We tried to make the identified or tagged skills more specific than the default skills in MGS. We identified the unique skills from both CBSUA and ADMU questions (Table 2), and counted the number of instances per skill per school group. Considering only those skills with instances of mastery, we also identified the instances of mastery per skill in order to generate the learning curve for skill validation. (Figure 4A and 4B).

Table 2 Number of Identified Skills per School Group during Initial Iteration

	CBSUA	ADMU	Unique Skills
Skill Count	19	16	24

There were 19 identified skills from CBSUA questions while 16 skills were identified from ADMU questions. Since there were skills which were the same from both Universities, we counted only those that were unique skills with a total of 24. The total number of instances generated were 814; 456 from CBSUA and 358 from ADMU.

The graph shows that some of the skills were present in both University. This is because some questions are similar or identical. On the other hand, some skills were present only from either of the school groups. For instance, "Switch" skill was present only in CBSUA because it is not available in Python programming language used by ADMU participants.

Among the identified skills, there were 11 skills with instances of mastery and 13 skills with no instances of mastery. The questions tagged with the 13 skills having no instances of mastery were reviewed again for the re-identification of the skills involved and used them in the next iterations.

The graphs in Figure 4A and 4B show that some of the learning curves from both Universities are fluctuating which means that the error rates are not decreasing (e.g. both learning curves for *Tracing* skill of ADMU and CBSUA). We have also reviewed the instances of these skills in Tableau, and found out that most students struggle before achieving mastery. In addition, it is surprising to see that the students have made several mistakes even after achieving mastery, which also contributes to the fluctuations in the learning curves. Thus, we have decided to re-identify the skills and do more iterations.

4.1.2 Final Iteration. Prior to the final iteration, a number of iterations were conducted until the right skills were identified. Since the tagged skills in previous iterations failed to pass the skill validation, it was necessary to re-identify the skills involved in all the questions, only this time the identified skills were more specific. With the new identified skills, 662 instances from CBSUA and 950 instances from ADMU were generated.

Table 3 presents the number of identified skills per school group and the total number of unique skills. The instances of these 61 unique skills are used to generate the learning curves.

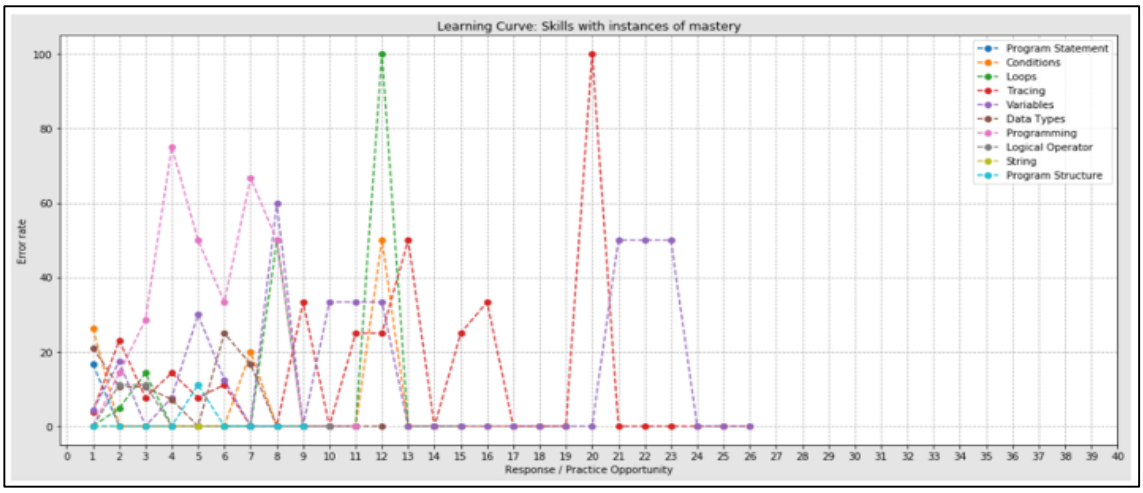


Figure 4A ADMU – Learning Curves of the Identified Skills – First Iteration

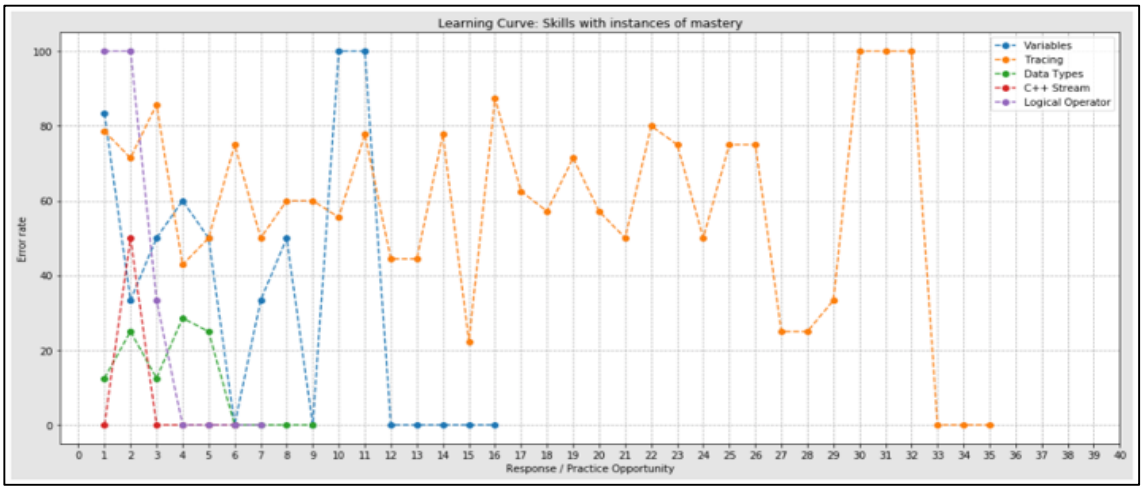


Figure 4B CBSUA – Learning Curves of the Identified Skills – First Iteration

Table 3 Number of Identified Skills per School Group during Final Iteration

	CBSUA	ADMU	Unique Skills
Skill Count	43	38	61

Only 14 skills had instances of mastery while the remaining 47 identified skills had no instances of mastery. We reviewed the instances per skill and found out that there were several indeterminate instances in those skills or instances that cannot be identified whether they were wheel-spinning or showing mastery.

The learning curves for CBSUA's skills (Figure 5A) show the decreasing error rates with fewer fluctuations compared to the learning curves in the previous iterations (i.e. Figure 4B). Thus, we settle for these identified skills.

On the other hand, the learning curves for ADMU's identified skills (Figure 5B) show a bit of fluctuations compared to CBSUA's. However, all of them eventually falls down to zero error rate and never reaches a 100% error rate compare to the previous iterations (i.e. Figure 4A).

We were doubtful of the skills because of the fluctuations on the learning curves. So, we reviewed the questions in ADMU and found out that the skills that they were tagged were not mislabeled and most of the questions were similar in CBSUA's. In addition, further iteration with a more specific set of skills was not ideal as it will just generate more indeterminate instances, leaving us with fewer number of instances. Thus, we also stick with these skills.



Figure 5A CBSUA – Learning Curve of the Identified Skills – Final Iteration

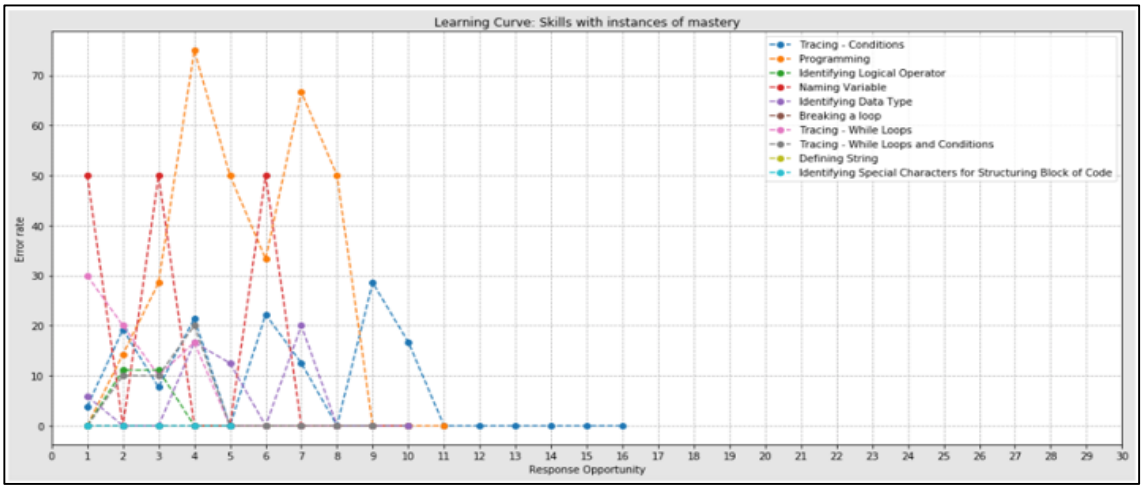


Figure 5B ADMU – Learning Curve of the Identified Skills – Final Iteration

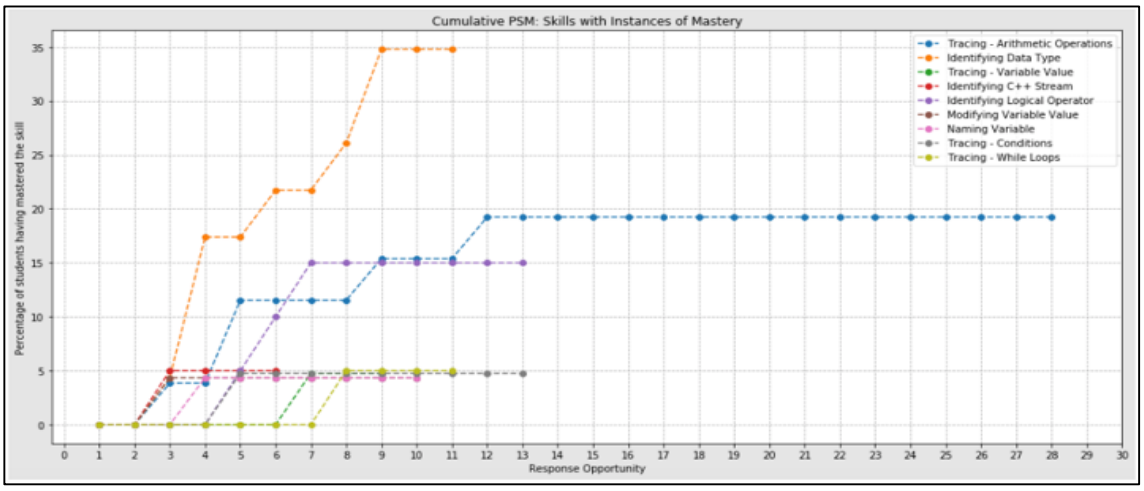


Figure 6A CBSUA – Cumulative PSM per Skill (9 skills)

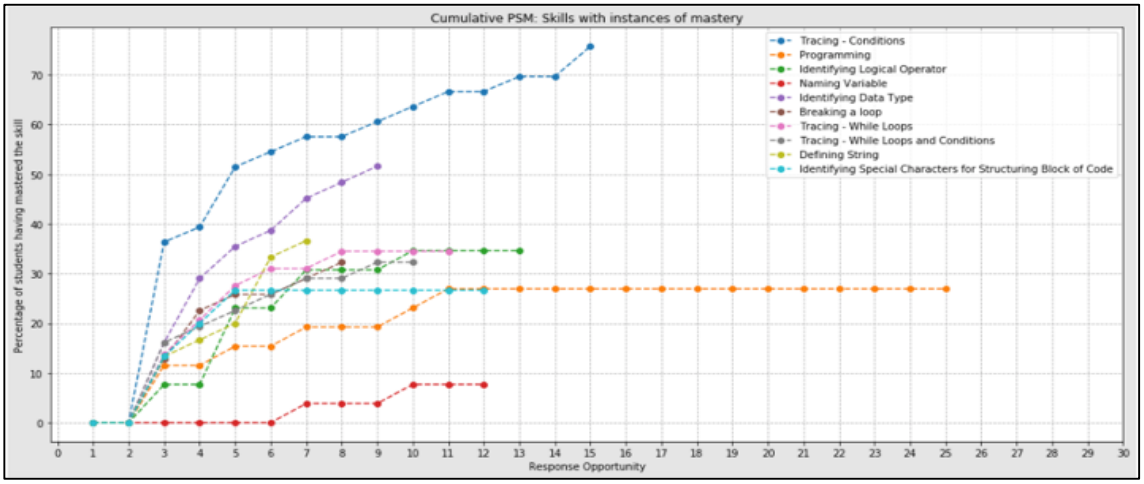


Figure 6B ADMU– Cumulative PSM per Skill (10 skills)

4.1.3 Cumulative Percentage of Students with Mastery (PSM). Using the 61 identified skills, 35 skills were composed only of indeterminate instances, thus, they were removed. However, out of the remaining 26 skills, only 14 unique skills from both school groups had instances of mastery. We used the 14 remaining skills to compute for the cumulative PSM per skill. We computed the cumulative PSM per school group as presented in Figures 6A and 6B.

Both graphs for CBSUA and ADMU groups show that wheel-spinning exists in a novice programming context as both eventually flatten multiple times and do not reach the 100% PSM for every identified skill. CBSUA group, though, show wheel-spinning occurrences in most of the identified skills compared to the AMDU group. Furthermore, there are fewer skills with instances of mastery in CBSUA than ADMU.

4.2 Skills with Instances of Wheel-spinning

We were able to generate 187 instances using the 14 skills; 132 instances of mastery and 45 instances of wheel-spinning. Out of the 14 skills, only 9 of them had instances of wheel-spinning. Then, we identified the skills of those 45 instances and counted the number of instances per skill (Figure 7).

It shows that *Programming* skill has the highest percentage of wheel-spinning instances with over 30%. It also appears that there are more skills with instances of wheel-spinning that falls under program comprehension with the average percentage of 10% such as *Tracing in Arithmetic Operations, Tracing a Variable Value, Tracing in While Loops,* and *Tracing in Condition Statements* (i.e. if statements). There are also 10% of the instances of wheel-spinning on skills concerning Variables such as *Modifying Variable Value, Naming Variable,* and *Tracing a Variable Value.* Moreover, there are skills with fewer instances of wheel-spinning such as the skills about condition statements, data types, and logical operators.

5 Conclusion

In this paper, we attempted to determine the existence of wheel-spinning in a novice programming context within a Meaningful Gamification System (MGS). We also tried to identify on which skills do novice programmers show more instances of wheel-spinning. An iterative prerequisite stages were performed prior to computing the cumulative Percentage of Students with Mastery (PSM) which helped this study determine the existence of wheel-spinning. These prerequisite stages were the identification of skills, identification of instances of mastery per skill, and validation of skill.

There were a few number of iterations of the prerequisite stages as the identified skills in previous iterations failed to pass the skill validation. In the initial iteration, we made the skills more specific than the default skills in MGS (e.g. Variables, Tracing). However, the identified skills were becoming more specific as further iterations were done. In the final iteration, we were able to identify 61 unique skills from the questions given to ADMU and CBSUA groups. But, only 14 skills had instances of mastery while the remaining 47 skills had no instances of mastery. Thus, these 47 skills were excluded in the computation of cumulative PSM. Based on the 14 skills, we were able to determine that wheel-spinning exists in a novice programming context.

Out of 14 skills, there were 9 skills identified with instances of wheel-spinning. We learned what skills do students had difficulties by identifying which of the tagged skills they wheel-spin. Most of the students struggled and fail to master the skill in program generation than in program comprehension and conceptual skills.

Although we were only able to identify few students who wheel-spin in this study, we would like to emphasize that wheel-spinning exists in these novice programmers and these instances of students who wheel-spin should be addressed in order to further understand them as they need proper intervention on the skills that they have difficulties of mastering.

Doing a number of iterations of the prerequisite stages allows us to see the importance of identifying what skill is exactly involved per question or problem that the student is trying to answer. Because if ever a certain skill is mistakenly mislabeled, we could make incorrect conclusions that the students have mastered the skill when in reality, they are mastering more than just one skill on the problems that they are trying to solve. On the other hand, we could say that they have failed to master the skill, when in fact they have not. As a result, we could give interventions that they don't really need as they have mastered the skill already, or we might fail to provide the proper interventions that they need as we fail to identify which skill they are actually struggling to learn. Therefore, if we want our students to learn a particular programming skill and evaluate student's mastery on that skill, it might be ideal to make questions specific, involving just the skill that we want them to learn.

To sum up, in this work, we were able to determine the existence of wheel-spinning using cumulative PSM. Prior to computing the cumulative PSM, we used learning curves to validate the identified skills with instances of mastery. Finally, we identified which skills had the highest to lowest number of student instances who wheel-spin.

6 Future Work

In the future work, we plan to include the skills with no instances of mastery. However, another skill validation method should be used to see whether those skills without instances of mastery are the right skills since a learning curve as the method to validate such skills are not applicable. Then, we will identify whether wheel-spinning exists on those skills. Also, we plan to model wheel-spinning using the same data set and identify what factors have strong correlation in order to understand students who wheel-spin – to also figure out what ideal interventions are possible. Lastly, we will compare how wheel-spinning from a programming context differs to a non-programming context.

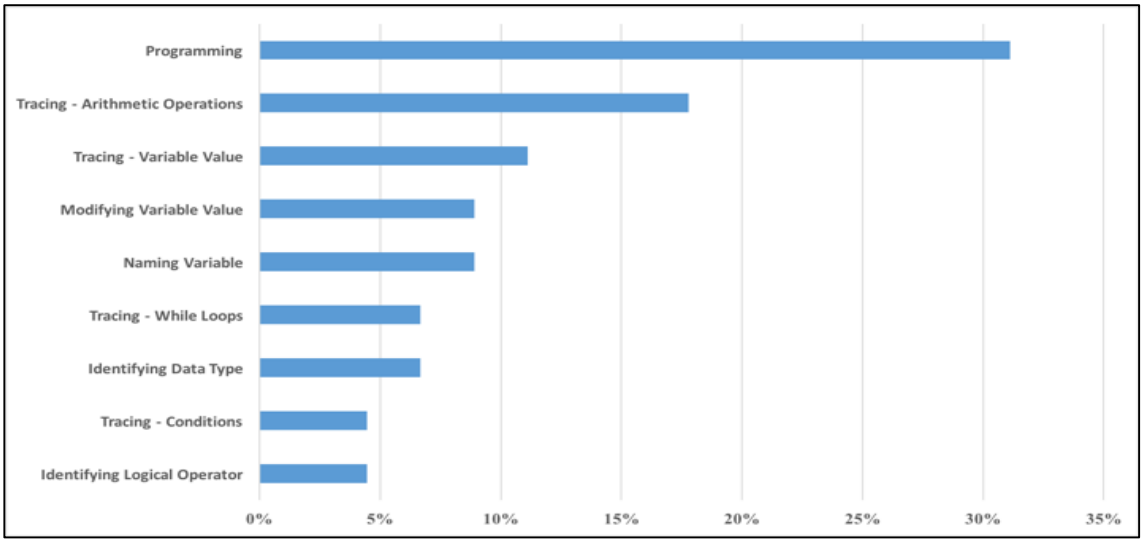


Figure 7 Number Instances of Wheel-spinning per Skill

ACKNOWLEDGMENTS

We thank the Commission on Higher Education (CHED) for granting scholarship to the student researcher. Also, we would like to thank the Ateneo Laboratory for Learning Sciences (ALLS), Ateneo de Manila University (ADMU) for supporting this study. Lastly, we express our gratitude to those people who extended their help in making this study a success.

REFERENCES

[1] B Xie, G.L Nelson, and A.J Ko. 2018. An Explicit Strategy to Scaffold Novice Program Tracing. In Proceedings of the 49th ACM Technical Symposium on Computer Science Education (SIGCSE '18). ACM, New York, NY, USA, 344-349. DOI: https://doi.org/10.1145/3159450.3159527

[2] B.S Alqadi. and J.I Maletic. 2017. An Empirical Study of Debugging Patterns Among Novices Programmers. In Proceedings of the 2017 ACM SIGCSE Technical Symposium on Computer Science Education (SIGCSE '17). ACM, New York, NY, USA, 15-20. DOI: https://doi.org/10.1145/3017680.3017761

[3] C Watson, F.W Li, and J.L Godwin. 2013, July. Predicting performance in an introductory programming course by logging and analyzing student programming behavior. In 2013 IEEE 13th International Conference on Advanced Learning Technologies (ICALT), 319-323, DOI: https://doi.org/10.1109/ICALT.2013.99

[4] C.L.S Tablatin, and M.M.T Rodrigo. 2017. The Relationship of Compilation Behavior Metrics and Student Performance in Introductory Programming Course. In Proceedings of 15th National Conference on Information Technology Education (NCITE 2017), Leyte Normal University, Tacloban, Leyte , October 2017 (NCITE), 9 pages.

[5] E Lahtinen, K Ala-Mutka, and H Järvinen. 2005. A study of the difficulties of novice programmers. In Proceedings of the 10th annual SIGCSE conference on Innovation and technology in computer science education (ITiCSE '05). ACM, New York, NY, USA, 14-18. DOI: https://doi.org/10.1145/1067445.1067453

[6] E.S Tabanao, M.M.T Rodrigo, and M.C Jadud. 2008. Identifying at-risk novice Java programmers through the analysis of online protocols. Philippine Science Congress (2008).

[7] E.S Tabanao, M.M.T Rodrigo, and M.C Jadud. 2011. Predicting at-risk novice Java programmers through the analysis of online protocols. In Proceedings of the seventh international workshop on Computing education research (ICER '11). ACM, New York, NY, USA, 85-92. DOI: http://dx.doi.org/10.1145/2016911.2016930

[8] H Wan, and J.B Beck. 2015. Considering the influence of prerequisite performance on wheel spinning. International Educational Data Mining Society (June 2015)

[9] J.B Beck and M.M.T Rodrigo. 2014. Understanding Wheel Spinning in the Context of Affective Factors. International Journal of People-Oriented Programming3, 2 (2014), 75–90. DOI: https://doi.org/10.1007/978-3-319-07221-0_20

[10] J.E Beck and Y Gong. 2013. Wheel-Spinning: Students Who Fail to Master a Skill. Lecture Notes in Computer Science Artificial Intelligence in Education (2013), 431–440. DOI: http://dx.doi.org/10.1007/978-3-642-39112-5_44

[11] J.L Agapito and M.M.T Rodrigo. 2017. Designing an Intervention for Novice Programmers Based On Meaningful Gamification: An Expert Evaluation. In Proceedings of the 25th International Conference on Computers in Education (ICCE 2017).

[12] J.L Agapito and M.M.T Rodrigo. 2018. An Investigation of the Impact of Gamification on Novice Programmers' achievement and learning experience. Ph.D. Dissertation. Ateneo de Manila University.

[13] K Rivers and K.R Koedinger. 2017. Data-driven hint generation in vast solution spaces: a self-improving python programming tutor. International Journal of Artificial Intelligence in Education, 27(1), 37-64, DOI: 10.1007/s40593-015-0070-z

[14] M.C Jadud. 2006. An exploration of novice compilation behaviour in BlueJ (Doctoral dissertation, University of Kent).

[15] M.M Mhashi and A.M Alakeel. 2013. Difficulties facing students in learning computer programming skills at Tabuk University. In Proceedings of the 12th International Conference on Education and Educational Technology (EDU' 13), Iwate, Japan, 15-24.

[16] M.M.T Rodrigo, E Tabanao, M.B.E Lahoz, and M.C Jadud. 2009. Analyzing online protocols to characterize novice java programmers. Philippine Journal of Science, 138(2), 177-190.

[17] N Matsuda, S Chandrasekaran, and J Stamper. 2016. How quickly can wheel spinning be detected? International Conference on Educational Data Mining (EDM) 2016, 607-608.

[18] P.K Sevella, Y Lee, and J Yang. 2013. Determining The Barriers Faced By Novice Programmers (Doctoral dissertation, Texas A & M University-Kingsville). International Journal of Software Engineering (IJSE), 10-12.

[19] R Cardell-Oliver. 2011. How can software metrics help novice programmers ? In Proceedings of the Thirteenth Australasian Computing Education Conference - Volume 114 (ACE '11), John Hamer and Michael de Raadt (Eds.), Vol. 114. Australian Computer Society, Inc., Darlinghurst, Australia, Australia, 55-62.

[20] S Nicholson. 2015. A recipe for meaningful gamification. In Gamification in education and business. Springer, Cham, 1-20.

[21] T Blok and A Fehnker. 2017. Automated program analysis for novice programmers. arXiv preprint arXiv:1710.00163.

[22] T.D Palaoag, M.M.T Rodrigo, J.M.L. Andres, J.M.A.L Andres, and J.E Beck . 2016. Wheel-Spinning in a Game-Based Learning Environment for Physics. \xa0Intelligent Tutoring Systems Lecture Notes in Computer Science(2016), 234–239. DOI: http://dx.doi.org/10.1007/978-3-319-39583-8_23

[23] Y Gong and J.E Beck. 2015. Towards Detecting Wheel-Spinning: Future Failure in Mastery Learning. In Proceedings of the Second (2015) ACM Conference on Learning @ Scale (L@S '15). ACM, New York, NY, USA, 67-74. DOI: http://dx.doi.org/10.1145/2724660.2724673

[24] Y Gong, Y Wang, and J. Beck. 2016. How long must we spin our wheels? Analysis of student time and classifier inaccuracy. Student modeling from different aspects, 32-38.

CSP SIG on Information and Computing Education (SPICE)

SPICE Board Members, School Year 2018-2019

Jaime D.L. Caro
University of the Philippines, Diliman, Quezon City

Eric John G. Emberda
University of Immaculate Conception, Davao City

Gregg Victor D. Gabison
University of San Jose – Recoletos, Cebu City

Randy S. Gamboa
University of Southeastern Philippines, Davao City

Dave E. Marcial
Silliman University, Dumaguete City

Ramon L. Rodriguez
National University, City of Manila

Cherry Lyn Sta Romana
Cebu Institute of Technology – University, Cebu City

John Peter Abraham Ruero
Holy Angel University, Angeles City

Allan A. Sioson
Cobena Business Analytics and Strategy, Inc., BGC, Taguig City



COMPUTING SOCIETY OF THE PHILIPPINES

Board of Directors, School Year 2018-2019

President: **Rachel Edita O. Roxas, PhD**
National University, City of Manila

Vice President: **Randy S. Gamboa, PhD**
University of Southeastern Philippines, Davao City

Secretary: **Rafael A. Cabredo, PhD**
De La Salle University, Taft, City of Manila

Treasurer: **Nathaniel A. Oco**
National University, City of Manila

Asst. Treasurer: **Marrick C. Neri, PhD**
University of the Philippines, Diliman, Quezon City

Auditor: **Charibeth Cheng**
De La Salle University, Taft, City of Manila

Henry N. Adorna, PhD
University of the Philippines, Diliman, Quezon City

Proceso L. Fernandez, Jr., PhD
Ateneo de Manila University, Loyola Heights, Quezon City

Mary Jane Sabellano
University of San Carlos, Cebu City